Society in Silico

Simulating futures to build the one we want.

By Max Ghenis. Listen edition, generated from the draft manuscript — citations, tables, and editorial marks are omitted; the web edition at societyinsilico.com carries them.

Preface

In 2019, I was trying to understand how a universal basic income would affect poverty in the United States. Not the vibes-level version—"it would help a lot of people" or "it would cost too much"—but the precise version. How many families would cross the poverty line? Which families? What would it cost, net of reduced spending on existing programs? A $1,000-a-month basic income runs to roughly $3 trillion a year; the interesting questions all live in how you pay for it, and in how it interacts with the Earned Income Tax Credit, SNAP, Medicaid, and housing assistance.

I couldn't find the answer. Not because no one had modeled it, but because the models that could answer the question were locked inside government agencies and think tanks, built on confidential data, and inaccessible to anyone outside those institutions. The Congressional Budget Office had models. The Tax Policy Center had models. The Joint Committee on Taxation had models. None of them were available for me to query with my specific question.

So I started building. First a scrappy research outfit called the UBI Center, running analyses on existing open-source tools. Then, with Nikhil Woodruff, PolicyEngine—an attempt to make the same kind of policy simulation available to anyone with a web browser. (Footnote: A disclosure that is also, I think, part of the story: as of July 2026, Nikhil serves in the UK government at 10 Downing Street. Where this book discusses UK institutions evaluating tools he helped build, read it with that in mind.) Enter your household details, change a policy parameter, see what happens. No expertise required. No paywall. No waiting for an analyst to run the numbers.

That work led to a series of questions I hadn't anticipated. If you can encode tax law as code, what else can you encode? If AI systems need accurate financial calculations and can't do them on their own, who builds the tools they call? If simulation can tell you what a policy does to people's wallets, can it also tell you what people think about it? And if you can simulate policies, opinions, and values—what does that mean for how society governs itself?

This book is my attempt to trace those questions from their origins to their implications.

What this book is

This is a narrative about tools and the people who build them. It begins in 1957 with Guy Orcutt, an economist-engineer who imagined simulating society household by household, decades before the computers existed to do it. It moves through six decades of institutional microsimulation—mainframes at the Urban Institute, proprietary models at the Congressional Budget Office, the quiet revolution of open-source policy tools in Europe and the United States. It arrives at the present moment, where AI agents write and verify the code that once took institutions years to build, where open simulation is becoming public infrastructure, and where the boundaries of what we can simulate are expanding in directions that raise hard questions about democracy, values, and what it means to model a society.

Along the way, I describe projects I've built or helped build—PolicyEngine, HiveSight, Democrasim, the Axiom Foundation, and the Thesis Institute. I try to be honest about what works, what doesn't, and what remains aspiration rather than accomplishment. The honest accounting turned out to be a moving target. A chapter drafted in 2025 as a sketch of infrastructure that didn't yet exist had to be rewritten in 2026 as a field report on infrastructure that does—law encoded by AI agents and checked, line by line, against the models governments already trust. Other parts remain exactly as speculative as they were. I label the difference throughout, because the distinction between "this works," "this is early," and "this might work someday" is the book's whole currency—and the speed at which things moved between those categories is itself part of the story.

What this book is not

This is not a textbook. I won't teach you microsimulation methodology, and the technical details I include are meant to illuminate ideas, not to serve as documentation. If you want to learn how to build a tax-benefit model, the references will point you to better sources.

This is not a memoir, though my own path through this work provides the connective tissue. I include personal experience where it illuminates the ideas and omit it where it doesn't. The story of microsimulation is bigger than any one person's involvement.

This is not a manifesto. I have views—about open source, about democratic access to information, about how AI should handle policy questions—but I try to present them alongside the strongest counterarguments I can find. The chapters on simulated opinion and simulated democracy, in particular, are structured around the tensions and limitations of the ideas, not as advocacy.

And this is not finished. I'm writing while the work is ongoing. The Axiom Foundation launches publicly this summer; the Thesis Institute follows in the fall; the forecasts its scoreboard publishes will take years to grade themselves against reality, and as of this writing not one has resolved. The research directions in the later chapters—simulating opinion, democracy, values—are genuinely speculative, and I say so where they are. Readers should expect that

some of what I describe will look different in two years. Between the first draft and this one, it already does.

Who this book is for

I've written for three readers, and any one lens is enough.

Policy people who want to understand how computational tools are changing policy analysis. You don't need to know how to code. I explain the technical concepts through stories and examples, and the details that matter—how a tax-benefit model works, what microsimulation actually does, why AI can't just learn tax law—are explained in plain language.

Technologists who are building AI systems, fintech applications, or data infrastructure and want to understand the policy domain. You'll find the architecture discussions relevant, and the policy context will help you understand why this domain is harder than it looks.

Curious readers who follow the intersection of technology and governance. If you've wondered why AI gets tax questions wrong, why government budget models are secret, or what it would mean to simulate a society—this book is an attempt to answer those questions accessibly.

How to read this book

The book runs in five parts, and they build on each other but tolerate skipping.

Part I: The closed stack is history—how government built an analytic machinery the public couldn't see, from Orcutt's 1957 vision through the tax model wars, ending with the question that disciplines everything after it: how would you know whether any of these models is right?

Part II: The open engine is the PolicyEngine story—proof that the closed stack could be rebuilt in public, what the engine shows one household and a whole country, and the three ingredients every such model must solve.

Part III: The agent turn is the new heart of the book: why AI systems need deterministic policy tools, how AI agents came to write verified law-as-code at scale, how you trust machine-encoded law, and what happened when the method reached countries that never had public models at all.

Part IV: The prediction pole crosses from calculation to forecasting—the uncertainty that point estimates hide, the scoreboard being built to price it honestly, and what large language models actually add to opinion research once you benchmark them.

Part V: The horizon is the speculative edge—simulated democracy, simulated values—clearly labeled, and the return to the fork in the road the introduction opens.

If you're impatient, read the introduction, then Part III, and dip into Part V for the ideas that interest you. If you have time, the history in Part I makes everything else richer.

I started this project because I believed—and still believe—that the tools we use to understand society shape the decisions we make about it. When those tools are locked inside institutions, the understanding is locked too. When they're open, transparent, and accessible, more people can participate in the conversation about what policies we should adopt and why.

That's an optimistic view. Whether it survives contact with the evidence is something you can judge for yourself.

Max Ghenis Washington, DC, 2026

Introduction: The model and the world

"I don't predict the future. I create it."

That line comes from Engerraund Serac, the antagonist of Westworld's third season. After watching a thermonuclear incident destroy Paris, Serac built an AI called Rehoboam that predicted individual human lives—when you'd get sick, lose your job, die. The system didn't just forecast; it manipulated society to make its predictions come true. People who deviated from their predicted paths got flagged for "reconditioning."

The show's premise: humans are "just a brief algorithm," reducible to code. The horror: one man controlling that algorithm without anyone else knowing.

When I watched this in 2020, I recognized the technology. I'd been building microsimulation systems since 2018—computational models that predict how tax policies affect households, that simulate entire economies with millions of synthetic people. The Westworld writers had taken the same tools and followed them to their darkest conclusion.

They'd also revealed a choice we're making right now.

The fictional version is extreme—a single AI controlling human destiny. But the underlying dynamic is already real. Governments use algorithmic risk scores to allocate child welfare investigations, set bail amounts, and flag potential fraud in benefit claims. Insurance companies use predictive models you can't inspect to set your premiums. Credit agencies reduce your financial life to a three-digit number using methods they won't fully explain. Each of these systems makes predictions about individuals based on computational models of human behavior—and each operates with minimal transparency about its assumptions, its uncertainty, or its error rates.

Serac's system was closed: he decided what "optimal" meant, and everyone else lived inside his model without consent. But computational models of society don't have to work that way. What if anyone could query the model? Challenge its assumptions? Propose alternatives?

What if simulation became public infrastructure for democratic deliberation, not a tool for autocratic control?

That's the fork in the road. That's what this book is about.

The AI can't do your taxes

In July 2025, Column Tax released TaxCalcBench—a benchmark testing whether AI could correctly compute complete federal tax returns. They gave frontier models everything they needed: the tax rules, the taxpayer's information, and as much time to think as they wanted. The best-performing model, Gemini 2.5 Pro, got fewer than one in three returns right. Claude Opus 4 managed 27%. These are the most capable AI systems in the world, and they can't reliably do what a $50 copy of TurboTax does without breaking a sweat.

The errors weren't random. Models used percentage-based bracket calculations instead of the IRS tax tables, producing returns that were off by $3-5 each time. They miscalculated credit eligibility. They stumbled on supplementary forms. The pattern was consistent across every model tested.

This wasn't a surprise to researchers who'd been tracking the problem. In 2023, researchers at Johns Hopkins and the University of Maryland found GPT-4 answered only 67% of true/false tax questions correctly—better than chance, but nowhere near reliable enough for financial decisions. And the limitation has survived every model generation since. By mid-2026, PolicyEngine's own benchmark—PolicyBench, which scores some twenty frontier and open-weight models on complete household tax-and-benefit calculations drawn from realistic family circumstances—found the best model still getting roughly one in six households wrong by more than a dollar, and most models missing a quarter or more. The errors aren't rounding noise. They're family-level mistakes: the wrong Medicaid eligibility, the wrong SNAP amount, a credit granted to a household that doesn't qualify.

Language models can generate fluent explanations of tax law. They can summarize complex regulations. What they can't do is *calculate*—apply specific rules to specific circumstances and produce a number that's guaranteed to be correct. Tax law changes every year. Fifty states have different rules. Eligibility for benefits depends on dozens of interacting variables. The combinatorial complexity exceeds what pretraining can memorize.

The solution isn't better training. It's better tools—deterministic, auditable computational tools that AI systems can call when they need exact answers.

This is the central technical insight of the book: AI needs tools, and the tools that matter most for policy questions are microsimulation models. The pattern is familiar from other domains. AI systems don't memorize multiplication tables—they call calculators. They don't learn orbital mechanics from scratch—they call physics engines. For the same reasons, they

shouldn't try to memorize tax law. They should call a tax engine that encodes the rules exactly, traces each calculation to statute, and produces guaranteed-correct results.

The trouble is that no comprehensive tax-and-benefit engine exists as unified public infrastructure. Avalara handles sales tax. ADP handles payroll. TurboTax handles filing. But nothing answers the full-stack question that matters for policy: given this specific household in this specific state, what are all their taxes and all their benefits, how do they interact, and what happens when you change the rules? That question—the question microsimulation was built to answer—is what connects a 1957 academic paper to a 2025 AI benchmark failure.

Why now

Twenty years ago, if you wanted to know how a tax reform would affect American families, you had two options: read the CBO score (a single number for the nation) or hire a consulting firm to run proprietary models. Ten years ago, open-source tools like Tax-Calculator and EUROMOD existed, but using them required programming expertise and statistical training. Five years ago, PolicyEngine put microsimulation in a web browser, but the audience was still mostly policy professionals.

Three trends are now converging to make this moment different from any previous era of policy analysis.

AI systems are becoming the primary interface for information. When someone asks "how much would I get from the Child Tax Credit?" they increasingly ask an AI assistant rather than navigating IRS.gov. If the assistant hallucinates an answer, the person may make financial decisions based on fiction. The infrastructure that AI systems call when they need accurate answers doesn't exist as a unified, production-ready service. Each company building AI assistants has to solve the tax-and-benefits problem from scratch—or get it wrong.

Policy complexity is accelerating. The US tax code has roughly doubled in length since 1985. The interaction between federal taxes, state taxes, and means-tested benefit programs creates a combinatorial explosion that no individual can navigate without computational tools. A single parent earning $35,000 in Texas faces a different effective tax rate than the same parent in New York—not by a small amount, but potentially by thousands of dollars. The complexity isn't a bug in the system; it's the accumulated result of decades of legislation, each layer addressing a real problem while creating new interactions with existing programs.

Open-source infrastructure has matured. A decade ago, the idea of encoding an entire country's tax-benefit system as publicly inspectable code was aspirational. Today, PolicyEngine covers US federal taxes plus all fifty states, the UK, and Canada—maintained by more than 100 open-source contributors. EUROMOD covers twenty-seven EU countries. OpenFisca has been deployed on four continents. The question is no longer whether open policy simulation is

possible. It's whether it can become the shared infrastructure that AI systems, governments, and citizens all rely on.

These trends create both opportunity and risk. The opportunity: a world where anyone can understand how policy affects them, where AI assistants give accurate answers about taxes and benefits, where policymakers can model proposals against a shared baseline that everyone trusts because everyone can inspect it. The risk: a world where opaque models make decisions about benefits and creditworthiness, where AI systems confidently give wrong answers about financial matters, where the tools of policy analysis remain locked inside institutions while the public debates with anecdotes.

What simulation reveals

Consider a single parent in Ohio earning $45,000 with two children. She's trying to decide whether to accept a promotion that would raise her salary to $55,000.

Without calculation, the answer seems obvious—more money is better. But the tax-benefit system creates a labyrinth of interactions. As her earnings rise toward $55,000, the Earned Income Tax Credit she still collected at $45,000 phases out entirely, her SNAP benefits shrink, and—depending on her state and childcare arrangement—a subsidy can vanish at a hard threshold. The share of each additional dollar she actually keeps can stay strikingly low across that range.

For families who hit one of those hard thresholds, the losses can bunch tightly enough that earning more leaves them with less—a true benefit cliff. This isn't a theoretical curiosity. It affects millions of families every year. And you can't see it without running the numbers through a model that captures the full interaction of taxes and benefits.

That's what microsimulation does. It calculates how policies affect specific households by applying the actual rules—every bracket, every phase-out, every eligibility test—to specific circumstances. Then it aggregates across a representative sample of the population to estimate what a policy does to everyone.

When Congress debates expanding the Child Tax Credit, microsimulation answers: How much would it cost? Which families would benefit? Would it reduce poverty? By how much? These aren't guesses. They're calculations—the product of applying the proposed rules to millions of synthetic households drawn from actual survey data.

The same logic applies across the Atlantic. When the UK Chancellor announces changes to Universal Credit taper rates, a microsimulation model can show that a single parent working 25 hours per week at minimum wage keeps an additional £1,000 per year, while a two-earner couple sees a smaller gain. When Scotland sets different income tax rates from the rest of the

UK, simulation reveals who wins and who loses across the income distribution. The tool is country-agnostic; the rules are country-specific.

The idea goes back to 1957, when an economist named Guy Orcutt proposed modeling society from the bottom up—simulating individual households rather than relying on aggregate equations. His insight was that you could learn more about how the economy works by tracking what happens to millions of individual families than by studying national averages. A 2% change in GDP tells you nothing about whether the people who need help are getting it. But simulate the change household by household—applying actual tax rules to actual income data—and you see what aggregate numbers hide: the family that falls through the cracks, the benefit cliff that punishes a raise, the interaction between programs that no single-program analysis reveals.

For decades, the tools to realize Orcutt's vision were locked inside government agencies: the Congressional Budget Office, the Joint Committee on Taxation, the Treasury Department. If you wanted to know how a policy would affect you, you had to trust their numbers. You couldn't check their work.

That's changing. Open-source microsimulation models now make it possible for anyone—researchers, journalists, advocates, curious citizens—to run the same kinds of analyses that once required government resources. The code is public. The methodology is inspectable. The results are reproducible.

The stakes

We don't have Rehoboam. But we're not starting from zero.

Governments already use predictive models to allocate benefits, assess fraud risk, and shape policy. Insurance companies price your premiums with algorithms you can't inspect. Banks decide your creditworthiness with models they won't explain. And AI assistants—the primary interface through which many people will encounter policy information—get tax questions wrong a third of the time.

These models exist. The question is who controls them.

Who builds them? Closed institutions or open communities?

Who can access them? Only the powerful or everyone?

What are they for? Optimization or understanding?

The gap between policy debates and policy analysis is vast. Political arguments run on emotion and tribal loyalty. Policy analysis runs on computation and precision. In a typical election, candidates propose tax plans that would affect millions of families by thousands of dollars

each—and most voters have no way to know whether they'd come out ahead or behind. They rely on campaign claims, pundit interpretations, and gut feeling. The tools to calculate the actual impact exist, but they're not available to the people the policies affect. Bridging that gap—making analysis accessible without sliding into technocracy—is the central challenge.

Consider the stakes concretely. When Congress debated extending the 2017 Tax Cuts and Jobs Act in 2025—an extension ultimately enacted that July—the ten-year cost estimates ranged from $4 trillion to $5 trillion depending on the assumptions—a trillion-dollar range that could fund entire new programs or blow a hole in the deficit. The official scorekeepers at the Joint Committee on Taxation used models the public couldn't inspect. Independent groups like the Tax Policy Center, the Penn Wharton Budget Model, and PolicyEngine produced their own estimates, each with different data and methods. Members of Congress voted based on whichever number supported their position.

This isn't how it has to work. Imagine instead that every estimate was produced by open models—inspectable, reproducible, challengeable. That disagreements could be traced to specific assumptions rather than handwaved as "different methodologies." That a citizen could enter their own household circumstances and see, before a vote, exactly what the proposal would mean for them. Not approximately. Not according to a talking head. Precisely, calculated from the same rules that would govern their taxes.

Imagine further that the uncertainty was visible. Not "this reform costs $4.5 trillion" but "this reform costs $4.5 trillion, with a 90% confidence interval of $3.8 to $5.2 trillion, and here are the assumptions that matter most." That AI assistants, when asked about policy, could call these models and return accurate, citable answers instead of hallucinated approximations. That a nonprofit screening families for benefit eligibility could use the same calculation engine as the Congressional Budget Office, producing consistent results because they're running the same code against the same rules.

That infrastructure is closer to reality than most people realize. And building it—making it accurate, accessible, and trustworthy—is the project this book describes.

What's ahead

Every claim anyone makes about public policy decomposes into four questions. **What does the law say?** — a question of rules, answerable exactly, checkable against the statute. **Who are the people?** — a question of data, answerable statistically, checkable against surveys and administrative totals. **What will happen?** — a question of prediction, checkable only when reality arrives. And **what do we want?** — a question of values, the hardest to answer and the easiest to smuggle into the other three. This book is the story of building open computational infrastructure for each question in turn, and of the discipline that keeps such infrastructure

honest: every layer verified against ground truth, at the level of the individual unit—one rule, one household, one forecast.

Part I: The closed stack traces the history. Chapter 1 follows Guy Orcutt from his engineering background to the invention of microsimulation. Chapter 2 maps the tax model wars—how scoring became institutional authority, and how a confidentiality statute became an analytical moat. Chapter 3 confronts the accuracy question: how good are these models, really—and how would you know? Chapter 4 is the personal origin: the benefit cliff in my own family that turned the question from academic to urgent.

Part II: The open engine follows the building of PolicyEngine. Chapter 5 tells the origin story. Chapter 6 shows what the engine reveals—for one household, and for a country. Chapter 7 dissects the three ingredients every microsimulation must solve: rules, data, and dynamics.

Part III: The agent turn is where the story changes register. Chapter 8 shows why AI systems can't compute policy unaided—and why the answer is tools, not training. Chapter 9 reports on encoding the law at scale, by agents, under gates. Chapter 10 confronts the question everything depends on: how do you trust law encoded by machines? Chapter 11 takes the method to countries that never had public models at all. Chapter 12 explains why the organizations ended up mirroring the epistemology.

Part IV: The prediction pole crosses from calculation to forecasting. Chapter 13 exposes the uncertainty that point estimates hide, and the scoreboard being built to price it. Chapter 14 benchmarks what language models actually add to opinion research.

Part V: The horizon is the speculative edge, labeled as such. Chapter 15 models democratic processes. Chapter 16 asks whether we can forecast how human values evolve—and what that might mean for AI alignment. Chapter 17 returns to the fork in the road: Serac's closed system versus the democratic alternative.

The book ends where it started: at the choice between models that concentrate power and models that distribute it. That choice is being made right now, in code and policy and institutional design.

Some warnings before we begin. This is not a neutral survey. I've built many of the tools I describe, and I'll try to be honest about their limitations, but I'm not a disinterested observer. The chapters on systems I've created—PolicyEngine, HiveSight, Democrasim, the institutions of Part III—are marked as such. The speculative chapters are labeled clearly. Where something is early, unproven, or ungraded by reality, I say so—including, more than once, about my own projects.

I've also tried to resist the temptation to oversell. The technology described in this book won't save democracy, eliminate poverty, or solve AI alignment. What it can do is make the invisible visible—the distributional effects hidden in policy details, the uncertainty masked by point estimates, the benefit cliffs that trap families. Whether that visibility translates into better decisions depends on what people do with it.

This is the case for the open path.

Part I: The closed stack

Chapter 1: The birth of microsimulation

In 1957, an economist named Guy Orcutt published a paper that almost nobody read. It appeared in the *Review of Economics and Statistics*, respectable but not glamorous. The title was dry—"A New Type of Socio-Economic System"—and the prose was dense with equations. The idea at its core ran so far ahead of the available machines that building a working version would take nearly two decades.

It planted a seed anyway. Every tax calculator, every agency estimate of who gains from a policy change, every attempt to answer "what would this reform mean for families like mine" traces back to the inversion Orcutt proposed in those dense pages: stop modeling the economy as a set of aggregate equations, and start modeling the people inside it.

The engineer who became an economist

Orcutt came to economics sideways. He took a B.S. in physics from the University of Michigan in 1939 before switching fields for his M.A. in 1940 and his Ph.D. in 1944. He never lost the physicist's habit of treating the world as a system to be understood and, where possible, improved. Looking back near the end of his career, he described "my early fascination with science, my transition from engineering to economics".

The engineering showed in how he worked. Early on, inspired by Jan Tinbergen's econometric models, he designed and built an analogue electrical-mechanical "regression analyzer" to grind out statistical estimates, and he nursed what he called a "Tinbergen dream"—a single model that could capture an entire national economy. He thought like a builder: to understand a system, construct one.

He was no outsider to the profession he would unsettle. In 1949, with Donald Cochrane, he published a method for handling serial correlation in regression that became standard practice—the Cochrane-Orcutt procedure, still taught today. He directed the Littauer Statistical Laboratory at Harvard and consulted for the Federal Reserve Board and the International Monetary Fund. By any measure he was a successful academic economist working comfortably inside the mainstream. And yet, the deeper he worked with its tools through the 1940s and early 1950s, the more one limitation gnawed at him. The models could predict aggregates—GDP, inflation, unemployment. What they could not do was tell you what would happen to actual

people. He had spent years refining instruments that answered the wrong question with steadily improving precision: a model might pin the change in national income to a decimal and stay silent on the only thing a legislator ultimately had to defend—which of his constituents came out ahead, and which fell behind.

The problem with averages

In the 1950s, economists pictured the economy from the top down. The dominant approach was macroeconomic modeling: systems of equations describing aggregate relationships—total consumption as a function of total income, investment as a function of interest rates, employment as a function of output. The Cowles Commission, then the leading center for mathematical economics, was pursuing Keynesian simultaneous-equation models with enormous ambition and mixed results. Milton Friedman argued their forecasts were no better than naive extrapolation; others noted that the structural relationships could rarely be identified from the data, which left the theories nearly impossible to falsify. The data themselves were thin, and they sharply limited what could be estimated.

These models had real uses. They could predict GDP growth, illuminate business cycles, guide fiscal planning. But they shared a blind spot, which Orcutt named with characteristic understatement: "Existing models of our socio-economic system have proved to be of rather limited predictive usefulness."

The trouble was not the mathematics. It was the question the mathematics could answer. A macro model might estimate a tax cut's effect on total consumption, but it could not say which families gained most. It could not distinguish a tax cut that helped the middle class from one that helped the wealthy, or show that two policies with identical aggregate cost might land in wildly different places—one lifting families out of poverty, the other barely touching them. In a model whose smallest object was a national total, that difference had nowhere to live; it fell between the equations. Those distributional questions were precisely the ones policymakers needed answered—and precisely the ones the reigning method was built to lose.

The aggregation problem

Orcutt's insight was that aggregation itself was the culprit—not a mere limitation but a mathematical error. He made it concrete with a toy example. Suppose the relationship between an input X and an output Y is nonlinear—say $Y = X^2$—and you have 100 people. If every person has $X = 1$, total output is 100. Now split them: 50 people at $X = 0$, 50 at $X = 2$. The average X is still 1, so a macro model built on that average still predicts 100. But compute at the individual level, and the fifty zeros contribute nothing while the fifty twos contribute four apiece, for a total of 200.

The aggregate is identical; the truth is twice as large.

This was no curiosity. Tax systems are nonlinear—dense with thresholds, phase-outs, and cliffs. Benefit programs turn on the specific circumstances of specific households. The real world is full of exactly the discontinuities that make aggregation treacherous, and Orcutt's conclusion followed hard from the arithmetic: to understand how policy affects people, you have to model people. As he wrote, "There is an inherent difficulty, if not practical impossibility, in aggregating anything but absurdly simple relationships about elemental decision-making units."

A new type of model

The alternative Orcutt proposed was a model built from "interacting units which receive inputs and generate outputs." The units would be real decision-makers—individuals, households, firms—each carrying characteristics drawn from data, each following behavioral rules, some deterministic and some probabilistic. "The most distinctive feature of this new type of model," he wrote, "is the key role played by actual decision-making units of the real world such as the individual, the household, and the firm".

Instead of aggregate equations, a simulated population. Instead of predicting economy-wide averages, you would simulate what happens to each unit and add up the results: "Predictions about aggregates will still be needed but will be obtained by aggregating behavior of elemental units rather than by attempting to aggregate behavioral relationships." Derive aggregates from individuals; don't impose relationships on aggregates. That inversion was the breakthrough.

Orcutt was not the first to imagine modeling society mathematically. Isaac Asimov's *Foundation* trilogy (1951–53) had introduced "psychohistory," a fictional science that predicted the behavior of vast populations through statistical law—but Asimov built in a hard limit: it worked only for masses, never for a single person. "The reaction of one man could be forecast by no known mathematics," he wrote; "the reaction of a billion is something else again".

Orcutt inverted that too. His microsimulation would begin with individuals and build upward—predict the household, understand the society. The difference was not merely technical. Where psychohistory was elegant but autocratic, a plan only Hari Seldon could read and only a secret Foundation could steer, Orcutt's vision was granular and, at least in principle, democratic: if you could simulate any household, then anyone could ask how a policy would land on families like their own, and check the answer against their own life. It was a subtle reversal with large consequences. Psychohistory asked the public to trust a science it could never inspect; Orcutt's method, carried to its conclusion, invited the public to interrogate one—to run the model on a household it knew by heart and judge whether the machine got that household right. He called the method "microanalytic simulation." Later hands shortened it to microsimulation.

The vision exceeds the technology

The vision ran decades ahead of the tools. In 1957, computers filled rooms and cost millions. Programming meant punch cards and assembly language. Even a modest dataset of households was a crushing computational load. The IBM 704, state of the art when Orcutt published, managed roughly 12,000 floating-point operations per second; a modern laptop does billions. The entire machine available to Orcutt could do less arithmetic than a phone now spends animating a menu.

And he did not want to simulate households once. He wanted to push them forward through time—births and deaths, marriages and divorces, jobs taken and left, retirement—each household accumulating a life history the model would track, each branch multiplying the paths to follow. "The problem of keeping track of all possible paths and their respective probabilities," he admitted, "appears rather appalling."

Orcutt knew it was hard. He published anyway.

He was not alone in reaching past his hardware. The late 1950s were a high-water moment for computational ambition: John von Neumann had cast weather forecasting as a computing problem before his death in 1957, and Herbert Simon and Allen Newell were writing the first artificial-intelligence programs at the Carnegie Institute of Technology. What set Orcutt apart was his subject. He was not simulating storms or chess; he was simulating people—their economic choices, their entanglements with government, their trajectories through a life—so that policymakers might see how their decisions actually fell on citizens. For the next four years he built a prototype. In 1961, with three co-authors—one of them Alice Rivlin, his doctoral student—he published *Microanalysis of Socioeconomic Systems: A Simulation Study*. The book demonstrated a working, if limited, microsimulation of the US economy, modeling fertility, mortality, labor-force participation, and income through time—narrow proof that the concept was viable.

That co-author list matters. Fourteen years later Rivlin became the first director of the Congressional Budget Office, the institution that would make microsimulation a routine instrument of American governance; she later served as vice chair of the Federal Reserve. A method sketched in a paper almost nobody read would, within a single generation, run inside the agency that tells Congress what its bills will cost. The line from Orcutt's 1957 paper to the CBO's scoring machinery runs through one advisor-student relationship.

The Treasury connection

While Orcutt built academic prototypes, microsimulation was quietly entering government. Between 1962 and 1965, a young economist named George Sadowsky brought computers to revenue estimation at the Treasury's Office of Tax Analysis. The work was unglamorous—systems to estimate what a proposed tax change would cost or raise—but it was Orcutt's inversion applied to a live deadline: simulate individual taxpayers, apply the proposed rules,

sum the results. Sadowsky built a microanalytic model to analyze preliminary versions of the Revenue Act of 1964, and although it was simpler than Orcutt's dynamic vision—it did not age households forward—it proved the approach could answer questions a legislature was actually asking. For the first time, a proposed change to the tax code could be run against a sample of real taxpayers before it was enacted, and the answer handed to the people writing the bill. By the late 1960s, tax microsimulation was becoming standard in budget analysis. Sadowsky passed through Brookings and then the Urban Institute—where, not by coincidence, Orcutt was about to attempt his most ambitious project.

DYNASIM

Orcutt had spent the decade after 1958 trying to build a full-scale microsimulation at the University of Wisconsin–Madison, where he founded the Social Systems Research Institute in 1959. It did not work. The computing was insufficient, institutional support wavered, and the ambition outran what a university lab could sustain—Cheng's intellectual biography calls the Wisconsin years a "failed trial," a decade of methodological advance with no working policy tool at the end of it.

The Urban Institute, a newly founded Washington think tank, hired him in 1968 to lead a project called DYNASIM—Dynamic Simulation of Income Model. By 1969 it had the resources to begin in earnest, and the ambition was total: simulate every major life event—births, deaths, marriages, divorces, schooling, employment, disability, retirement, taxes, benefits. The machine was a DEC System-10 mainframe running a custom framework called MASH, and it simulated 10,000 people—a sliver of the population, but enough to draw statistical inferences. The team worked six years. The first version was complete in 1975, eighteen years after Orcutt's original paper.

That gap tells you what kind of idea this was. Orcutt had not proposed an improvement to existing methods; he had proposed a different way of thinking, and the technology had to climb to meet it. What it bought, once built, were questions no macro model could touch. Ask how a proposed Social Security reform would fall across generations, and DYNASIM could age cohorts through time, accumulating earnings histories, reaching retirement, and drawing benefits under alternative rules. Ask about the long-run fiscal weight of an aging population, and it could project the demographic shift and its effects on revenue and spending. Ask how one program's rules interacted with another's, and it could trace the combined result rather than studying each in isolation. Its modules were organized by domain—one for demographic events, one for the labor market, one for taxes, transfers, and wealth—and it modeled the whole as an integrated system, demographic events feeding labor-market outcomes feeding taxes and benefits. That scope was expensive to run and hard to modify, but it captured interactions that piecemeal analysis missed, and it proved the vision was more than a theoretical curiosity.

The family of models

DYNASIM did not stay alone. The Urban Institute carried it through successive generations—a narrower retirement-income version in the early 1980s, a major overhaul around 2000 that projected the US population across seventy-five years on the Social Security and Medicare trustees' assumptions, and a fourth generation still active today, among the longest-lived computational models in the social sciences.

Orcutt's framework spread abroad and into new forms. Steven Caldwell built CORSIM at Cornell, a direct descendant that in turn seeded POLISIM for Social Security analysis and DYNACAN for Canadian pensions. Statistics Canada developed SPSD/M, a static Social Policy Simulation Database and Model that became a workhorse of Canadian tax-benefit analysis. Sweden's SVERIGE modeled the country's entire population. A parallel tradition grew alongside these dynamic models: static microsimulation, which asked the simpler question of what would happen to today's population under a policy changed today. Static models traded the life-course view for tractability—cheaper, faster, easier to point at a specific reform—and the IRS, CBO, and Treasury all built them. In 1974, Joseph Pechman and Benjamin Okner of Brookings merged data on 72,000 households to ask who really bore the burden of American taxes, a distributional question aggregate models could never answer. It was a founding demonstration of static microsimulation's distinctive power: to take a question about fairness that aggregates could only gesture at, and return a number. Both traditions, dynamic and static, descend from the same insight—model individuals, and the distribution reveals itself.

The micro-macro distinction

The difference between microsimulation and macro modeling is not only technical; the two answer different questions.

(A table appears here in the print and web editions.)

Macro models remain powerful for their intended purpose—aggregate dynamics, business cycles, growth. They are simply the wrong instrument for distribution. And distribution is what democratic argument runs on: when legislators weigh a policy they need to know who wins and who loses, and when citizens judge one they want to know what it means for people like them.

The same insight has since spread far past economics. Health researchers use microsimulation to project how a population will age, develop disease, and respond to a vaccination program. Climate models pair physical projections with economic ones to trace how warming lands differently on different households. Transportation planners simulate how individuals choose routes, modes, and destinations before committing to an infrastructure investment. Epidemiologists model person-to-person transmission across contact networks—Orcutt's "interacting units" under another name. The common thread is the one he found in 1957: an aggregate

relationship hides the distributional detail, and if you want to know how a system affects the people inside it, you have to model the people.

The constraint dissolves

What Orcutt could not anticipate was how completely the technology constraint would fall away. DYNASIM simulated 10,000 people on a mainframe that cost a fortune to run; a present-day laptop simulates millions, and the computing power that ran to millions of dollars in 1975 now fits in a pocket. The change is not only one of degree. Larger samples make it possible to study small subgroups that the early models could never have supported statistically. Calculations that once took hours return in seconds, which turns a batch model into an interactive one—a policy calculator that answers while you watch and lets an analyst iterate in an afternoon what used to take a season. And the machine itself stopped being a barrier: a microsimulation can now live behind a web page, so exploring a policy's effects no longer requires mainframe time, a research grant, or a graduate degree. Much of what Orcutt had proposed and been denied became ordinary. He had wanted to age a whole population through decades of births and jobs and retirements; his hardware could barely carry a single year of the exercise, and the idea had, in effect, waited half a century for machines that could finally hold it. Orcutt built tools for experts to inform policymakers. He did not picture ordinary citizens running their own analyses—but that is where the vision led.

What Orcutt built

Guy Orcutt died on March 5, 2006, having watched an impractical dream become standard method. The intellectual line ran on through his own family: his daughter, Alice Orcutt Nakamura, became an economist, and his granddaughter, Emi Nakamura, won the John Bates Clark Medal in 2019, awarded to the most promising American economist under forty—three generations descended from a physicist who thought the economy could be modeled like a circuit. Recent scholarship stresses that Orcutt saw the method as more than a lens. As the historian Hsiang-Ke Cheng put it, microsimulation was "an engine designed for not only scrutinizing the system but reengineering the society". The engineer never left the economist.

The deeper legacy was never any single model. It was a way of seeing. Before Orcutt, economists reasoned about the economy in aggregates; after him, they could reason about it as a population of individuals, each with its own circumstances, each touched differently by the same law. A policy that looks fine in aggregate can still harm millions of specific people, and the only way to see them is to model them. That is not, in the end, a claim about better arithmetic; it is a claim about whose reality the arithmetic represents—the average of a nation, or the nation's actual members, one at a time. Sixty-nine years on, that shift is still working itself out.

Orcutt could imagine the world his tools implied, but not build it. What if anyone could run a microsimulation, not just an agency with a mainframe? What if an AI could call one in plain language, on behalf of a person who never learned to code?

But the payoff that matters most in 2026 hides inside his original definition. Orcutt's "interacting units which receive inputs and generate outputs" are exactly the per-unit encoding at the center of this book—and building a society one unit at a time turns out to buy something aggregation never could: per-unit verifiability. Each simulated household can be checked against ground truth—its tax against the statute, its benefits against an administrative total, its forecast against the official number when that number finally lands. That check is the discipline everything ahead is built on.

Chapter 2: The tax model wars

In 1983, a small London think tank did something that would reshape policy analysis for decades: it built a working model of the British tax and benefit system and turned it on the government's own budget.

The Institute for Fiscal Studies had been founded in 1969 by four financial professionals frustrated with the opacity of UK tax policy. TAXBEN, their microsimulation model of British taxes and benefits, gave them something no outside body had held before—the ability to run the numbers themselves. When a Chancellor rose to announce a budget, IFS could feed the new rules into TAXBEN, simulate their effects on families up and down the income distribution within hours, and publish the results before the official gloss had time to set. Its annual Green Budget analyses became required reading for the journalists, civil servants, and politicians whose arithmetic the institute was now in a position to check. For the first time an independent organization could contest an official estimate with a calculation of its own, and the information asymmetry that had always favored the people in power began to crack.

But only began. IFS could contest the government's numbers; it could not see the government's data, and neither could anyone else outside a few rooms in Washington and Whitehall. Four decades later that asymmetry persists, and understanding why—what makes it so durable, and what it would actually take to answer—is the thread that runs through this chapter and, eventually, through the rest of the book.

The machinery behind the curtain

While IFS was building TAXBEN in London, the American government was assembling a far larger analytic apparatus, and assembling it out of public view.

The story runs through Alice Rivlin. In 1974, Congress created the Congressional Budget Office to give itself analytical capacity independent of the executive branch, so that it would no longer have to accept the President's budget numbers on trust. Rivlin, an economist from Brookings, became CBO's first director, and she was specific about what she wanted: an office that was nonpartisan, analytical, and willing to publish numbers that discomfited both parties. She hired economists rather than political operatives and established a culture of methodological transparency that has outlasted her by five decades. CBO built microsimulation models for health insurance, Social Security, and tax analysis, and for the long-run fiscal projections that entitlement debates turned on it built CBOLT, a long-term model of the whole federal budget. For the first time, Congress could evaluate a President's proposals against numbers of its own.

Congress's official scorekeeper for tax legislation was older still. The Joint Committee on Taxation, created in 1926, had by the 1970s developed a suite of microsimulation models—an individual income model, a corporate model, an estate and gift model, an international cross-border model—and its estimates carried legal weight. The budget process, built on the Congressional Budget Act of 1974 and its later amendments, made JCT's numbers the official revenue estimates for tax bills. That gave a modeling shop an unusual measure of power over what Congress could actually do.

The mechanics are worth seeing up close, because they are where the abstraction turns consequential. A senator drafts a tax amendment. JCT's staff feed the proposal into their models, which apply the new rules to a representative sample of actual tax returns and, within days—sometimes hours—produce a score: the projected change in federal revenue over ten years. That single number decides whether the amendment fits the budget resolution, whether it survives under reconciliation, whether it is fiscally viable at all. A score that comes in too high kills a proposal before it ever reaches a vote. No hearing, no floor debate—just a number from a model that most members of Congress had never seen and could not have inspected if they tried. The score did not merely inform the decision; on a reconciliation bill it often was the decision.

The executive branch built its own version of the same machinery. Treasury's Office of Tax Analysis had been experimenting with computerized tax models since George Sadowsky's work in the early 1960s; by the 1980s it maintained the Individual Income Tax Model, refreshed regularly from IRS returns, producing the revenue estimates behind the Administration's proposals as JCT's served Congress. When the two disagreed—and on a contested proposal they often did—there was no neutral way for anyone outside to adjudicate, because the disagreement lived inside two sets of models that neither the press nor the public could open. Outside government, the Urban Institute ran TRIM3, updated every year from the Current Population Survey to model the benefit programs—SNAP, Medicaid, TANF, SSI, housing assistance—that the tax-focused models left out. It was the kind of work Karen Smith had been doing at Urban since the 1980s (chapter 1), and it filled a genuine gap, capturing the interaction between taxes and transfers that neither JCT nor Treasury modeled comprehensively; but, like the rest, it served government clients rather than the public.

Three federal institutions, plus Urban's adjacent capacity. Billions of dollars in policy decisions riding on their outputs, and a fraternity of a few hundred specialists who could actually read them. And almost none of it visible to the people those decisions fell on.

The analytical moat

The tax model wars were never really about models. They were about power—the power to define what a policy would cost and whom it would help—and that power rested on something no competitor could buy, borrow, or reverse-engineer: the data.

Section 6103 of the Internal Revenue Code forbids the disclosure of individual tax-return information, with a narrow set of exceptions. Under Section 6103(l), JCT and Treasury have statutory access to the returns themselves. External researchers do not. So the government's models run on the full universe of actual returns—every filer, every dollar, the true joint distribution of income and deductions and credits—while everyone outside works from public-use files that have been sampled, anonymized, and top-coded—the very highest incomes collapsed into a single category or swapped for an average, deliberately blurred at precisely the point where the biggest tax fights are decided. Anyone trying to score a change to the top rate is working with the one slice of the distribution that has been smoothed away for privacy's sake. The confidentiality is entirely justified; taxpayers should not have their returns exposed to the public. But its byproduct is an analytical moat that no amount of open-source software can drain. You can publish your code, your methods, and your assumptions, and you still will not have the data.

This is the constraint the rest of the book keeps returning to, so it is worth being exact about what the secrecy cost. An official estimate a politician disliked could be dismissed as biased, and no one outside could check the work; when an estimate turned out wrong, as estimates sometimes did, no one outside could reconstruct why. Debate narrowed to what the scorekeepers were willing to model, so proposals that didn't fit the existing frameworks often couldn't be scored at all, which made them politically impossible whatever their merits. The expertise to build and maintain this machinery pooled inside a handful of institutions, where academic economists could study tax policy for a career without ever being able to replicate the infrastructure that priced it. And ordinary citizens who wanted to know how a reform would land on families like theirs had to take the official number on faith. The asymmetry between the governed and those who govern them reached all the way down into the tools used to judge the choices.

Some academics pushed against it anyway. At the National Bureau of Economic Research, Daniel Feenberg began building TAXSIM in the early 1980s; by the 1990s it was reachable over the internet, one of the first tax calculators anyone could run remotely. Feenberg maintained it for the rest of his career—a quiet, decades-long act of infrastructure that let generations of researchers compute tax liabilities for their survey samples without asking anyone's permission. It became a fixture of the empirical tax literature precisely because it asked so little of the

people who used it: send it a record, get back a liability, cite the paper, move on. But TAXSIM calculated taxes for individual records; it did not produce the aggregate revenue estimates and distributional tables that drove policy fights. Feenberg had proved that useful tools could be built outside government. He had not closed the moat.

Breaking the monopoly

The first serious challenge to the government's monopoly came from inside the establishment but outside the government. In 2002, Gene Steuerle and Len Burman—who understood the official models because they had helped build them, Steuerle at Treasury, Burman at CBO and Treasury—founded the Tax Policy Center as a joint venture of the Urban Institute and the Brookings Institution. They wanted something the government shops were not: an independent, publicly accessible model that would analyze the proposals the official scorekeepers wouldn't touch. TPC built its microsimulation on the same IRS public-use files that everyone outside government had to use—the same foundation as the government models, though less granular than the confidential returns—and it was enough to estimate the revenue and distributional effects of a tax proposal within days.

The payoff arrived in 2012. When Mitt Romney campaigned on a tax plan that promised to cut rates while staying revenue-neutral through unspecified base-broadening, TPC ran the arithmetic and showed that the promise was mathematically impossible: the numbers could not be made to balance without raising taxes on the middle class or losing revenue. The finding dominated coverage and reshaped the campaign's debate, and Romney's team—having no comparable model of their own—could dispute it but could not rebut it with numbers. During the 2017 Tax Cuts and Jobs Act debate, TPC again supplied what the official score left out, showing how heavily the bill's benefits tilted toward high-income households, a detail that complicated the messaging about help for the middle class. An outside model had become a fixture of the debate rather than a curiosity at its edge.

Then the field multiplied. Kent Smetters launched the Penn Wharton Budget Model at the University of Pennsylvania in 2016, and it distinguished itself by producing dynamic estimates—scores that let a tax change move the size of the economy rather than holding it fixed. The Budget Lab at Yale followed later with yet another lens. In little more than a decade, the number of independent shops able to score a major tax proposal had gone from essentially zero to half a dozen, each with its own data, its own methods, and—inevitably—its own political center of gravity. The Tax Policy Center and the Institute on Taxation and Economic Policy leaned left on distributional questions; the Tax Foundation leaned right on growth effects; Penn Wharton positioned itself in the middle. The practical consequence was that any major proposal now drew a barrage of competing estimates, each defensible and each different, so that a reader had to understand the assumptions to know why the numbers diverged. The monopoly on analytical capability was breaking, even as the moat around the data stayed dry and impassable.

The dynamic scoring wars

Underneath the institutional competition ran a deeper and more explosive dispute: should a budget score account for the economy's response to the very policy being scored?

The question sounds technical and is politically radioactive, because it decides how expensive a tax cut is allowed to look. Traditionally, JCT scored a tax cut statically—a hundred-billion-dollar cut costs a hundred billion, full stop, with the size of the economy held fixed. Supply-side economists had long argued that this overstated the true cost, because lower rates sharpen the incentives to work and invest, the economy grows, and some of that growth flows back as revenue; under dynamic scoring, the same cut might book at seventy billion. The argument had run for decades. Alan Auerbach and William Gale worked through it in forums and papers across the 2000s, and in 2005 Auerbach laid out the issues formally: dynamic scoring was conceptually correct and practically treacherous, because the macroeconomic models needed to estimate the feedback were themselves deeply uncertain. In January 2015, the new Republican House majority stopped arguing and made it a rule, requiring JCT to produce dynamic scores for legislation with budgetary effects above a quarter of a percent of GDP—a victory for the supply-siders who believed static scoring had been systematically overstating the cost of tax cuts.

The first real test was the TCJA. JCT's static score put the ten-year cost at $1.46 trillion; its dynamic score, crediting the law with the economic growth it was projected to generate, came in at $1.07 trillion, reflecting roughly $385 billion of revenue recovered through faster growth. The dynamic estimate knocked about 27 percent off the cost—real money, and nowhere near the supply-side dream that the cuts would pay for themselves. Penn Wharton, running the feedback through its own model, put the dynamic cost higher still, in the range of $1.8 to $2.2 trillion, judging the growth effects modest enough to offset some but not most of the revenue loss. The models disagreed about the magnitude; none of them found a free lunch. And the 2017 law's individual provisions did not, in the end, expire on the schedule it set—the One Big Beautiful Bill Act extended them in 2025—which is a reminder that the ten-year windows these scores are built on rarely close the way the legislation assumes.

When Douglas Elmendorf, Glenn Hubbard, and Heidi Williams assessed the whole experiment years later, their verdict was measured and, for both camps, deflating: dynamic scoring had improved budget analysis at the margins without resolving the fundamental uncertainty about macroeconomic feedback. The models were better than guessing. They were not the precision instruments that advocates on either side wanted them to be.

An alternative across the Atlantic

Britain and Europe took a different route to the same destination, and in some respects got closer to opening the black box than the American institutions did.

Australia had shown early what was possible beyond the Anglo-American core. From 1993, Ann Harding built a microsimulation program at the National Centre for Social and Economic Modelling in Canberra, and her edited volume *Microsimulation and Public Policy* (1996) served for years as the field's standard reference. Harding's argument was as much institutional as technical: these models belonged in the hands of policymakers and the press, not locked inside academic departments, and NATSEM's models were used across Australian government agencies, parliamentary committees, and media, on everything from tax reform to childcare policy. Microsimulation, in her telling, was not a specialist's instrument to be guarded but a public utility to be spread—a conviction the field outside the United States absorbed earlier and more fully than the American institutions did.

The European breakthrough was more ambitious still. From 1996, a team led by Holly Sutherland set out to build EUROMOD, a single tax-benefit model that would eventually span every EU member state—harmonizing wildly different national systems into one framework so that a reform in Portugal and a reform in Finland could be compared on the same terms—cross-national comparisons that had simply not been possible before. The ambition was staggering, and it worked. Developed at the University of Essex on European Commission research grants, EUROMOD was built for access: researchers could apply to use it, learn its methods, and run their own analyses. The code was not fully open, but the ethos was sharing rather than hoarding, and the project became successful enough that the European Commission eventually took over its maintenance through its Joint Research Centre—a university experiment turned official EU infrastructure.

Essex went further with the UK piece. With funding from the Nuffield Foundation, a project led by the economist Mike Brewer turned EUROMOD's UK component into a standalone model, UKMOD, housed at the university's Centre for Microsimulation and Policy Analysis under the direction of Matteo Richiardi and introduced in a 2021 paper by Richiardi, Diego Collado, and Daria Popova. Its first free public release, in October 2019, made it the UK's first freely available tax-benefit model, fully open and usable by anyone who wanted it. "We wanted to democratize access to tax-benefit analysis," Brewer said. The Scottish Parliament's research service picked it up, and so did NHS Health Scotland and the Welsh Government; for the first time, the UK's devolved governments could run the same analysis for themselves instead of waiting on London to run it for them. Independence of analysis, it turned out, could follow from a freely downloadable model as surely as from a new institution.

Openness of that kind seeded a generation of independent analysts. Howard Reed had run TAXBEN inside the IFS from 2000 to 2004, learning the craft of institutional model maintenance from the inside; after a spell as chief economist at IPPR, he founded Landman Economics in 2008 and built his own tax-transfer model, using it to analyze basic income, welfare reform, and the cumulative toll of austerity—the kind of detailed distributional work that had once been the exclusive preserve of government and the large think tanks. His stated ambition was "a new settlement of the same scale and sustainability as the Beveridge-inspired reforms of 1945". Malcolm Torry, directing the Citizen's Basic Income Trust, used EUROMOD across

nearly a decade of research to cost basic-income schemes in granular detail, demonstrating that an advocacy organization could also be a rigorous analyst—provided it had the tools.

Rules as code

Openness of *access* was one thing. Openness of the *engine* was the revolution that arrived next, and it began, improbably, with a bestselling book.

In January 2011, Camille Landais, Thomas Piketty, and Emmanuel Saez published *Pour une révolution fiscale*—"For a Tax Revolution"—arguing for a wholesale restructuring of French income tax into a single progressive levy. The book was ordinary public economics. The website that came with it was not: revolution-fiscale.fr let any French citizen design a tax reform and watch its budgetary and distributional consequences resolve on the screen, and it drew hundreds of thousands of visitors. Until then, simulations like these had been possible only inside the French Finance Ministry; unlike the United States, with its independent CBO, France gave its parliament no analytical shop of its own, so evaluating a proposed amendment meant going through the Ministry. Three economists had broken that monopoly—not with institutional authority but with code that anyone could run.

Four months later, a small team inside the French government went further and released not a simulator but the source code beneath one. They called it OpenFisca. It came out of the Centre d'analyse stratégique—the government's strategy-and-analysis unit, later renamed France Stratégie—and was written by Mahdi Ben Jelloul and Clément Schaff on a simple premise: that tax and benefit rules should exist not only as legal prose but as executable code. Give the system a household's circumstances—income, family structure, location—and it returned their taxes and benefits; change a rate or a threshold and the effects propagated at once. Where revolution-fiscale.fr had opened the *results* of microsimulation, OpenFisca opened the *machinery*.

The idea went by several names—rules as code, legislation as code, machine-consumable rules—but the claim underneath was always the same: the rules that govern citizens' lives should be not merely published but publicly computable. Tax agencies had been encoding rules in software for decades, ever since Sadowsky's Treasury experiments, but always privately, inside their own systems, so that citizens met the *outputs* of that code—a tax bill, a benefit award—without any access to its *logic*. Now governments began, tentatively, to open it. In 2018, New Zealand's Service Innovation Lab spent three weeks translating a piece of legislation into Python and human-readable rules at once, demonstrating that a law could be drafted in both forms simultaneously; the OECD took notice and in 2020 published *Cracking the Code*, a primer for governments on what the approach could mean. OpenFisca, meanwhile, spread across four continents—powering France's Mes Aides benefits calculator, a New Zealand rates rebate, and adaptations in Tunisia, Senegal, and beyond—and collected an Innovation of the Year nod from the OECD and recognition from the European Commission as its most innovative open-source software.

In the United States, the same push came from an unlikely quarter. In 2013, Matt Jensen founded the Open Source Policy Center at the American Enterprise Institute, a market-oriented think tank not otherwise known for software. Jensen's diagnosis was blunt—that closed-source estimation left the public and most policymakers in the dark—and his fix was to build in the open. He recruited Martin Holmer, an MIT-trained economist with decades of modeling behind him, to write Tax-Calculator, an open-source model of US federal income and payroll taxes. The irony was productive: a conservative institution building open tools meant the project could not be waved away as a left-wing effort to undermine the official numbers, and it drew contributors from across the political spectrum.

The two flagship projects embodied two philosophies. OpenFisca offered a single unified framework—a country-independent engine written in Python, with separate packages encoding each jurisdiction's rules—which made a new country quick to stand up; at the 2016 Open Government Partnership summit in Paris, a team of French and Tunisian volunteers modeled Senegal's income tax in under thirty-six hours and won the hackathon. By 2019, France's National Assembly had built LexImpact on top of it, letting legislators simulate the effects of proposed amendments, and within two years the tool had informed well over a hundred simulations in parliamentary debate—a think-tank instrument turned legislative infrastructure. Tax-Calculator, by contrast, spawned a federation: it grew into the Policy Simulation Library, a loose confederation of independent open-source models that shared transparency and interoperability standards while each specialized—overlapping-generations macro models for dynamic scoring, a capital-cost-recovery model from the Tax Foundation, computational-economics tools from QuantEcon—and accumulated the governance and institutional memory that let researchers from CBO to the City of New York build on it. One framework prized coherence and cross-national comparison; the other prized specialization and rapid experimentation. Even the Federal Reserve joined the turn, releasing a Python implementation of FRB/US, its several-hundred-equation macroeconomic model, in 2022. The central bank's primary forecasting tool was now open source.

And yet the revolution kept stopping short of its own promise. Open code was not the same as usable code: OpenFisca and Tax-Calculator both demanded real Python skill, which meant the rules were public in roughly the way a research library's manuscripts are public—present, technically available, and legible to almost no one who might want to read them. Tax codes change every year, so someone had to keep the models current, and open source meant that anyone *could* do the maintenance while guaranteeing that no one in particular *would*. And even where the code ran and the answer looked right, trust remained its own problem: governments were wary of unofficial numbers that contradicted their own, and a citizen staring at two different estimates had no reliable way to tell which one deserved belief. The engine was open. The last mile—to something an ordinary person could use, and rely on—was not yet built.

Synthesize and calibrate

By the mid-2020s the wars had produced something that would have been unimaginable in 1990: a reasonably informed person could gather half a dozen independent estimates of any major tax proposal within days of its introduction, and a decade of open-source work meant that some of those estimates came from tools anyone could inspect, run, and adapt—one of which, PolicyEngine, would eventually put the open approach in front of anyone with a web browser.

But notice what all of it left untouched. Independence, transparency, open code, a plurality of competing models—every gain of the previous forty years was a gain in *method*. Rules as code had a second limit hiding inside the first: encoding the rules gave you an engine that could compute any individual scenario, but to produce an aggregate revenue estimate or a distributional table you needed something the engine didn't contain—a representative population to run it over. And a representative population at the fidelity that policy fights demand ran straight back into Section 6103. The confidential returns stayed exactly where they were. The government's structural advantage in data survived every open-source victory intact, because you cannot open-source a dataset you are legally forbidden to hold.

Which points to the only answer the constraint actually permits. If you cannot have the real returns, you build a synthetic population that behaves like them—assembled from the public files you are allowed to use, then calibrated against the administrative totals the government does publish, so that the aggregates it reports and the distribution those aggregates imply are both matched even though no real person's return is ever exposed. The government's own published numbers become the answer key: you fit an artificial population to them until it reproduces the world the confidential data describes, and then you run the open rules over the population you built. You do not breach the moat. You reconstruct, statistically, what stands behind it. That the reconstruction can be made faithful enough to matter is not obvious—it is the problem the next layer of this story has to solve, and much of the rest of this book is about whether, and how, it can be done.

Chapter 3: The accuracy question

In 2017, the Joint Committee on Taxation estimated that the Tax Cuts and Jobs Act would reduce federal revenue by $1.46 trillion over ten years. The Penn Wharton Budget Model projected larger losses—$1.8 to $2.2 trillion on a dynamic basis, once economic effects were counted. Congressional Republicans disputed both; supply-siders predicted the cuts would pay for themselves through growth.

The supply-side prediction didn't materialize. The microsimulations had been approximately right about direction and magnitude.

But "approximately right" is the best anyone can honestly say.

Do these things actually work?

Through the 1990s and 2000s the models grew more sophisticated. The IFS refined TAXBEN; the Urban Institute expanded TRIM3; policy shops on every side produced estimates that happened to support their preferred conclusions. A reasonable observer might ask whether any of it works. The models are complex, their assumptions buried in code, their data imperfect—and two of them, handed the same reform, will sometimes return different answers.

The institutional fight over who gets to produce those numbers belongs to the previous chapter. What concerns us here is narrower and harder: whether the numbers are any good. The honest response is not "trust us." It is "here is how we test ourselves."

Three ways to be right

Microsimulation validation happens at three levels, and they are not equally easy.

Component validation asks whether individual calculations match the rules. If the model says a married couple earning $100,000 owes a particular amount of federal tax, is that amount correct? This is the tractable case—you can check it against IRS worksheets or commercial tax software.

Aggregate validation asks whether the model, summed across the population, matches administrative totals. Does its estimate of total SNAP enrollment match USDA records? Does its total income-tax revenue match IRS collections?

Predictive validation asks whether the model's forecast of a change holds up. This is the hard one. You rarely get a clean experiment: policy changes arrive bundled with economic shifts, other reforms, and behavioral responses no one anticipated.

Good models pass the first two levels reliably. The third is where humility enters—and where most of this chapter lives.

The ACA test case

The Affordable Care Act is one of the better natural experiments for testing that third level. In March 2010, the CBO predicted the ACA would cut the non-elderly uninsured rate from over 18 percent to about 7.6 percent by 2016, assuming every state expanded Medicaid. Then the Supreme Court made expansion optional, and nineteen states declined. Adjust for that, and the CBO's projected 2016 uninsured rate becomes 9.4 percent. The actual rate, from CDC data, was 10.4 percent—within a point, across a six-year horizon interrupted by a major legal shock.

The aggregate accuracy hid offsetting component errors.

(A table appears here in the print and web editions.)

Source:,

The CBO overestimated exchange enrollment by more than half and underestimated Medicaid enrollment by nearly half. Yet the total coverage gain came out roughly right, because the two errors ran in opposite directions and partly cancelled. As one analysis put it, the CBO's mistake "was in estimating *where* the uninsured would get covered, not *how many* of them would gain coverage". Right about the total, wrong about the composition: that is the characteristic signature of a model that passes aggregate validation while failing at the component level, and it is a warning against reading a good headline number as proof the machinery underneath is sound. The cancellation here was closer to luck than to skill—two large errors that happened to point in opposite directions—and the next reform's errors might just as easily compound. A number that comes out right for the wrong reasons has not been validated; it has been lucky, and luck does not generalize.

Are the forecasts improving?

Here is a question rarely asked. Is government forecasting getting better? The CBO is one of the few institutions that grades itself in public, publishing systematic retrospectives that compare its projections to what actually happened. For sixth-year deficit projections—the medium-term forecasts that anchor major policy debates—the average absolute error fell from 3.2 percent of GDP over 1989–2001 to 1.0 percent over 2002–2019, a threefold improvement in two decades. Richer IRS administrative data, more capable computing, and decades of retrospective analysis that exposed and corrected systematic biases all contributed. The mechanism worth noting is the feedback loop itself: because the CBO scored its own past forecasts against outcomes, it could find the places it had been reliably wrong and adjust—an accountability loop most forecasters never build, and much of the reason its numbers improved at all.

The improvement has limits. The CBO's projections remain about as accurate as the Blue Chip consensus of some fifty private forecasters, and about as accurate as the Administration's own numbers; no one has found a way to consistently beat the collective wisdom of informed forecasters. And recent years brought more volatility, not less. The 2021 projection was the CBO's largest overestimate on record, and the 2023 its largest underestimate—off by 3.9 percent of GDP, more than triple the historical average. The pandemic scrambled every model built for normal times.

The random walk

The most humbling result comes from academic work. A Berkeley thesis examining CBO forecasts from 1976 to 2007 found that a "random walk"—simply assuming next year's deficit equals this year's—would have beaten the CBO on average, over both short and medium horizons.

This is not an indictment of the CBO's competence. It is a statement about irreducible uncertainty. The events that dominate fiscal outcomes—recessions, financial crises, pandemics—are precisely the ones no model foresees, and a model calibrated on ordinary times fails exactly when the times stop being ordinary. That a naive rule can match a sophisticated one, on the questions that matter most, is the first hint that forecasting is a different kind of problem from computing a household's taxes—and may need a different kind of accountability.

What the TCJA record shows

For the Tax Cuts and Jobs Act we now have seven years of data. Real (inflation-adjusted) revenue for 2018 through 2024—excluding the anomalous 2022 surge, driven by capital-gains realizations and inflation rather than the tax law—came in within 0.5 percent of CBO's 2018 projections. That is remarkably accurate for a major tax overhaul. The supply-side claim that the cuts would pay for themselves proved false; the microsimulation estimates proved roughly correct.

One note on currency. The TCJA's individual provisions did not expire on schedule: the One Big Beautiful Bill Act extended them in July 2025, making the pass-through deduction permanent and setting the Child Tax Credit at $2,200 for 2026. So the 2018–2024 window is a completed natural experiment, not the end of the law—the record over those years is closed and gradable, even as the statute itself runs on.

The prediction-market benchmark

There is a way to forecast that sidesteps models entirely: let people bet, and read the price. Prediction markets aggregate dispersed information, put money on the line, and update quickly when experts with reputations to protect would not.

For macroeconomic variables, an NBER study found prediction markets "weakly more accurate than survey forecasts" across GDP, inflation, and employment—a modest but consistent edge. For elections the gap is sharper: the Monday before the 2024 presidential vote, with polls showing a coin flip, Polymarket had Trump at 58 percent, and academic work comparing market prices to FiveThirtyEight has found the markets competitive with or better than the model. For Fed decisions, Good Judgment's "superforecasters"—individuals identified through tournaments as unusually well-calibrated—reportedly beat financial futures markets by about 30 percent in 2024–2025. Philip Tetlock's research supplies the frame: most experts forecast little better than chance, but a small minority, roughly 2 percent of participants in the Good Judgment Project, consistently outperform—by updating often, thinking in probabilities, and refusing to marry a prediction to an ideology.

Two lessons follow for microsimulation. Where a market exists for a policy-relevant question—will inflation top 3 percent, will unemployment rise—it offers a live calibration check against which a model's forecast can be scored. But markets need clear resolution dates and objective

outcomes; they cannot price a counterfactual policy that was never enacted, which is exactly the terrain microsimulation has to itself. The synthesis is the interesting part: structural estimates from a model, calibration from a market, each covering the other's blind spot. That same idea returns in Part IV, as an institution that publishes interval forecasts of government metrics and grades them when the official numbers land—though none have resolved yet, so nothing there is validated, only staked.

The survey-data crisis

Before asking whether the outputs are accurate, it is worth asking about the inputs—and the answer is unsettling. Bruce Meyer, an economist at the University of Chicago, has spent two decades documenting what he calls a crisis in household survey data. People do not accurately report their income and benefits to survey-takers, and the errors are not random. When Meyer and colleagues linked survey responses to administrative records, they found that roughly 40 to 50 percent of SNAP recipients did not report receiving benefits in the Current Population Survey. More than 60 percent of Temporary Assistance for Needy Families and General Assistance went unreported. About a third of housing-assistance recipients did not mention it. Even Social Security, which arrives monthly and ought to be the easiest transfer to remember, was underreported by about a tenth of recipients.

This cascades through every model built on survey data. When a microsimulation estimates the poverty effect of SNAP, it assigns benefits to households that look eligible from their reported income. But if half of actual recipients never reported the benefit—and may also have misreported the income and household composition that determine eligibility—the model is reasoning from a distorted picture of who gets SNAP and what their circumstances are. And the distortion has grown: survey response rates have fallen since the 1990s, the non-responders skew low-income, young, and mobile, and the Census Bureau's imputations increasingly fill the gaps with inferences that reflect the imputation model as much as the household.

The bias runs both ways. Seniors systematically underreport retirement income. A Census Bureau study linking CPS responses to IRS and Social Security records found median income for those 65 and older was 30 percent higher in administrative data than in survey reports; CPS-based senior poverty of 9.1 percent was 2.2 points higher than the 6.9 percent the validated records showed. Senior poverty is real; the headline survey number overstates it. At the other end of the distribution, very high earnings are top-coded in public-use data—masked to protect confidentiality—which compresses the top and makes reforms aimed at high earners harder to model accurately. These problems touch nearly every model that leans on household surveys, and the models with confidential IRS access escape only some of them: tax returns miss non-filers, omit non-taxable transfers, and describe legal tax units rather than economic households. Microsimulation is only as good as its data, and the data is imperfect in systematic, consequential ways. Which means accuracy is not only a matter of better equations; it is a matter of better inputs—of reconstructing, from imperfect surveys and partial administrative records, a population that matches the totals we can actually observe.

TAXSIM, the academic benchmark

TAXSIM, the NBER tax calculator introduced in Chapter 2, doubles as a validation benchmark. Daniel Feenberg maintained it for more than four decades, and the thousand-plus published papers that cite it created an informal validation network: when TAXSIM returned something a researcher didn't expect, someone investigated and reported the bug, and the model improved under collective scrutiny. For its core job—computing federal income tax from a given set of inputs—its accuracy is high, the rules being deterministic and well documented; the discrepancies that surface tend to sit in state-code edge cases or credit interactions. The lesson in that record is quiet but important: a tool becomes trustworthy less through any single act of certification than through years of exposure to users who had every incentive to catch it in a mistake.

But TAXSIM computes taxes only. It does not simulate SNAP, Medicaid, SSI, or housing assistance—the benefit programs whose cliffs and interactions later chapters turn on—and it works record by record, producing no aggregate revenue estimate or distributional table. What it established was narrower and still important: an independent, inspectable tool could match proprietary accuracy for a well-defined calculation. The limit was scope, not capability.

When models disagree

If the models were simply accurate, they would agree. They often don't. During the 2017 TCJA debate, four institutions scored the bill and landed a factor of five apart: the JCT's static estimate of $1.46 trillion in revenue loss, Penn Wharton's $1.8-to-$2.2-trillion dynamic range, the Tax Foundation's far more optimistic $448 billion, and the Tax Policy Center's distributional work, broadly consistent with the JCT on revenue. All four employed competent economists and microsimulation. Why the gap?

Three sources account for most cross-model disagreement. The first is data: the JCT uses confidential IRS returns, the most comprehensive source available, while the TPC works from less granular public-use files and others from different combinations again—and different baselines yield different reform estimates. The second is behavioral assumptions: a static model assumes no response, dynamic models add some mix of labor-supply, saving, and investment effects, and the Tax Foundation's larger assumed growth response against Penn Wharton's smaller one drove the gap between $448 billion and $2.2 trillion more than anything else. The third is modeling choices: how to handle income shifting between corporate and individual returns, how to project growth, how to treat expiring provisions—each defensible on its own, and compounding together.

The disagreements are not failures. They are evidence that policy analysis rests on judgment—about data, behavior, and method—that no amount of technical polish removes. The value of running several models is that the disagreement becomes specific: not "who is right?" but "which assumptions differ, and which do I find more plausible?" The model does not settle

the argument; it relocates it, from a clash of conclusions to a clash of stated premises—the only kind of policy argument that can actually be adjudicated. Without the models the debate would be pure assertion; with them, it is at least tractable.

Where models fail

The failures are instructive, and they cluster. Static models assume people don't change behavior, but tax changes trigger income-shifting and benefit changes move labor supply—the TCJA set off a surge of pass-through income as high earners restructured to claim the new 20 percent deduction, a shift no static model saw coming. Take-up is a second fault line: models often assume people claim what they are eligible for, when EITC take-up runs around 78 to 80 percent, SNAP around 82 percent nationally with wide state variation, and SSI for eligible elderly individuals perhaps as low as 50 to 60 percent. Get take-up wrong and both program cost and poverty reduction go with it.

Data limitations, as Meyer's work shows, sit underneath all of it, and administrative records bring their own gaps: non-filers missing from tax data, a two-year lag before IRS files are research-ready, no common identifier to link records cleanly across agencies. And structural change defeats models calibrated on the past—the ACA exchange miss reflected market dynamics history couldn't anticipate, just as the 2021 Child Tax Credit expansion produced take-up patterns among non-filers that no prior model would have forecast. These are also where microsimulation earns its keep, because they name the failure precisely enough to fix it, which vaguer methods never do.

The AHCA counterfactual

In 2017 the CBO estimated that repealing the ACA under the American Health Care Act would cost 23 million people their coverage over a decade. The prediction was never tested; the bill failed. But the number forced a conversation that would not otherwise have happened—*which* 23 million? Low-income, rural, elderly?—and the specificity of the model, even unproven, structured the argument. That is both the power and the boundary of these tools: they cannot tell you with certainty what an unenacted policy would have done, but they can tell you who would have been affected, and through what mechanism, which is more than assertion ever manages. A counterfactual can never be checked against reality—that is what makes it a counterfactual—so its only claim to trust is the verifiability of its parts: whether each rule it applied matches the statute, and whether the population it applied them to matches the country. The whole may be untestable; the pieces need not be.

Better than the alternatives

The accuracy question has a second half: compared with what? Before microsimulation, policy analysis ran on rules of thumb ("a 10 percent tax cut pays for itself"—the TCJA data says otherwise), on expert judgment (which Tetlock showed forecasts little better than chance), on partisan assertion, and on back-of-the-envelope arithmetic that applied average rates to aggregate income and missed the distribution entirely. Against that field, microsimulation's demand for specificity is the whole point: which families, how much, through what mechanism.

The 2021 expansion of the Child Tax Credit shows what the specificity buys. Without a model, the debate is "does a bigger credit help families?" With one, it becomes precise: the expansion would cost roughly $105 billion a year, cut child poverty by about 40 percent, and deliver its largest gains to the lowest-income families, who had received little or no credit because they owed no federal income tax. Those claims were checkable—and were checked. Census data showed child poverty falling from 9.7 to 5.2 percent in 2021, Columbia's poverty center tracked the monthly declines as the payments went out, and when the expansion lapsed and the credit reverted, child poverty climbed back. The models had the direction, the magnitude, and the distributional shape roughly right—more than any alternative delivered. They did not nail everything, missing some non-filer take-up and the macroeconomic cross-currents of a credit expansion during a pandemic recovery. Weather forecasting is the right mental model: tomorrow's forecast is reliable, next week's roughly right, next month's a best guess, and the responsible move is to calibrate confidence to the horizon rather than abandon the forecast.

What the models are for

George Box's line—"all models are wrong, but some are useful"—is not cynicism but epistemic hygiene. The modeler who believes the model captures the whole truth is more dangerous than the one who knows its limits. The honest verdict on microsimulation is exactly as unheroic as it sounds: approximately right, sometimes wrong, better than the alternatives, and always improvable. Practitioners live with that tension daily—they know the surveys undercount transfers, that behavioral responses resist modeling, that their confidence intervals should be wider than their point estimates admit—and they publish anyway, because approximate knowledge beats ignorance.

What separates responsible microsimulation from false precision is not accuracy; it is verifiability. When the CBO publishes a score, it publishes its methodology and its track record. When an open model like PolicyEngine or Tax-Calculator produces an estimate, anyone can read the code, find the assumptions, and propose a fix. That points at the discipline this whole book is built to enforce, and it is worth stating as a rule.

A simulation is admissible only where its verification chain terminates in ground truth. The three levels that opened this chapter are the terms of admission: a component check against the statute, an aggregate check against administrative totals, a forecast graded when the

official number finally lands. Where that chain holds, a number can be trusted no matter who produced it. Where it breaks, the number is assertion wearing the costume of arithmetic, and no amount of computational sophistication redeems it. Everything the rest of this book describes—rules encoded by machines, populations synthesized from surveys, forecasts posted with intervals—is asked to pass that same test. It is a demanding rule, and it disqualifies a great deal: most of what passes for confident analysis cannot say where its chain of checks bottoms out in something an outsider could verify. But it is the only rule that lets a reader trust a number produced by a machine, or a stranger, or an institution with a stake in the answer.

It begins, though, somewhere much smaller than a national model: with one household, one benefit cliff, and the frustration that turned a family's spreadsheet into a reason to build any of this at all.

Chapter 4: A wall of frustration

Simulation as a tool

In 2008 I took a course at UC Berkeley that would end up shaping my career: IEOR 131, Discrete-Event Simulation. The premise was elegant. You model a complex system not as a set of equations but as a collection of individuals moving through states—simulate a hospital emergency room by tracking each patient's arrival, each nurse assignment, each treatment decision, and then change something, add a nurse or reorganize triage, and watch how the whole system responds. Instead of writing an equation for the average wait time, you built the room and let the wait time emerge.

The course ran on Excel and Visual Basic for Applications, and its examples were resolutely operational: healthcare facilities, manufacturing lines, service queues. But the conceptual frame stuck with me long after the syntax faded. You could understand the emergent behavior of a system by simulating its individuals rather than modeling its aggregates—and the behavior you got out that way was often one you would never have guessed from the averages going in.

My first internship put the idea to work. At Finelite, a lighting-fixture manufacturer in Union City, California, I used Matlab to simulate production decisions. Should the plant pre-cut wire to standard lengths or cut it to order? How should the assembly lines be sequenced to minimize changeover time? The questions were small and the models were simple, but they worked—computational experiments that surfaced non-obvious facts about how a factory actually behaved, facts the plant managers recognized once the model showed them but hadn't been able to see from the spreadsheets alone.

After two years in consulting, I joined Google's People Analytics team in 2010—a group whose premise was that decisions about people deserved the same standard of evidence the rest of the company brought to decisions about products.

Project Lorenz

In 2012 a colleague and I founded a small data-science team inside People Analytics. We built tools across natural-language processing of employee feedback, social-network analysis of the org chart, and—my piece—simulation models for workforce planning.

Google's staffing group processed between one and two million job applications a year. Hundreds of recruiters managed thousands of open roles across divisions with very different hiring needs, and leadership wanted to project headcount growth against targets in a way that accounted for recruiting capacity, candidate pipeline dynamics, internal mobility, and attrition. The existing tool was a set of spreadsheets: functional, but blunt. I talked leadership into letting me try something different, a bottoms-up simulation I called Project Lorenz, after Edward Lorenz, whose pioneering weather research had shown how micro-level dynamics could drive macro-level phenomena.

I built it on microsimulation principles, though I didn't yet know to call them that. Model candidates entering through different channels; estimate the probabilities of moving between hiring stages using survival models—the same statistical machinery health researchers use to predict disease progression, repurposed here to predict how a candidate moved through Google's pipeline; account for variation in recruiter productivity, for attrition, for transfers between divisions. I implemented it in R, used Monte Carlo methods to quantify the uncertainty, and had it produce not a single number but a distribution of outcomes—a spread that captured how genuinely uncertain hiring dynamics are, rather than pretending to a false precision.

Project Lorenz never fully materialized. The complexity proved hard to manage; there were too many moving parts, each reasonable on its own but producing unstable results once they were wired together, and I could never quite get the integrated model to behave—a small change to one transition probability would ripple through the whole thing and hand back a headcount forecast that no one believed. We ran the spreadsheets for another cycle. But the conceptual seed was planted, and it survived the failure: complex social systems could be understood by simulating individuals and their transitions through states, even when a particular attempt to do so fell apart. The idea was right even where my execution wasn't.

The personal motivation

My interest in economic policy became personal through my brother Alex.

In June 2004—a month after my high school graduation, two months before I left our Menlo Park home for UC Berkeley—Alex suffered a spinal cord injury in a mountain biking accident.

He was sixteen; I was seventeen. He became a quadriplegic, dependent on attendant care for the daily mechanics of living, from cooking and cleaning to getting in and out of bed.

Like me, Alex went to Berkeley, earning an undergraduate degree and then a master's in public policy. And when he entered the workforce, our family ran headlong into the question of how his benefits would interact with his earnings.

Medicaid covered his In-Home Supportive Services—attendant care that would otherwise have cost tens of thousands of dollars a year. But Medicaid had an income limit. If Alex earned more than roughly $70,000, he would lose eligibility; his medical expenses would then become tax-deductible, but the trade was brutal, and he would have had to earn something like $160,000 for the added income to make up for the coverage he had lost. Across that whole range, the effective marginal tax rate exceeded 100 percent—the arithmetic our spreadsheets kept producing, no matter how many times we rebuilt them looking for a way through. Earning more would leave him worse off, and the system offered no way around it.

We modeled scenario after scenario, and the complexity was overwhelming: tax brackets, benefit phase-outs, deductions, the interaction of state and federal programs, each rule sensible on its own and the combination punishing. The tools to understand how policy actually landed on one person's particular circumstances simply did not exist; we were building them from scratch, badly, in a spreadsheet, for an audience of one.

And none of it was visible from the outside. A poverty statistic or an average tax rate would have shown nothing of what Alex faced; his cliff lived in the interaction of specific programs at a specific income for a specific person, and you could only see it if you modeled that person directly. The aggregates were silent about exactly the thing that was reshaping my brother's decisions about whether to work.

This was my introduction to what economists call means-tested benefit cliffs and implicit marginal tax rates. For the person living inside the system, it was just a wall of frustration.

The UBI thread

Around the same time, the conversation inside Google was turning to technological unemployment, to what artificial intelligence might do to the labor market, and to whether society needed new institutions to guarantee basic needs as automation advanced. Some people were talking about universal basic income—unconditional cash payments that could put a floor under people without the means-testing that produced cliffs like the one Alex faced. For someone who had just spent months watching a means-tested program turn earning into a trap, the appeal was both obvious and personal: a floor you could not earn your way off of.

In 2012, Google.org gave GiveDirectly a $2.4 million Global Impact Award. The organization was making unconditional cash transfers to extremely poor households in Kenya—families living on roughly a dollar a day receiving about a thousand dollars, no strings attached—and, as economists, its founders were running randomized controlled trials to measure what actually

happened when you did that. The results were encouraging: gains in earnings, in assets, in nutrition, in educational outcomes. I volunteered with them on the side, helping them use their data more efficiently and hosting their researchers for talks at Google, and the more time I spent with the evidence the more the cash-transfer model came to feel like a clarifying thought experiment. Instead of targeting help through complex eligibility rules that taxed people for earning more, you could simply give people cash and pay for it through explicit taxes.

The point was never that UBI was necessarily the optimal policy. It was that UBI worked as a benchmark—a clean reference case against which to reason about the tradeoff between targeting, which is cheaper but builds in disincentives and cliffs, and universality, which costs more but is simpler and has no cliffs at all. It was the same move I had learned in that Berkeley simulation course, really: hold one thing constant, vary another, and watch what the system does. Alex's cliff was a data point; UBI was the counterfactual that made the data point mean something. Once you had that reference case in mind, the existing system's strangest features became easier to see for what they were.

The policy turn

In 2015 I moved to YouTube's data-science team, working on growth models, experiment analysis, and the launch of YouTube Go, a product built for markets with poor connectivity and lower-end phones, primarily in India and sub-Saharan Africa.

But my attention kept drifting to policy. Proposals to expand the Child Tax Credit were gaining traction; Senators Michael Bennet and Sherrod Brown had introduced the American Family Act, and I wanted to know what a larger child allowance would actually do to poverty rates, and who would gain. These were answerable questions—the kind with a number at the end—and yet there was no obvious place a curious person could go to get the number. So I went looking for a tool, and I found one: Tax-Calculator, the open-source model of US federal income and payroll taxes that had come out of the American Enterprise Institute. It was written in Python, maintained by economists with decades of experience, and—this was the part that changed things for me—its code was on GitHub. Anyone could read the formulas, run the model, and check the results.

So I did. I started using Tax-Calculator, then contributing to it, then spending my evenings and weekends on policy analysis with open-source tools while working full-time at YouTube. The realization crept up on me and then would not leave: serious public-policy analysis, the kind that moves millions of people and billions of dollars, could be done with open-source software, from a laptop, without leaving to join a think tank or a government agency. It didn't require the machinery behind the curtain. It required a good model, public data, and the willingness to do the work. This was how policy analysis *should* work.

The UBI Center

In 2018 I took three months off from Google to work on policy full-time, and enrolled in MIT's MicroMasters program in Data, Economics, and Development Policy—graduate-level coursework that could ladder into a full master's degree if I decided to pursue it. The three months were clarifying. I worked with the Open Source Policy Center at AEI, contributed to Tax-Calculator's technical infrastructure, and ran distributional analyses of tax reforms, and the work felt more important than anything else available to me. Leaving Google was not an obvious move—it was a good job, and the thing I was leaving it for did not yet exist, no organization to join, no salary, only a conviction and a set of half-built tools—but the three months had made the choice feel less like a risk than like an admission of where my attention had already gone. I returned to YouTube briefly, then left Google in July 2018, after eight years, to commit to independent policy research, supported by savings and the growing conviction that open-source policy analysis was infrastructure the world needed and mostly didn't have.

In 2019 I founded the UBI Center, an explicitly open-source think tank focused on universal basic income policy, with all of its code and data public so that anyone could verify the findings. The first problem announced itself immediately. A $1,000-a-month basic income costs something like $3 trillion a year, and to fund a number like that you have to change taxes or benefits—which means that to model UBI honestly you have to model the entire tax-and-benefit system, not because a basic income necessarily interacts with the other programs, but because you have to show what would pay for it. And UBI doubled as a lens on the existing system. Without it, low-income families faced implicit marginal tax rates above 50 percent from stacked phase-outs, and hit cliffs where earning a little more meant losing thousands in benefits. Setting universality beside means-testing was a way to make those features visible: it gave you a comparison point, a version of the system with the cliffs deliberately removed, so you could see exactly what the cliffs were doing.

Tax-Calculator could handle federal income and payroll taxes, but not benefits like SNAP or Medicaid. OpenFisca offered a framework for encoding rules, but OpenFisca-US was young and incomplete. Neither had an interface that would let a non-programmer explore a policy for themselves. And for state-level analysis—which mattered, because so many UBI proposals were state-specific—the tools barely existed at all.

The first researcher I recruited was Nate Golden, a middle-school math teacher in Washington, DC, who cared about fighting poverty with evidence and would later found the Maryland Child Alliance to push child-poverty policy at the state level. The second came from the internet. I had posted on the Basic Income subreddit asking for help, and Nikhil Woodruff, a college student in the UK, replied; he turned out to have a rare pairing of genuine economic-policy interest and real software-engineering skill. (Footnote: As of July 2026, Nikhil Woodruff serves in the UK government at 10 Downing Street. I note it here as a disclosure—much of what follows in this book touches UK policy and the institutions that model it—and because the shape of the thing is hard to improve on: the co-founder I recruited from a subreddit now works inside the government whose analytical machinery these tools were built to open up.)

The whole operation, in those first months, amounted to a few people—a math teacher, a college student from a message board, me—working nights and weekends on a model of the American tax-and-benefit system. It did not look like infrastructure. It looked like a hobby with unusually high stakes.

The UBI Center began producing real work—analyses of Andrew Yang's Freedom Dividend, of carbon-dividend designs, of child allowances—but every one of them required cobbling partial tools together, writing custom code, and working around the gaps in what already existed.

And I kept hitting walls. I'd want to model a policy and discover that nobody had encoded the relevant benefit program. Or the model existed but hadn't been updated in years. Or it worked but demanded expertise I didn't have to run it. I would set out to answer a simple-sounding question—how would this state credit change child poverty?—and lose a week discovering that the credit interacted with a benefit no open model had ever encoded, and that I would have to encode it myself before I could even begin to answer. The frustration was not peculiar to UBI research; anyone trying to analyze a cross-cutting reform ran into the same barriers, because the open-source revolution had produced components—a tax model here, a benefits framework there—but nobody had assembled them into something an ordinary researcher, let alone an ordinary citizen, could actually use.

And there was no open-source model of the UK tax-and-benefit system at all. If Nikhil and I wanted to do comparative UBI analysis across the US and the UK, we would have to build the UK model from scratch.

So we did.

By 2020 the UBI Center had grown to ten researchers, our biggest year, and we had stopped only writing reports and started building infrastructure: microdf for analyzing survey data, openfisca-uk for simulating UK policy, two of our Python packages accepted into the Policy Simulation Library catalog. Building openfisca-uk meant encoding an entire national system that no one had put in the open before—Universal Credit, the personal allowance, the benefit taper—line by line from the legislation and the guidance, and testing it against every case we could check by hand. It was slow, unglamorous work, and it was the first time the UBI Center felt less like a research shop than like a builder of the infrastructure its research kept needing. Golden worked out whether a basic income should target adults, children, or both, finding child allowances the most effective for poverty reduction up to a given budget; Nikhil built on the UK Family Resources Survey while I worked on enhancing the US Current Population Survey. And we tried to be honest about the numbers even when they undercut the case we might have been expected to make. Our analysis of Yang's proposal showed a 74 percent reduction in poverty, but it also showed that the plan would cost about $2.8 trillion a year while his five proposed taxes would raise only $1.2 trillion, leaving a $1.6 trillion gap. Our carbon-dividend work found that a £100-per-tonne UK carbon tax would cut poverty by 14 percent and deep child poverty by 33 percent. We weren't advocates. We were modelers, showing the tradeoffs and letting the numbers say what they said.

But the deeper problem never went away: all of it took Python. The work was open-source in principle and inaccessible in practice to exactly the journalists, advocates, and policymakers who needed it most. The code was public; the ability to use it was not.

The frustration, I finally understood, was infrastructure-shaped. The tools were fragmented—tax models that didn't talk to benefit models, federal systems that didn't connect to state ones—and running any of them meant installing software, preparing data, and writing code. The gap was not a missing model. It was that no one had assembled rules, data, and an interface into a single thing that a person without a programming background could sit down and use to ask a real question and get an answer they could trust. Rules, data, and an interface, welded into one instrument and made reliable enough to believe—that was the whole of it, and it had gone unbuilt because each piece belonged to a different discipline and a different institution, and putting them together was nobody's job. That was the wall, and it was the same wall my family had hit around Alex's kitchen table, scaled up to a country. Someone would have to build the thing that tore it down.

Part II: The open engine

Chapter 5: Proof of concept

The state of play, 2019–2021

For a few years around the turn of the decade, open-source policy modeling went through a quiet boom—and I spent most of it unable to answer a simple question.

In October 2019, UKMOD became the first freely available tax-benefit microsimulation model of the United Kingdom. Britain already had proprietary tools—the Institute for Fiscal Studies had run TAXBEN since 1983, the Treasury had IGOTM, its internal tax and benefit model, the Department for Work and Pensions had its Policy Simulation Model—but UKMOD, built on EUROMOD's UK component and funded by the Nuffield Foundation, was open to anyone who asked.

Then COVID-19 arrived, and everyone needed policy analysis at once. The CARES Act, expanded unemployment insurance, recovery rebates—the rules changed weekly. NBER updated its TAXSIM calculator for the new provisions; Tax-Calculator shipped a pandemic release, version 3.2.1, in August 2021. In December 2020, EUROMOD—the backbone of European tax-benefit modeling for two decades—went fully open source, with the European Commission's Joint Research Centre taking over the following month. The International Microsimulation Association convened a conference that December devoted entirely to pandemic policy responses, and the open-source Policy Simulation Library kept adding models. The ecosystem was growing fast, and almost none of it was usable by anyone who couldn't write code.

That was the gap I kept falling into. I had started the UBI Center in 2019 out of a specific frustration: I wanted to know what various universal basic income proposals would cost and whom they would help, and no public tool could answer with any specificity. The UBI Center published open-source analyses of basic income proposals using whatever existed—mostly Tax-Calculator and custom Python scripts running on Census survey data.

The work taught me two things. The tools existed only in fragments. Tax-Calculator handled federal income tax beautifully but ignored benefits; TAXSIM handled state taxes but ran in batches; nothing captured the full interaction between taxes and benefits that decides whether a basic income comes out progressive, regressive, or poverty-reducing. Fund a basic income by tapering an existing benefit, for instance, and you could lift some families while stranding others at a cliff—and no available tool would show both effects at once. And the audience for policy analysis was far larger than the audience that could run the tools. When I published a UBI Center analysis, people wanted to explore it themselves—vary the parameters, change the household, model the version their local group was actually pushing—and they couldn't, because the analysis lived in a Jupyter notebook.

Building the application

By late 2020, Nikhil Woodruff and I had a working microsimulation model of the UK tax-and-benefit system, built at the UBI Center. Nikhil was an undergraduate at University College London reading mathematics and computer science, and the stronger engineer of the two of us; he could turn a tangle of policy rules into clean, fast code. Our model—OpenFisca UK—could calculate taxes, simulate benefits, and estimate the effect of a reform. But it lived in Python scripts only a programmer could run.

We had seen that pattern everywhere: powerful tools locked behind technical barriers. TAXSIM wanted batch files; Tax-Calculator wanted Python; UKMOD was built for academics; TAXBEN was proprietary. The fix was obvious and ambitious—put the full power of microsimulation in a browser, so that anyone could design a reform, see its cost and its distributional effect, and check what it meant for their own household, without writing a line of code.

We called it PolicyEngine.

The hard part was turning a fundamentally computational process into something that felt like a form you could fill out. We built it in layers. **The model layer** ran in Python on cloud servers: feed OpenFisca UK a household and a reform, and it returned the exact tax and benefit before and after the change. **The API layer** wrapped that model in HTTP endpoints, so the interface could improve without touching the rules and the rules could improve without touching the interface. **The application layer** was a React web app that let users describe a household by answering questions—income sources, number of children, housing costs—and specify a reform by moving sliders and toggles, then turned those choices into API calls and drew the results.

The architecture encoded one decision that mattered more than any other: the model had to be usable without the interface. The API was the product, and the web app was merely its first consumer. Anyone—a researcher, a journalist, another program—could call the same engine that ran the website. At the time it felt like overengineering. In retrospect it was the choice that made everything else possible, because when developers later wanted PolicyEngine's numbers inside their own tools, the API was already there.

Nikhil handled most of the engineering—API design, the React front end, state management—while I worked the policy logic: which parameters to expose, how to present results, what comparisons mattered. We argued endlessly about how much complexity to show. Exposing every parameter in the tax-benefit system would drown a user; there are thousands. Hiding too many would gut what anyone could model. We settled on a compromise—a curated set of frequently debated parameters, income tax rates and benefit levels and credit amounts, with the option to reach any parameter in the model for users who knew what they were after—and on progressive disclosure for the output: headline numbers first, net cost and poverty change, then breakdowns by income decile, household type, and region for anyone who wanted to dig. The output was contested for the same reason as the input—show a household its budget change, its poverty impact, and its place in the distribution all at once and the screen collapsed into noise—and deciding what to lead with was an argument we re-ran for every view.

We launched PolicyEngine UK in September 2021. Enter a reform—raise income tax, increase child benefits, introduce a carbon dividend—and see the effects: the cost to the Treasury, the change in poverty, the shift in inequality, and, critically, what it would mean for your own household. Within weeks, advocacy groups were using it to design proposals, and when the Chancellor's Autumn Budget changed Universal Credit, we published distributional analysis within a day.

In October 2021 we spun PolicyEngine out as a separate nonprofit, and the mission statement shifted from "make everyone a policymaker" to "help people understand and change public policy." The second verb was the point: understanding is passive; changing takes agency. Incorporating as a 501(c)(3) was itself a design choice about trust. We wanted a progressive group modeling a basic income and a conservative group modeling a flat tax to have equal confidence the engine wasn't tilted toward the other—so the code stayed open for anyone to inspect, and the organization had no shareholders whose interests could bend the results. Behind it were two people, a rotating cast of volunteers and interns, and a website that kept going viral faster than we could fix the bugs the attention exposed. When the Autumn Budget analysis took off, we weren't a media operation—we were two people with a model, fielding interview requests with one hand and patching the code the traffic had broken with the other.

Crossing the Atlantic

We launched PolicyEngine US in March 2022, and it was a different kind of problem—not just different policies, but a different shape of complexity. The UK has one national tax

system, though Scotland sets its own income tax rates, and Universal Credit had folded most means-tested support into a single program. The US has fifty states with separate income tax codes layered on a federal system of bewildering complexity, and its benefits are scattered across agencies: SNAP at the USDA, SSI at the Social Security Administration, Medicaid at HHS, housing vouchers at HUD, childcare subsidies at the states. Each program defines income its own way, counts the household its own way, sets eligibility its own way. Modeling the US wasn't harder than modeling the UK by some constant factor; the complexity grew multiplicatively rather than additively.

We started with what we could model—federal income and payroll taxes, SNAP, the EITC—and shipped the household calculator first. In July 2022 we added population-level impacts using the Current Population Survey, the same microdata foundation behind official government estimates. State coverage came one state at a time. Some states piggyback on federal adjusted gross income and fall into place once the federal model exists; others, like New York with its multiple income tax schedules, city taxes, and supplemental credits, were miniature ecosystems of their own. Each had its own structure, its own EITC variant, its own quirks, and what a contributor learned encoding one rarely transferred to the next. By the end of 2022 we had modeled six states. Forty-four remained.

The gap between ambition and capacity was constant. We wanted comprehensive coverage and had a small volunteer team encoding rules on weekends. We wanted models that updated the moment a policy changed and struggled to keep them current—a contributor might encode Oregon's working-family credit only to find the legislature had amended the eligibility rules mid-session, with no updated documentation to work from. The contributor model was open source in the truest sense: anyone could submit a pull request encoding a state's rules, and our team reviewed it for accuracy and code quality. That worked where a local expert cared enough to volunteer, and failed where no one did, which left coverage that tracked contributor geography rather than policy importance. The proof of concept held—people wanted accessible policy analysis—but scaling it would take more than enthusiasm and weekends.

Who used it

The most revealing users were the ones who changed how they worked because the tool existed. A congressional aide could use PolicyEngine to check whether a proposed child tax credit expansion would actually reach the families a member cared about—finding, say, that a credit phased in over the first few thousand dollars of earnings missed exactly the poorest households the office meant to help—and then use that to reshape the legislative language before requesting a formal score from the Congressional Budget Office. The informal tool didn't replace the formal process; it changed the questions that arrived at it. Instead of leading with "what does this cost?", staff could walk into CBO with a proposal already refined for distributional impact.

The other pattern was democratization. Advocacy groups and academic researchers who had relied on government estimates or expensive consultants could now run their own simulations—test a specific variant, compare alternatives, model the reform their organization was actually proposing—and argue with numbers rather than narratives. A volunteer-run outfit like UBI Lab Northern Ireland could model a recovery basic income on its own, without commissioning a think tank; an organization that had once needed a consultant's retainer to put a number on a proposal could now generate its own distributional table before a meeting. And increasingly, developers built on the API rather than the website: the Fund for Guaranteed Income, for instance, wired PolicyEngine into a tool that showed cash-pilot participants how their other benefits would change as their income rose. We had set out to build a product and were turning into infrastructure.

The validation challenge

Open source meant anyone could inspect the code, but inspection is not correctness. We invested heavily in validation, comparing PolicyEngine's output against official calculators and published statistics, and the results were mixed in an instructive way. Each model carried a test suite checking outputs against official calculations across hundreds of household scenarios, and every discrepancy raised the same three-way question: was the bug in our code, in the official calculator, or in our reading of an ambiguous rule?

Some were simple. An outside contributor comparing our results against IRS tables caught a transposed phase-out rate—21.60 percent entered where the EITC's two-child schedule specifies 21.06 percent. Others were genuinely ambiguous. When a regulation says income is figured "net of applicable deductions," which deductions apply? Different official sources sometimes gave different answers, and we kept a running log of these cases—places where reasonable readings of the same statute produced different numbers. The log was itself a finding: the tax-benefit system is complex enough that the agencies administering it sometimes disagree about its own rules.

For the UK model, HM Treasury eventually ran its own evaluation and found that 60 percent of National Insurance calculations fell within 0.5 percent of its internal model. Income tax proved harder to compare—the two systems had trouble even identifying which variables corresponded—and edge cases surfaced everywhere: housing benefit interacting with legacy benefits, Scotland-specific provisions, transition rules for households moving between programs. The lesson was that validating against a government model is hard not because the arithmetic is difficult but because two systems can define "income," aggregate units, and handle household composition in quietly incompatible ways.

And we volunteered the limitation we couldn't code our way out of. PolicyEngine ran on household survey data; governments run on administrative tax records. Surveys under-sample the very rich—the CPS might hold a few hundred households above $500,000 where the IRS has vastly more—and because high earners generate a disproportionate share of revenue, that

sampling gap cascaded into totals. Our revenue estimates ran about a third below official ones. This was not a bug we could fix in code; it was a structural constraint of survey-based microsimulation, one that only better data could ease. So we learned to lead with relative impacts—how much a reform raised or lowered revenue against current law—because the survey's sampling error moved baseline and reform alike and largely cancelled. Sophisticated users still wanted absolute numbers, and we couldn't always explain the gap with CBO without a short lecture on survey methodology.

From tool to infrastructure

In 2024 the National Science Foundation awarded PolicyEngine a Pathways to Enable Open-Source Ecosystems grant, a sign that open-source microsimulation had come to count as infrastructure worth funding. That year we finally closed the gap that had frustrated me at the UBI Center: in April 2024, PolicyEngine launched income tax modeling for all fifty states and DC, the work of more than a hundred open-source contributors parsing the country's patchwork of tax codes. The final push was a campaign in itself—finding a contributor with local expertise for each missing state or recruiting an engineer willing to parse an unfamiliar code, then validating each implementation against whatever official calculator or published table existed. Flat-tax states with a handful of credits took days; New York, California, and Hawaii took weeks of iterative debugging. What I couldn't find in 2019, we had built by 2024.

The data improved in parallel. In August 2025 we launched the Enhanced Current Population Survey, integrating five datasets and calibrating to 9,168 administrative totals; the pipeline paired Quantile Regression Forests for imputation with gradient-descent reweighting, and cut our deviations from official statistics by 97 percent.

That data line was later rebuilt as populace, the open commons of calibrated microdata this book returns to.

The strongest validation, though, came from who wanted to build alongside us. In September 2025 we signed a memorandum of understanding with the National Bureau of Economic Research to build an open-source emulator of TAXSIM, the tax calculator that had powered academic research since the 1970s; more than 1,200 papers had relied on it. Daniel Feenberg, TAXSIM's creator, joined our advisory board and served as external mentor under the NSF grant. We were building the emulator partly to guarantee researchers kept these capabilities, and partly to build something larger—a framework in which several independently developed models could cross-check one another.

A month later, we signed a second MOU with the Federal Reserve Bank of Atlanta. Their Policy Rules Database covered benefits—SNAP, Medicaid, housing vouchers, childcare subsidies—complementing TAXSIM's focus on taxes. The database powered the Atlanta Fed's CLIFF tools, which Colorado's Workforce Development Council and New Mexico's Caregivers Coalition used to help families understand how earning more would affect their benefits.

The three-way validation approach was deliberate. When independently developed models agree on a calculation, researchers can trust the result. When they diverge, the differences reveal important questions about policy interpretation or implementation details. Having three models—PolicyEngine, TAXSIM, and the Policy Rules Database—helped isolate where discrepancies originated.

We were positioning PolicyEngine as more than another microsimulation model—as an integrator that could cross-validate across Python and R, across taxes and benefits, across academic and government implementations.

And then came the UK.

In March 2025, HM Treasury published an Algorithmic Transparency Record disclosing that it had piloted PolicyEngine UK as a possible supplement to IGOTM, the closed model that had produced government policy estimates for decades. The Treasury singled out our machine-learning approach to survey error—combining multiple years of the Family Resources Survey, imputing missing income with random forests, reweighting households with gradient descent—the same techniques we had built for the Enhanced CPS, now under evaluation inside the government. The Times reported that the tool could more accurately predict the outcomes of ministers' decisions and was expected to be used mainly in budget periods. Nikhil Woodruff, PolicyEngine's co-founder, has since joined the UK government at 10 Downing Street —the co-founder I had recruited off a subreddit, now working inside the machinery an outside tool had helped pry open.

The significance ran past validation. Government microsimulation had been closed by default—proprietary code, restricted data, results that could be doubted but not reproduced. An outside model good enough for the Treasury to pilot put pressure on that default: if an open tool could even partly reproduce the government's internal estimates, and the government thought it worth testing, the case for keeping the models closed weakened. In 2019 I couldn't find a tool to answer a basic question about a universal basic income; by 2025, HM Treasury was piloting one we had built.

Rules, data, and prediction

By late 2025 the tool was real and still plainly unfinished—household calculations sharp, unusual cases still exposing gaps, revenue still running below official totals. But the most useful thing building it gave us wasn't a feature; it was a distinction. A policy question that looks like a single question is really three, and they come apart cleanly the moment you try to compute them. What does the law say—the exact tax owed and benefit due for a specific household, in a specific place and year? Who are the people—how many households look like that one, and how do they sum to a country? And what will happen—how will people respond, how will the economy move, how will next year differ from this one?

Each of the three is answerable, and each is checkable, but against a different kind of truth. An encoded rule is right or wrong against the statute, and against another calculator built from the same statute; you can prove it to the dollar. A population estimate is right or wrong against a survey and against administrative totals; you calibrate it and measure what remains. A prediction is right or wrong only when the future arrives and the official number lands; you commit, you wait, and then you grade. Three concerns, three verification loops, three different clocks.

For years we ran all three inside one engine and one organization, because that was how the tool had grown. But the seams were already showing: the revenue gap was a data problem, not a rules problem; the rules stayed exact precisely where the data was uncertain; the behavioral responses belonged to neither. What we had built as one thing was, underneath, three things—each buildable, each checkable, each trustworthy on its own terms. Pulling them apart would come later. First, though, the engine has to show what it can do: for a single household, and then for a country.

Chapter 6: The household and the society

Consider a single parent in New York with one child who has a disability, earning $30,000 a year. How much do they owe in taxes? How much do they receive in benefits? What happens if they take on an extra shift?

These are not abstract questions. They decide whether the parent can make rent, whether a raise is worth chasing, whether a job across town pays off once it changes their benefits. The answers run through federal income tax, state income tax, payroll taxes, the Earned Income Tax Credit, the Child Tax Credit, Supplemental Security Income, SNAP, and a dozen other programs — each with its own definition of income, its own household unit, its own phase-out schedule, none of them designed to fit together.

No one can calculate this in their head. Most accountants would struggle. Yet the answer shapes every financial decision the family makes. This is the problem PolicyEngine's household calculator was built to solve — and it is where the engine's work begins: with what it can show one family, before it can show anything true about a country.

What you owe, what you get

A typical low-income family might draw on the EITC, a partially refundable Child Tax Credit, SNAP, SSI for a disabled member, Medicaid, a housing voucher, a state child-care subsidy, and free school meals at the same time — eight programs, each written independently, each with its own idea of income and its own household unit, accreted over decades of legislation with

no coordinating hand. The result is a system no participant fully understands. Caseworkers specialize in one program. Tax preparers work one side of the ledger. Benefits counselors know eligibility rules but not tax consequences. No one sees the whole picture. The fragmentation is not harmless: it is why eligible families leave benefits unclaimed, why a raise can quietly cost more than it pays, and why the same decision can look smart to a tax preparer and ruinous to a benefits counselor.

Seeing the whole picture is the calculator's single promise: enter your circumstances and it shows your taxes and benefits together, under current law. It walks through the household — how many adults, how many children, what ages — then income, wages, self-employment, investment, retirement, then the specifics of housing cost, child-care expense, disability. From those inputs it computes dozens of outputs: federal and state income tax, payroll taxes, the alphabet of credits, the benefit programs a household may qualify for, and a single net figure for what the family keeps after everything. PolicyEngine's US model spans federal income and payroll taxes, the EITC and CTC, SNAP, SSI, WIC, TANF, and dozens of state programs; the UK model spans income tax, National Insurance, Universal Credit, Child Benefit, and the full run of means-tested support. Each program is a separate module, its logic drawn from statute, regulation, and agency guidance.

The comprehensiveness is the whole point, because the programs interact. A family's SNAP benefit depends on its net income after taxes and deductions; its tax depends on credits that depend on the same children who might also generate SSI or a child-care subsidy. Change one input and the effect ripples across the others. No single-program calculator captures that. The microsimulation engine does, because it evaluates every program at once for the same household. And once the whole calculation is visible, a family can act on it — plan around a raise that won't lift take-home pay as much as expected, or claim a program they qualify for and never knew to ask about.

Rates above a billionaire's

Plot a household's net income against its earnings and the hidden structure of the system becomes visible. For many low-income families the line is not a smooth upward slope. It has flat stretches where more work barely raises take-home pay, and here and there a vertical drop — a cliff — where crossing a threshold means losing a benefit worth more than the raise that triggered it.

A cliff is just that: the point where earning more leaves a household worse off, because the benefits withdrawn and the taxes owed on the next dollar add up to more than the dollar itself. Analysts have worried about cliffs for decades. But until tools like this one made them visible, most of the people standing on one never saw it coming.

The gentler version is everywhere. As the New York parent's earnings climb from $20,000 toward $30,000, several things happen at once: SNAP tapers, the EITC phases out, the child's SSI falls, and income and payroll taxes begin to bite. Economists watch marginal rates because they shape behavior — keep ninety cents of your next dollar and you might work more; keep twenty and you might not. For high earners those numbers are debated endlessly: the 37 percent top federal rate, capital gains, state tax. The counterintuitive reality is that a low-income worker often faces a higher marginal rate than any billionaire, because benefit phase-outs stack on top of taxes. SNAP withdraws about thirty cents per additional dollar of net income. The EITC phases out at roughly sixteen cents per dollar for a family with one child, twenty-one for two or more. SSI falls by about fifty cents per dollar of countable income. Stack them and the combined rate can approach eighty percent — more than double what a hedge-fund manager pays on his last dollar. This is not an edge case but the ordinary structure of a low-income budget, and it is exactly the kind of pattern that disappears into any average. At a true cliff the chart spikes past 100 percent, the unmistakable signature of earning more and keeping less. The calculator shades the earnings range where extra work barely moves net income — a dead zone made visible — and that visibility serves two ends at once: a family can plan around a cliff instead of walking off it, and a policymaker can see the work disincentives that well-meaning programs create only in combination, never one at a time.

The mechanics show up as specific traps, each one the calculator can make exact:

The SNAP categorical cliff. Receiving SSI confers categorical eligibility for SNAP — you qualify regardless of the usual income test. But once earnings push SSI to zero, that categorical eligibility vanishes and the household must suddenly satisfy SNAP's own income test. The cliff lands at precisely the earnings level where someone is trying to become self-sufficient.

The marriage penalty. Two single parents, each earning $25,000 with one child, each qualify separately for substantial EITC and CTC. Marry them and their combined $50,000 pushes both credits into higher phase-out ranges, and the household can end up with less in total transfers than the two separate households received. The penalty is written in no single line of the tax code; it emerges from several programs' different treatment of household structure, and the calculator surfaces it by running the two adults separately and then as one.

The aging-out cliff. A family receiving SSI for a disabled child hits a discontinuity when the child turns eighteen and SSI stops counting the parents' income and starts counting the child's own — often raising the SSI payment while stripping family-based benefits, as the same birthday moves the child out of pediatric Medicaid provisions, school meals, and child-care subsidies. Nudge the child's age in the model and the whole cascade appears.

The what-if machine

The calculator is not only a reader of current law. Open the policy editor and you can change the rules — raise the EITC, smooth a SNAP cliff, add a child allowance, make the Child Tax Credit fully refundable, set a universal basic income, flatten the income tax — then return to your household and watch what moves. Baseline shows in gray, reform in blue; the dead zones and the marginal-rate spikes redraw themselves, revealing whether the reform helps your specific family or quietly weakens its incentive to work. The reform space is as deep as the model is wide: someone exploring basic-income proposals can set a flat tax at any rate against a universal benefit at any amount and see at once whether the simplification leaves their household ahead or behind, and whether it smooths the cliffs or just trades them for a higher average rate. The charts update as you type — two net-income lines, two sets of dead zones, the marginal-rate curves stacked so it is immediately clear where a reform flattens the benefit system and where it carves a new cliff.

This is the "what if" ordinary people never had. What would a CTC reform mean for your family? Under 2026 law the Child Tax Credit is $2,200 per child; any reform is a change measured against that. Take the clearest recent one. In 2021 Congress temporarily raised the credit to $3,600 for children under six and $3,000 for those six to seventeen, and made it fully refundable, so families who owed no federal income tax received the full amount. For a single parent earning $15,000 with two young children, that was no marginal adjustment: the engine computes roughly $1,800 under the old rules, held down by the phase-in on earnings above $2,500, against $7,200 under the expansion. When the expansion lapsed at the end of 2021, the same family's income fell by more than $5,000. The tool shows that not as a national average but as a dollar figure for one household.

It's your life, under different rules.

The same capability changes how policy gets designed. Rather than propose a reform and wait weeks for a score, a legislative aide can model dozens of variants in an afternoon: set the benefit at $3,000 per child or $4,000, phase it in from the first dollar or from $2,500, cap it at $75,000 or $150,000, and read a different household chart for each — design trade-offs invisible in the legislative text and immediate in simulation.

Where the system is most complex

The household calculator originated in the UK, where PolicyEngine launched in 2021 before crossing to the US, and the UK version proved a rule the US one later confirmed: the household view is most powerful where the system is most tangled. Universal Credit was meant to simplify welfare by folding six legacy benefits into one; in practice its taper — the rate at which the award is withdrawn as earnings rise — interacts with income tax, National Insurance, and

council-tax support to push some working families' marginal rates above 70 percent. When Chancellor Rishi Sunak cut the UC taper from 63 to 55 percent in the Autumn 2021 Budget, the tool could show what that meant household by household: a single parent working 25 hours a week at minimum wage would keep about £1,000 more a year, while a two-earner couple would gain less — distributional detail that aggregate statistics hid. HM Treasury later published a formal evaluation benchmarking PolicyEngine UK against its own internal models: a research prototype the government took seriously enough to test against its own machinery.

From calculator to primitive

Every household calculation carries a caveat, shown plainly: PolicyEngine provides estimates, not benefit determinations. The model encodes the rules as written; an actual determination turns on discretion, documentation, asset tests, and local variation that no model fully captures. So the team validates continuously against official calculators and published tables and treats every discrepancy as a bug to investigate — while admitting that perfect accuracy in a system this complex is unattainable, and that claiming it would be worse than naming the gap. Showing the logic openly is the point: a user can judge whether the calculation matches their own situation, which a black box asserting false precision would never allow.

That honesty is what let the calculation become infrastructure. In 2025 MyFriendBen, a benefits-screening service, launched in North Carolina, Illinois, Colorado, and New York on PolicyEngine's API: plain-language questions in a dozen languages, and in about six minutes a household learns which programs it likely qualifies for. In Colorado, users surfaced an average of $1,500 a month in benefits they appeared eligible for but had not claimed. PolicyEngine's own note reported that the estimates matched expected amounts more than 90 percent of the time — a self-reported figure, not an independent audit. The engine underneath is the same whether a policy wonk runs a scenario on the website or a navigator helps a family in Charlotte. A prototype pushes it further still, reframing the whole calculation around life events — a birth, a move, a marriage, a job loss, turning 65 — because that is how people actually meet policy, not by editing tax parameters.

The household view, it turned out, was a primitive — and in two senses at once. It is the basic computation that benefits screening, financial coaching, guaranteed-income design, and legislative analysis all separately need: given this household, what are its taxes and benefits? And it is the atomic unit of ground-truth verification — the level at which an encoded rule can be proven against a reference calculator or a real benefit determination. A national estimate is nothing but a weighted sum of household calculations; if the household calculation is wrong, everything built on top of it is wrong in ways no aggregate will reveal. Getting the single family right is what makes the country-level numbers mean anything.

Which is the bridge to the society view. A macroeconomic model might say a tax cut costs $100 billion and lifts GDP by 0.3 percent. Microsimulation can say that and also tell you the same cut hands $12,000 to a high-income household in Connecticut and $200 to a low-income household in Mississippi — and that the Mississippi family now faces a higher marginal rate, because the cut nudged it into a phase-out. The aggregate is assembled from millions of those specific calculations.

From one household to many

The society view answers what the household view cannot: not "what would this mean for me?" but "what would this mean for everyone?" It has a lineage. In 1974 Joseph Pechman and Benjamin Okner of Brookings published *Who Bears the Tax Burden?*, the first comprehensive attempt to allocate US tax burdens across the income distribution using household-level data. Merging records for 72,000 households, they found the system roughly proportional across most of the distribution — a result that irritated partisans on both sides — Their finding challenged conservatives who called the system too progressive and liberals who called it too regressive at the same time, and their method of running micro-data through incidence assumptions became the template for every distributional analysis since, at Treasury's Office of Tax Analysis, the Joint Committee on Taxation, and the CBO.

The move from one household to the nation sounds trivial — run the household calculation for everybody and add it up — and in principle it is exactly that. In practice it means solving some of the hardest problems in policy analysis, starting with a plain one: you cannot survey everybody. The monthly Current Population Survey samples about 60,000 households; its Annual Social and Economic Supplement, the income workhorse most policy analysis draws on, reaches roughly 100,000 — either way a sliver of some 130 million American households. So each surveyed household stands in for many others like it, carrying a weight that says how many: a household in rural Wyoming might represent 5,000, one in Manhattan 500 — weights the Census computes to correct for the survey's sampling design and for who does not respond. To score a reform, the engine computes the change for each sampled household, multiplies by that weight, and sums across the sample — a national estimate that inherits all the uncertainty a weighted survey carries.

The data underneath

That uncertainty is the binding constraint, because the survey is flawed in ways that bias the answer rather than merely blur it. High incomes are top-coded and under-reported; benefits are

undercounted — Bruce Meyer and colleagues found that 40 to 50 percent of SNAP recipients do not report their benefits, and the problem runs across programs. The sample is too thin for most state-level work, and asset questions are sparse, so wealth policies are hard to see at all. None of this is random noise: survey-based scores tend to understate both the cost of reforming means-tested programs and the revenue from taxing top earners, because the data undercounts exactly the benefits and incomes in play.

The answer is to stop treating the raw survey as ground truth and calibrate it to administrative reality — pulling the undercounted top incomes and underreported benefits back toward the totals the tax and benefit agencies actually record. PolicyEngine's earlier public milestone was the Enhanced CPS, released in August 2025: five datasets integrated, reported taxes and benefits replaced with computed amounts for internal consistency, income distributions corrected against IRS records, and survey weights re-solved to match 9,168 administrative totals — which cut deviations from those targets by about 97 percent. That line of work has since been folded into populace, a calibrated-microdata commons that treats data construction as shared infrastructure rather than a per-project chore: synthesize records where the survey is thin, impute missing variables with quantile-regression forests, calibrate weights to administrative aggregates, and publish certified releases the way software ships versioned builds. Local-area analysis then becomes a matter of filtering one national calibrated dataset rather than building fifty bespoke ones.

Cost, poverty, and who gains

The first question about any reform is what it costs. PolicyEngine scores it by running two simulations across the weighted sample — one under current law, one under the reform — and summing the difference in tax revenue and benefit spending. The arithmetic is simple; the computation is not, since each of the roughly 100,000 records triggers thousands of eligibility checks and bracket calculations, though the whole run still finishes in seconds. Scores like these are static — they hold behavior fixed; layering in labor-supply elasticities can pull an estimate down when people are assumed to work more, or push it up when they work less, which is one reason two honest analysts can publish different numbers for the same bill.

A single cost figure hides how contingent it is. The same reform carries different price tags depending on the year you score it in, as provisions phase in and thresholds inflate; on whether you let people change their behavior in response; and, most subtly, on the baseline you compare against. Baseline is the one that swings estimates by hundreds of billions. Before July 2025, scoring an extension of the 2017 tax cuts meant first deciding whether the baseline assumed the individual provisions would expire on schedule or continue — an assumption worth hundreds of billions before any policy changed. OBBBA settled that particular question by making the provisions permanent, but it sharpened the general lesson: an estimate is only as meaningful as the world it is measured against.

Consider making the Child Tax Credit fully refundable — removing the refundable cap and the earnings phase-in so that families who owe no federal income tax receive the whole credit — scored against 2026 law, in which the credit is $2,200 per child. The net cost is roughly $23 billion for 2026, and child poverty falls by about 2.6 percentage points—on the order of 1.9 million children. The distributional picture is the opposite of what the headline might suggest: the families who gain are those whose credit currently exceeds the income tax they owe, and who therefore forfeit part of it today — low- and moderate-income working families with children. Higher-income households, already claiming the full credit, see nothing change.

Poverty is the next question, and it depends on how you measure. PolicyEngine uses the Supplemental Poverty Measure, which unlike the older official measure counts taxes and in-kind benefits like SNAP, adjusts for geographic differences in housing cost, and nets out work and medical expenses; for two adults and two children renting, the 2024 SPM threshold is about $39,400. In a 2023 analysis PolicyEngine put roughly 9.6 percent of Americans below their threshold, with children poorer than working-age adults and seniors, and women poorer than men. Apply a reform and the engine recomputes each household's resources against its threshold; run the change by demographic group and a program's uneven reach appears — it can cut child poverty sharply while barely moving the rate for seniors, or help women more than men. WIC is a concrete case: it lowers overall poverty by about 0.8 percent and deep poverty by 2.2 percent, but child poverty by 2.6 percent, and women's poverty by 0.9 percent against men's 0.7 — a program that looks gender-neutral on its face landing unevenly once the microdata is split.

Beyond poverty lies the question of who wins, who loses, and what happens to the distance between top and bottom. Sorting households into income deciles and showing the share of each that gains or loses reveals whether a reform is progressive or regressive, and the same cut can be taken by wealth decile using imputed wealth, by age, by sex, by geography, and by race where the data supports it. Inequality gets its own measures. The Gini coefficient is the standard, but it is most sensitive to the middle of the distribution, so a reform that transforms life at the very bottom can barely move it; the Atkinson index, tuned by an inequality-aversion parameter, weights the bottom more heavily and responds more to anti-poverty reforms. The measures can disagree in instructive ways — a reform can cut poverty while raising inequality by helping the near-poor more than the poorest, or shrink inequality while barely touching poverty by redistributing within the middle. A cost figure means little on its own: fifty billion dollars concentrated on families in poverty is a different policy from fifty billion spread evenly across the income spectrum, and only the distribution says which one a reform is.

PolicyEngine does not tell you which of those to prefer. It tells you what the outcomes are.

The neutrality is deliberate, and imperfect. The tool shows budget, poverty, inequality, and distribution across several cuts and lets users weigh what they value; it adds no editorial verdict, documents its behavioral assumptions and its data limits, and keeps its methodology open to challenge. But the choice of what to display is itself normative — showing a poverty rate implies poverty matters, and framing by income decile frames the debate a particular way.

The honest claim is not perfect neutrality, which is impossible, but a deliberate separation of analytical infrastructure from advocacy.

Where the numbers are solid, and where they're soft

Society-level estimates carry more uncertainty than household ones — the sample is weighted, variables are imputed, interactions compound — so it matters to say plainly where the outputs are trustworthy and where they are not.

Household calculations are the most reliable thing the engine produces, because they depend on the encoded rules rather than on the data distribution: for a family with stated characteristics, the answer is only as good as the statute encoding, and no better or worse for the survey behind it. Directional and comparative results are nearly as solid — whether a reform helps low-income families more than high-income ones, whether Reform A costs more than Reform B — because the shapes of distributions and the ratios between reforms hold steadier than their absolute levels, and the same data limits press on both sides of a comparison.

Absolute budget scores deserve the most caution. PolicyEngine's aggregate revenue estimates have historically run well below official totals — by roughly a third — because survey microdata undercounts top incomes and underreports benefits, and a static model omits the behavioral and macroeconomic feedback that moves real collections. For an order-of-magnitude figure, or a comparison across reforms, that is good enough; for a precise revenue estimate, the Joint Committee on Taxation and the CBO remain the standard, because they work from confidential tax returns that capture the whole income distribution. The calibration itself is validated by holding out some administrative targets and checking the reweighted sample against them — the same procedure that cut deviations by about 97 percent — but even a 97 percent improvement from a large gap still leaves a meaningful gap. No microsimulation matches reality exactly. The question is whether the model's limits are knowable, and open models, uncomfortably, expose theirs where closed ones hide behind institutional authority.

Analysis as infrastructure

The society view enables a kind of analysis that used to live only inside government: a response to a policy proposal while the proposal is still live. Traditional analysis takes weeks or months — a team reads the legislation, codes it, runs the model, writes it up, clears review — and by the time it publishes, the legislative moment may have passed. The model's speed comes from pre-investment: building and maintaining the engine takes years, but once it exists, scoring a

new reform takes hours, because the fixed cost is already paid and the marginal analysis is cheap. That changes who can take part in the debate, and when.

It also changes what the debate is about. When analysis is a one-time product — a report from a think tank — the argument is whether to trust that particular report. When analysis is infrastructure that anyone can run, through a web app, a Python package, or an API that other products build on, the argument shifts to whether the underlying model is sound. That is a more productive fight, because a model's quality is testable and improvable in ways a report's credibility is not. The same engine meets different users through different surfaces — a researcher iterating over thousands of scenarios in the Python package, an advocate embedding a shareable chart in an op-ed, a journalist who wants to show her work, a legislator comparing reform options side by side — and each new analysis adds an example, exposes an edge case, and compounds the value of the shared model.

Two reforms, seen clearly

The 2021 Child Tax Credit expansion is the clearest recent demonstration of what the society view catches — and of what goes dark without it. The American Rescue Plan raised the credit and, more consequentially, made it fully refundable, removing the earnings phase-in that had excluded the poorest families, and paid half of it out in monthly installments from July through December. Before enactment, Columbia University's Center on Poverty and Social Policy projected that the broader relief package could cut child poverty by more than half. After enactment, the data agreed: SPM child poverty fell from 9.7 percent in 2020 to 5.2 percent in 2021, a record low. The credit as a whole kept 2.9 million children out of poverty that year; the expansion alone accounted for 2.1 million of them — two numbers the debate routinely conflates and the microdata keeps distinct. The one-year cost ran to roughly $105 billion. The predictions landed close not by luck but because the policy worked through mechanical channels — direct cash on simple eligibility rules — where behavioral uncertainty is low and microsimulation is at its best. When the expansion expired in January 2022, Columbia tracked child poverty nearly doubling within months; the rebound was legible only because the simulation had drawn the baseline. The episode carried a subtlety worth keeping: because the expansion also raised the maximum credit for every eligible family, the distributional tables showed the largest percentage gains at the bottom but real dollar gains further up the distribution — whether that read as a broad coalition or as loose targeting depended on values the model could not adjudicate.

The UK offers the contrast that shows the method's edge. In September 2022, Chancellor Kwasi Kwarteng's "mini-budget" proposed abolishing the 45p additional rate of income tax, cutting the basic rate, and reversing a National Insurance rise, all in the name of growth. PolicyEngine UK published household-level distributional analysis within hours — among the only independent estimates available in the critical first days — showing that the package

overwhelmingly favored high earners, the top decile gaining about £2,500 a year while the bottom gained almost nothing. Media outlets cited it; within weeks the government reversed the 45p abolition, and within a month the prime minister had resigned. Set beside it a quieter change with the opposite lesson: that same UC taper cut, from 63 to 55 percent. Microsimulation drew the distinction that mattered — the taper cut reached only working recipients, while the pandemic's £20-a-week uplift had reached everyone on Universal Credit, so per pound the uplift cut poverty roughly 40 percent more effectively, even as the taper cut improved work incentives, dropping the share of workers facing marginal rates above 70 percent from 26 to 9 percent. Two policies, similar budgets, opposite distributional signatures — a trade-off invisible without the society view.

By 2026 this work had begun to change shape. Rather than produce a score when someone asks, the same infrastructure started watching legislation as it moved, flagging which bills the model could already handle and routing the rest toward encoding — the on-ramp to encoding the policy corpus at scale, which is the subject of Part III.

One engine, three ingredients

Both views run on one engine. The household calculation and the national estimate are the same computation at two scales, and it works only because three things beneath it hold. The rules have to be encoded correctly. The data has to represent the people. And the behavior — the seam where the model stops describing the law and starts predicting how people respond to it — has to be handled honestly. Those are the three ingredients of any microsimulation, and the next chapter takes them one at a time.

Chapter 7: The three ingredients

Every microsimulation model rests on three foundations: rules encoding, microdata construction, and behavioral dynamics. The framework doesn't appear explicitly in the microsimulation literature—textbooks organize around techniques, not architecture—but it captures what any team that builds one of these models has to solve. Get the rules wrong and the calculations are incorrect. Get the data wrong and the estimates are biased. Ignore dynamics and the projections miss how people respond. PolicyEngine, Tax-Calculator, EUROMOD, and TAXSIM each answer the same three questions differently, and the differences are the most revealing thing about them.

They are also three separate construction projects, and increasingly ones that AI agents can take on: each ingredient is now a layer that can be built at a fraction of its former cost, and each is trustworthy only to the degree its output can be checked against something real. That last clause is the through-line. A layer an agent builds cheaply is worth nothing if you can't tell whether it is right, so the interesting question about each ingredient turns out to be not only how to build it but how to check it.

Ingredient one: rules

The first ingredient is encoding policy rules as executable logic. Every tax rate, benefit threshold, eligibility condition, and phase-out schedule has to become something a computer can evaluate.

This sounds simple. It isn't.

Take the Earned Income Tax Credit. Its phase-in rate depends on the number of qualifying children, and "qualifying child" carries its own age, relationship, residency, and joint-return tests. The credit depends on earned income, which excludes some income types and includes others; the phase-out threshold differs for single and married filers; and many states layer their own EITC on top, each with its own rules about whether it conforms to the federal definition. Encoding this one credit takes hundreds of parameters and dozens of functions, and a single misreading produces wrong answers for millions of households.

And the EITC is one program. A comprehensive US model also has to encode federal income tax with its brackets and dozens of credits, payroll taxes, fifty state income taxes with their own brackets and conformity rules, SNAP, Medicaid, SSI, TANF, housing subsidies, WIC, school meals, and more—on the order of a few thousand parameters, each traceable to a legislative or regulatory source, each changing on its own schedule. And the programs interact: eligibility for one benefit often turns on income defined by another, so a change to a single rule ripples through the rest. It is the edge cases and the interactions, far more than the headline rates, where encodings actually go wrong.

Three architectures

The field has produced three distinct answers to how those rules should be written and stored.

The first is the spreadsheet behind a graphical interface, and EUROMOD is its most developed example. Policy rules live in structured XML—parameters, a "spine" defining calculation order, switches for which policies are active—edited through EUROMOD's own application, which presents each policy as an expandable tree with dialogue boxes for the values. The design has real advantages: a researcher with no programming background can change a rate or a threshold, the interface enforces structure, and the files are openly licensed. Its costs show up at scale. The country models are large, navigation depends on knowing where in the tree to

look, tracking changes across versions means comparing XML, and contributing means learning EUROMOD's tooling rather than a skill that transfers elsewhere.

The second architecture is code-native, the approach PolicyEngine and Tax-Calculator take. Policy rules are ordinary source code—readable in any editor, browsable on GitHub, changed through the same version-control workflow developers use for everything else. A calculation is literally a function you can step through in a debugger and cover with unit tests. Here is a UK National Insurance contribution in PolicyEngine:

```
def formula(person, period, parameters):
    earnings = person("employee_earnings", period)
    ni = parameters(period).gov.hmrc.national_insurance
    pt = ni.class_1.thresholds.primary_threshold
    uel = ni.class_1.thresholds.upper_earnings_limit
    main_rate = ni.class_1.rates.main

    liable_at_main = max_(0, min_(earnings, uel) - pt)
    return liable_at_main * main_rate
```

Anyone who reads Python can follow it: pull earnings, look up thresholds and a rate, apply the rate to earnings in the liable band. The values themselves live separately, in human-readable YAML:

```
gov:
  hmrc:
    national_insurance:
      class_1:
        thresholds:
          primary_threshold:
            values:
              2024-04-06: 242  # weekly
        rates:
          main:
            values:
              2024-04-06: 0.08
```

Logic in Python, values in YAML. Updating for a new tax year is usually just editing parameter files; the code changes only when the structure of the policy changes.

Both architectures are genuinely open source, and both communities are right to call them so—EUROMOD publishes its XML under open licenses, PolicyEngine and Tax-Calculator publish their code on GitHub. What differs is the experience. Reading a EUROMOD policy means installing the software, opening the project, navigating menus to the right policy, and

reading values out of dialogue boxes; reading a PolicyEngine policy means opening a file in a browser. The mental models diverge accordingly—one is "use this application to view these configurations," the other is "read this code like any other software project." This isn't about better or worse—it's about different communities. EUROMOD serves researchers who work inside its ecosystem for years, and the investment in its tooling pays off in depth. The code-native tools serve people who might contribute once, or who work across many projects, or who want to wire microsimulation into other software, and the investment in general programming skills pays off in breadth.

A fourth tool sits slightly outside this axis and matters for what comes next: TAXSIM, maintained by Daniel Feenberg at the National Bureau of Economic Research since the early 1980s. TAXSIM is neither spreadsheet nor open repository—it's a compiled program behind a web interface—and its distinguishing feature is decades of validation against actual IRS return data, which makes it the benchmark other tax calculators are measured against. You can't step through its code, but you can trust its answers, and that combination—opaque internals, validated outputs—turns out to be exactly what the third architecture needs.

Agent-encoded and verified

The third architecture is the newest. Rules are still written as YAML—a format called RuleSpec, in which logic and parameters are versioned, testable files carrying their own effective dates—but the author is usually an AI agent rather than a person, and no encoding merges on the agent's say-so. The agent drafts against a corpus of the actual source law—statutes, regulations, and agency guidance ingested as anchored provisions—rather than from memory, and what it produces is treated as a proposal, not a result. Each one passes through merge-blocking gates before it lands. The code has to compile and the tests have to pass, and then two checks specific to law have to clear. Every monetary amount in the encoding must trace to a quoted passage of the governing statute or regulation—a "money-atom" grounding check that fails the build if any dollar figure floats free of a source. And the encoding's outputs are compared, case by case, against a reference calculator—TAXSIM for US federal tax, EUROMOD and UKMOD abroad, official calculators where they exist—with mismatches blocking the merge until they are explained. Human review sits on top of the automated gates rather than in place of them. The Axiom Foundation is building this layer, and at the time of writing it spans US federal law plus dozens of state codes and several other countries, each encoding carrying a signed manifest recording what was built and how it checked out.

What makes this admissible—what lets you trust law written by a machine—is not the agent and not the YAML. It's the criterion the gates enforce: every unit's output is checkable against ground truth. A statute has a fact of the matter, and an encoding either reproduces it, per household, or it doesn't; the reference calculators say which. That is the whole test, and it is what separates a verified encoding from a plausible-looking one. Agent scale without it is just confident fiction at volume. How that verification actually works—the oracles, the conformance

gates that only ratchet toward correctness, the countries encoded in days rather than years—is the subject of Part III.

Ingredient two: data

Rules encoding determines what the model computes. Microdata construction determines who it computes for. A microsimulation doesn't calculate taxes for one hypothetical family; it calculates them for tens of thousands of records standing in for the whole population, each with realistic income, family structure, location, and program participation. Each record computes its own taxes and benefits, and the model sums those results, weighted, into an estimate for the country—so every revenue score, every poverty rate, and every distributional table downstream inherits whatever is right or wrong about the records themselves. The records have to come from somewhere.

Most of them come from household surveys. In the US that is usually the Current Population Survey, whose monthly basic sample covers about 60,000 households and whose larger annual supplement carries the detailed income and program data microsimulation needs; in the UK, the Family Resources Survey; across Europe, EU-SILC. These surveys exist because governments need population statistics, and they measure income, employment, and poverty—much of what a model needs. The data already exists, built and maintained at public expense; the temptation is simply to use it. Using it is never that simple.

No survey has everything. The CPS captures wages well and capital income poorly, because people underreport dividends and gains and the very wealthy are undersampled; it top-codes very high earnings, compressing the top tail exactly where high-earner policies bite; and it omits tax-relevant detail like itemized deductions and retirement contributions entirely. The gaps run the other way too. Bruce Meyer and colleagues found that 40 to 50 percent of SNAP recipients don't report their benefits in surveys, with similar non-reporting for Medicaid, SSI, and TANF—which biases survey-based safety-net estimates downward and makes program expansions look cheaper than they are. Each survey also has its own blind spots: the Family Resources Survey lacks the council-tax detail UK local-tax modeling needs, EU-SILC misses the within-year timing that drives benefit calculations, and none of them holds everything a comprehensive model requires.

Closing those gaps is a pipeline, and over the past few years it has turned from model-specific plumbing into shared, versioned infrastructure.

Statistical matching combines surveys. Take a CPS household—$80,000, two children, California—and find a similar record in the Survey of Consumer Finances, which carries the wealth and asset detail the CPS lacks; merge their characteristics into one richer record. The craft is in "similar": match on too few variables and you get implausible combinations, a tech worker's income welded to a retiree's assets; match on too many and no two households pair at all. Tax-Data, part of the Policy Simulation Library, pioneered this for US tax modeling by matching CPS records to IRS Public Use File data, recovering the capital gains realizations and

itemized-deduction patterns the CPS alone would miss. Matching doesn't create information; it recombines what already exists in a way that plausibly represents real households, and its validity rests entirely on whether the matching assumptions hold. When they do, the records get richer. When they don't, you have manufactured a statistical artifact and biased everything computed on top of it.

Some variables aren't in any survey, and for those the model imputes—predicts the missing value from the observed ones. Traditional imputation fits a regression and applies its average relationship, so every $100,000 earner in California receives roughly the same predicted deduction. Quantile regression forests instead estimate the whole distribution of plausible values and sample from it, so a high-income California household might draw a deduction anywhere from $15,000 to $50,000. That matters whenever a reform bites unevenly across the distribution: a cap on itemized deductions affects people differently depending where in the range they fall, and mean imputation would systematically misstate it. Preserving that spread, rather than collapsing every similar household onto a single predicted value, is what lets a model see a reform's winners and losers instead of only its average.

Even perfect matching and imputation don't guarantee the microdata matches administrative totals; the survey might show $9 trillion in aggregate wages where IRS records show $10 trillion, and that gap propagates to every estimate. Calibration fixes it by adjusting each record's weight—how many people it represents, so a rural household might stand for 5,000 people and an urban one for 500—so that survey aggregates reproduce known totals. The mathematics is optimization: define a loss measuring how far the weighted aggregates sit from the targets, add a penalty for moving any weight too far from its survey value, and minimize the sum. The targets are specific and numerous—federal income tax by bracket, SNAP benefits by state, Social Security payments by age group, wages by industry—and the reweighting stays plausible only because the penalty holds the new weights close to the survey's original design.

Microdata is also a snapshot: the 2023 CPS describes 2023, while policy questions are usually about future years. So the pipeline ages the data forward. The simplest version scales everything by an index—multiply wages by projected wage growth, investment income by projected returns, advance everyone's age—preserving the structure while moving it to future levels. A more sophisticated version models who retires, who enters the workforce, and how individual incomes evolve over time, gaining accuracy at the cost of much more to build and validate. PolicyEngine draws its projection factors from the Congressional Budget Office's forecasts so its estimates line up with the baselines Washington already argues over.

These pieces are now built as reusable libraries rather than one-off code: imputation methods behind a common interface, weight calibration with holdout checks and diagnostics, and synthesis and projection alongside them. Consolidated, they form populace, a calibrated-microdata commons—population data built, tested, and versioned as infrastructure in its own right rather than as a hidden appendix to a tax model. The milestone the approach is measured against is the Enhanced CPS that PolicyEngine released in August 2025, which fused five datasets—each contributing what the others lacked: the CPS its income and employment backbone, the Survey of Consumer Finances its wealth, the Consumer Expenditure Survey its

spending, the Residential Energy Consumption Survey its energy use, and the American Time Use Survey its hours—then calibrated the result to 9,168 administrative targets, cutting the average deviation from those totals from 8.3 percent to 0.2 percent.

One thing calibration cannot do is worth stating plainly, because it is where the residual risk lives. Matching aggregates is matching margins: after calibration the totals are right by construction, but nothing guarantees the joint structure—that the right households hold the right combinations of income, assets, and benefits. A population can reproduce every published total and still assemble those totals out of people who don't quite exist. And joint structure is exactly what many reforms turn on: a benefit that phases out with income but only for households holding assets above some threshold depends on how income and wealth are paired in the data, not on either total alone—and calibration to the margins leaves that pairing unchecked. Testing that joint structure—holding a synthesized population against held-out ground truth rather than against the totals it was fit to—is verification work, of the same per-unit kind the rules layer demands, and Part III returns to it.

Ingredient three: dynamics

Rules handle the arithmetic and data handles the who; dynamics handle how people respond. Raise a marginal rate and some people work less; cut a phase-out and some work more; tax carbon and some drive less. These responses move revenue, distribution, and effectiveness, and they are much harder to model than the static calculation.

Most microsimulation is static: compute what each household owes and receives under current policy, compute it again under the reform holding behavior fixed, and report the difference as the day-one effect. Static analysis answers a great deal—who gains, who loses, and by how much barely depends on behavioral assumptions—but it misses responses that change the totals. A carbon tax that raises $100 billion assuming no one changes their driving raises less once people do. The gap between the static and dynamic numbers can be large, and it is largest exactly where behavior is most sensitive to the policy, which is why revenue estimates for high-end tax changes turn so heavily on assumptions the data cannot pin down.

The most-studied response is labor supply: how much do hours, or reported income, shift when the after-tax wage changes? Economists summarize it as an elasticity, and the most comprehensive summary is the elasticity of taxable income, which folds in not just hours but tax planning, income shifting, and avoidance. Carina Neisser's meta-regression—1,720 estimates from 61 studies—finds a mean ETI around 0.30, meaning a 10 percent rise in the net-of-tax rate raises reported income by roughly 3 percent. But the spread is enormous: some estimates sit near zero, others above 1.0, varying with income, tax system, time horizon, and method. That range is not a rounding concern. Raising the top rate from 37 to 39.6 percent brings in something like $50 billion more per year under a low elasticity than under a high one, and no amount of data has settled which is right—it is a genuine and consequential disagreement among honest estimators.

Some patterns within the range are robust. Primary earners in married couples barely move—they work about the same whatever the rate—while secondary earners and single parents respond more; the reported income of the very wealthy moves the most, though much of that reflects tax planning rather than any real change in work. A model that takes behavior seriously has to say which populations respond how much, and then be honest that the parameter carries its own uncertainty. PolicyEngine handles this by making the choice explicit: users can run no response, CBO-style elasticities, or their own parameters, and watch the revenue estimate move—uncertainty surfaced rather than buried. Labor supply is only the first margin. Capital gains realizations respond in their timing—higher rates encourage holding assets, lower rates encourage selling—so a rate change can swing revenue sharply from one year to the next. Taxpayers restructure around new rules, shifting income between years or reclassifying its type, as the speed of the response to the 2017 Tax Cuts and Jobs Act showed. Program take-up rises and falls with generosity and complexity, and because take-up for programs like SNAP and the EITC already runs well below 100 percent, a reform can change costs as much by moving enrollment as by changing the benefit itself. Each is a further response the same machinery can carry.

Elasticities take for granted that we already know how people respond. A structural model tries to derive the response instead: specify what people maximize—utility from consumption and leisure under a budget constraint—and let behavior fall out of the optimization, so that changing the tax schedule and re-solving produces the new behavior directly. Its appeal is reach. A structural model can price a 90 percent marginal rate no one has ever observed, where an elasticity estimated at lower rates may simply not carry. Its cost is assumption—that people truly maximize expected utility, that the chosen functional forms are right—and a steep climb in computational complexity. PolicyEngine doesn't yet implement structural responses; the architecture leaves room for them as modules over the same household data.

There is a seam running through this third ingredient, and it is worth naming because much of the rest of the book is built on it. Everything up to the behavioral response is deterministic. Given the rules and a household's inputs, the tax it owes and the benefits it receives are not estimated—they are computed, exactly, and can be checked, household by household, against the statute, against a reference calculator, or against what a caseworker actually determined. There is a fact of the matter, and you can verify it now, per unit, before the policy ever takes effect. This is the property the rules layer was built around, seen from the population side: a household's determination is either faithful to the law or it isn't, and you don't have to wait for anything to find out.

The behavioral response is different in kind. Ask how people will change their hours, their portfolios, or their take-up, and you have left the domain where the answer is computable and entered the domain of prediction. Structural models don't escape this; for all their principle, their outputs still land on the prediction side of the seam, answerable only once the behavior has actually happened. There is still a fact of the matter, but you don't get to check it today—you find out when reality arrives and the data comes in. It is the same model wearing two epistemic hats: one output verified against the rules as written, the other against outcomes

as they happen. Keeping those two straight—being exact where exactness is possible and honestly uncertain where it isn't—is most of what separates a trustworthy model from a merely confident one.

The integration problem

The three ingredients give you the questions to ask of any microsimulation model, and building one means answering all three at once:

Rules. Are the calculations right, validated against authoritative sources, and legible enough that you can see why they produce a given answer?

Data. Does the microdata represent the population, with missing variables filled defensibly and weights calibrated to real totals?

Dynamics. What behavioral assumptions are in play, are they stated or hidden, and how much do the results move when the assumptions change?

Different models are strong in different columns—TAXSIM in rules, validated against decades of returns; Tax-Data in microdata construction; OG-USA in dynamics, with a full overlapping-generations model of behavior. PolicyEngine's aim was to be strong across all three at once, and that is exactly what made it hard.

The three ingredients don't sit side by side; they constrain each other. The rules you want to simulate dictate the data you need—model capital gains and you need asset data, model childcare subsidies and you need childcare expenses. The data you have bounds the rules you can implement—without itemized-deduction detail, you cannot credibly score a deduction reform. And dynamics reach into both, because a behavioral response changes effective rates, which change the distributional story; a reform that looks progressive on the static tables can look different once the highest earners respond the most. Building the model means making these fit: rules that handle whatever data exists, data that supports whatever rules matter, dynamics calibrated to make sense with both.

That is why PolicyEngine took years rather than months. No single tax formula is hard; individual calculations are straightforward. What takes sustained engineering is making comprehensive rules work with realistic data while supporting flexible dynamics, all at once, across the whole system. Each of the three ingredients is now becoming a layer an agent can build and a machine can check on its own terms—rules against statute, data against ground truth, dynamics against outcomes as they arrive—and that decomposition is the story Part III tells.

Part III: The agent turn

Chapter 8: The AI can't do your taxes

In March 2023, PolicyEngine added a button labeled "Explain with AI." Click it, and the calculation behind your Child Tax Credit or your SNAP benefits turned into a paragraph of plain English tailored to your situation.

Consider a family of four in Connecticut earning $47,000 a year. PolicyEngine could compute that they qualified for about $475 in annual WIC benefits — but the "why" was buried in intermediate steps: income thresholds, categorical eligibility, participation windows, documentation rules. The model tracked every one of them. A user could, in principle, trace the whole tree from inputs to answer. In practice, almost no one would, and the family staring at a benefits portal at eleven at night least of all. Open-source code made the logic visible; it did not make it legible.

The button closed that gap. It passed the calculation tree — every intermediate value, every parameter — to a language model, which wrote back something a person could actually use: your family qualifies because you have a child under five and your income falls below 185 percent of the poverty line for a household your size.

The model did not compute the answer. It read the answer back.

That distinction was the whole architecture, and it was deliberate: a deterministic backend to produce the numbers, an AI frontend to explain them. The microsimulation engine encoded the rules exactly, and given the same inputs it returned the same outputs every time, auditable down to the line of statute. The language model never touched the arithmetic. Ask it "how much Child Tax Credit does this family get?" and it would not answer from its own weights — it would call the engine, take the engine's result, and put that result into words. The reason to keep the line exactly there is the rest of this chapter: ask a language model to do the taxes itself, and it fails.

The evidence that it can't

The failures are measurable, and they got measured. In 2023, researchers handed GPT-4 the full text of the Internal Revenue Code and asked it 276 true-or-false questions about it; it misread about a third. None of the errors were arithmetic. All of them were misreadings of the law — the model simulated comprehension it did not have, which is a more dangerous failure than getting a sum wrong, because it looks exactly like understanding.

Reading is the easy part. Computing is harder. In July 2025, Column Tax released TaxCalcBench, which gave frontier models everything they should need to prepare a return — the rules, the taxpayer's data, and effectively unlimited time to reason — and asked them

to produce the complete filing. The best model computed fewer than one in three complete returns correctly. These were not near misses off by a rounding error; they were returns a person could not file. Given every input a professional would have, and no clock running, the model still could not reliably turn the rules into the number at the bottom of the form.

A federal return, though, is only a slice of a household's real situation. Most families meet taxes and benefits at the same time — the Earned Income Tax Credit phasing out while SNAP phases down while Medicaid eligibility flips at a threshold two hundred dollars of income away. In 2026, PolicyEngine put that fuller problem on a public board. PolicyBench draws realistic household scenarios from calibrated microdata and asks around twenty frontier and open-weight models to compute the complete tax-and-benefit result, scoring how often each lands within a dollar of the engine's answer. Drawing the scenarios from real population data rather than tidy textbook cases is deliberate: the errors cluster exactly where households are complicated — multiple earners, mixed-age children, benefits stacked on benefits — which is not an edge case but where a great many families actually live. As of mid-2026, the best model still gets roughly one in six household calculations wrong by more than a dollar. Most models miss a quarter or more.

The dollar threshold matters because of what the misses are. They are not rounding. They are family-level: a household ruled ineligible for Medicaid that in fact qualifies, a SNAP allotment off by real money, a credit claimed at the wrong amount. A benefit determination is not a vibe to be approximated — it is a number that decides whether a family gets help this month. An error of a dollar in the fourteenth decimal of a physics constant is noise; an error that flips a Medicaid eligibility flag is a family turned away from something they were owed. That is the standard the benchmark holds models to, and by that standard the models are not close.

When the audit layer hallucinated

PolicyBench taught a second lesson, one it had not set out to teach. Alongside each scored scenario, the benchmark at first published an explanation narrative — an AI-written account of how the result had been derived, meant to make the board legible to anyone auditing it. Those narratives read beautifully, and at first they lied. One described a household of retirees as enrollees in a "non-elderly expansion" category that did not apply to them at all. The mechanism was invented; the prose was fluent; and that combination was precisely the danger. Auditors who read the narratives to check the benchmark inherited the fabrication — the explanation smuggled a false derivation past human review exactly because it sounded like an explanation. A bare wrong number at least invites a second look; a wrong number wrapped in a confident derivation actively discourages one.

The fix was not a better prompt. It was to stop letting the narrative come from the model's imagination and make it come from the engine's internals: regenerate every explanation from the actual variables and intermediate values the computation had produced, and add validators

that reject any mechanism the trace does not support. An ungrounded claim now fails the build rather than reaching a reader.

The lesson runs well past this one benchmark. Even the audit layer needs ground truth. If you want an AI to check work — its own or anyone else's — you have to give it traces, not prose: the numbers the system actually computed, not a plausible story about them. That principle, that you verify with traces and not with narration, is the one the next two chapters are built on.

Tools, not training

If frontier models cannot reliably compute a return, the tempting inference is that they need more training. The better inference is that they need a tool.

PolicyEngine's first stab at that, in the same season as the explain button, was crude. It generated a structured block of text — the reform's parameters against their baselines, the budgetary and distributional results, a note on style — for the user to paste into ChatGPT, which spun it into a readable analysis. The numbers were right because they came from the engine, not the model; the model only handled the prose. It saved a researcher the first hour of drafting. But the human was still the integration layer, ferrying data between two systems by hand.

Then the models learned to call tools themselves. GPT-4, Claude, and their successors could invoke a function in the middle of a response — hit an API, take the result, keep going. Now the flow inverted. Instead of PolicyEngine generating prompts for the AI, the AI could call PolicyEngine. Ask "what happens if we make the Child Tax Credit fully refundable?" and an assistant could translate the question into policy parameters, call the engine to run the simulation, receive the quantitative results, and explain what came back — language on the outside, computation on the inside.

Standardization followed the capability. Anthropic open-sourced the Model Context Protocol in November 2024; OpenAI adopted it in March 2025, and Google followed within months — a fast convergence on shared plumbing. MCP gave AI systems a common way to discover and invoke external tools, turning a thicket of one-off integrations into something closer to a pluggable ecosystem. Within a year, a sprawling collection of MCP-compatible tool servers had appeared, exposing everything from database queries to calendar management to code execution, and a PolicyEngine endpoint could now be found and called by any compatible assistant without bespoke wiring for each provider.

That shift moved the bottleneck, and moving the bottleneck is the argument of this chapter. In 2023, the binding constraint was model capability: could the AI reliably pick the right tool and call it with the right parameters at all? By 2025 and 2026, models had cleared that bar, and the constraint moved to the tools themselves — were they accurate, comprehensive, current, and legible to the people relying on them downstream? An AI that flawlessly calls

an inaccurate calculator is worse than useless, because it launders a wrong number through a confident, well-formatted sentence. But a technically correct calculator can fail the same way for duller reasons: if it is slow to update when the law changes, undocumented, or full of integration traps, an agent will call it correctly and still hand back an answer that is wrong or unusable. The quality of the deterministic backend became the thing that mattered.

And tool quality is not one property but several, each its own kind of work. Accuracy is the obvious one, but a calculator also has to be comprehensive — a tool that knows federal tax and three benefit programs will confidently return the wrong household total the moment a fourth program is in play. It has to be current, tracking a change in the law within days rather than months. And it has to be legible to whoever stands downstream of it, so a wrong answer can be caught and a right one trusted. None of these is a problem more training solves. Each is a problem someone has to build, and then keep building, inside the tool.

Which is why TaxCalcBench and PolicyBench matter beyond their leaderboards. They measure the exact quantity the whole architecture now rests on — not whether the AI is clever, but whether the tool it reaches for is right.

What the AI doesn't do

Draw that boundary clearly and it defines what stays outside the model's authority, in four places that never move into it.

The AI does not set policy parameters. The Child Tax Credit maximum is $2,200 because Congress set it at $2,200 — in the One Big Beautiful Bill Act of 2025 — not because a model inferred a plausible-looking figure from the shape of the tax code. Every parameter carries legislative or regulatory provenance, and that provenance, not the model's fluency, is what makes it correct.

The AI does not estimate behavioral responses. When a microsimulation models how people might change their behavior under a reform, the elasticities come from the economic literature and from explicit, recorded methodological choices — not from a language model's inference in the moment of being asked.

The AI does not judge whether a policy is good. The system can report that a reform cuts poverty by three percent and costs fifty billion dollars; whether that trade is worth making is a human question, and the tool is built not to pretend otherwise.

And the AI does not overrule the engine. If the computation says a household owes ten thousand dollars, the explanation explains ten thousand dollars. It does not talk its way toward a rounder figure it happens to find more agreeable.

None of these are constraints bolted on after the fact. Each is the same rule seen from a different side: the authority for a number belongs with whoever can be held to account for it — a legislature for a parameter, a documented literature for an elasticity, a voter for a value

judgment, the engine for a computation — and never with the system that is merely good at describing the number once it exists. They are the source of the trust: the AI made the tool easier to reach without making it any less exact.

AI as priors, not authorities

The behavioral-response boundary deserves a caveat, because it is the one place where a language model has real upstream value. A production policy model should never silently let an AI pick an elasticity and bury it in a result. But an AI can be a fast, auditable way to survey what the plausible values even are before a human commits to one.

One recent experiment put 26 economic quantities — canonical labor-supply and tax elasticities among them — to eleven frontier models, repeating each elicitation fifteen times to recover a distribution of answers rather than a single confident number. A related project with Jason DeBacker probes what language models imply about the elasticity of taxable income, including replications of lab experiments and surveys of simulated taxpayer personas.

Neither is a substitute for the literature, and neither is meant to be. Their value is diagnostic: they show what distribution a model carries, where models disagree with one another, and where a change of wording or definition quietly moves the answer. When eleven models cluster tightly around a labor-supply elasticity, the agreement is a weak signal at best; when they scatter across a range wide enough to flip a reform from progressive to regressive, that disagreement is the more useful finding, because it tells an analyst precisely which assumption the result is hostage to. That helps a human map the parameter space faster, and it surfaces the assumptions an AI would otherwise carry into a policy conversation unannounced. The discipline is the same as everywhere else in this chapter. A model may suggest a prior, gather evidence, and summarize disagreement — but the chosen parameter has to be written down explicitly and left open to review, where anyone can see it and argue with it.

The interface, not the authority

Put the pieces together and the role AI plays comes into focus. A researcher asks, in plain English, how extending premium tax credits would affect insurance coverage among middle-income families in a handful of states. The assistant translates that into computational steps — define the reform, select the relevant population, run the simulations, aggregate the results — the engine does the actual analysis, and the assistant renders the findings back into an answer. The person who once needed Python fluency and intimate knowledge of the model's internals can now work through the question in language; the expert who has both can iterate faster than before.

This is augmentation, not automation, and the limits show where judgment lives. When PolicyEngine tested letting several specialized agents coordinate on a research task, they handled standard distributional runs well but stumbled where programs interacted — implementing

each one correctly in isolation and then missing how a benefit taper in one interacts with a phase-out in another. The mechanical work translated cleanly; the coordination logic did not, and that is exactly the seam where a human analyst still has to stand. Widening access cuts both ways in the same spirit: it lets a congressional staffer who understands policy but not programming, or an advocacy group without a data team, run analyses that used to require an economist — and it opens room for cherry-picked scenarios and confident misreadings alongside genuine scrutiny. Those risks lived in the old system too. They were simply concentrated among the few people who could run the models at all, and broader access at least invites broader checking.

What holds the whole arrangement together is where the authority sits. The AI is the interface — natural, adaptive, forgiving of a half-formed question. The encoded rules and the calibrated data are the source of truth. Move that truth into the model's weights and you get fluency without provenance, which is the precise failure the benchmarks measure. Keep it in the deterministic layer, and the model is freed to do what it is genuinely good at: turning a hard question into the right call, and a wall of numbers into a sentence a person can act on. Shown that a worker faces a 67 percent marginal tax rate at their income, the engine can hand over the pieces — federal tax, payroll tax, an Earned Income Tax Credit phasing out, a partial credit clawback — and the model can assemble them into the one plain sentence that explains why an extra dollar of earnings mostly vanishes. The expertise to produce that breakdown by hand is real and scarce; the engine computes it, and the interface makes it readable.

Everyone lands on the same architecture

PolicyEngine was not alone in finding this shape. Governments arrived at it independently, which is the strongest kind of evidence that it is the shape the problem forces.

In November 2025, the IRS deployed Salesforce's Agentforce agents across three offices — the Office of Chief Counsel, the Taxpayer Advocate Service, and Appeals — for case summarization, document search, and policy navigation, the retrieval-heavy work where language models are strong. Every final decision stayed with a human agent; the AI could not disburse funds or make an eligibility determination. The deployment followed a workforce reduction that made the assistance less a bet on innovation than a practical necessity.

A month later, California launched Poppy, a digital assistant for state employees across fifty departments. Built in-house and run on state servers rather than an external cloud, drawing on eleven different models and grounded on public state data to hold down hallucination, it was used by more than two thousand employees during its pilot to navigate a dense catalog of policy and government-specific terminology.

Neither system computes an entitlement. The IRS agents help a human analyst find the relevant regulation faster; Poppy helps a state employee understand a rule already on the books. The actual rules, statutes, and calculations stay human-maintained and human-auditable underneath. An open-source tool, a commercial deployment, and a government-built assistant

converged on the same division of labor: deterministic systems for the answers, AI for the access.

A good tool to call

That convergence points at the question the rest of this book turns on. If an AI is now only as good as the tool it calls, then the tool worth building is a demanding thing. Not merely accurate today, but accountable: every number it returns checkable against ground truth — traced to a line of statute, matched against an independent calculator, reconciled to an administrative total.

Building a tool like that — rules encoded exactly enough to check, data calibrated well enough to trust — turned out to be possible. That is the next chapter.

Chapter 9: Encoding the law

The question sounds simple. What is a household's real income—after taxes, after benefits, after the way the two collide? A lending platform needs the answer to underwrite a gig worker. A benefits screener needs it to tell a family what it qualifies for. An AI assistant asked how much someone would receive from the Child Tax Credit needs it to answer without inventing the number. And through the mid-2020s none of them could simply buy the answer, because the market sold only slices of it.

Income-tax filing had well-funded specialists—April, embedding returns inside other companies' apps, and Column Tax, which had filed more than a million returns through its API and built an AI agent to help maintain the engine as the law shifted year to year. Sales tax had Avalara, acquired for $8.4 billion in 2022. Payroll had ADP and Gusto. Benefits screening had MyFriendBen and Benefit Kitchen. Policy research had PolicyEngine, EUROMOD, and Tax-Calculator. Every slice existed somewhere; the integration existed nowhere. The global tax-tech market was climbing toward $60 billion, and all of it was balkanized into verticals that each re-encoded the same overlapping rules behind their own walls.

The full-stack question—income tax plus benefits eligibility plus the population-level simulation that turns one household into a national estimate—had no home. And it is the question that matters, because benefits and taxes are not separable. SNAP falls as earnings rise while the Earned Income Tax Credit climbs; the net depends on household composition, state of residence, and a dozen interacting thresholds. Answer only the tax half and you mislead every low-income user you touch. And because no one sold the whole, every company that needed it rebuilt the overlapping rules privately—the same EITC phase-in, the same SNAP deductions, encoded again and again behind incompatible walls, each copy a fresh chance to be wrong.

Priya—a composite drawn from real fintech engineering teams—ran the problem to ground. Her company advanced cash to gig workers who needed to know their true take-home pay before

borrowing against it. A tax library pulled from GitHub handled federal brackets and nothing else, missing low-income users by thirty to forty percent. A commercial tax API charged per call and could not touch benefits. Prompting a frontier model produced a hallucination rate no financial product could ship. Building it in-house penciled out to eighteen months of engineering before the first law changed underneath it. Every path led to the same place: a partial, fragile solution, rebuilt privately by every company that hit the wall.

Why this can't be trained away

The tempting response is to wait for the models to get good enough. They won't, for reasons that are structural rather than a matter of time. Tax law changes every January, so last year's training data encodes rules that no longer apply, and a model cannot extrapolate to a bracket it has never seen. Fifty states each run their own income taxes, credits, and benefit programs—a combinatorial sprawl beyond what pretraining reliably memorizes. Eligibility turns on dozens of interacting variables, where a single wrong input silently yields a wrong result, and language models compress exactly that kind of exact logic into statistical approximation. And financial calculation tolerates neither 95 nor 99 percent: a model right nineteen times in twenty is a model that files one return in twenty wrong. Column Tax's own engineers put it flatly—today's language models "cannot 'do taxes' on their own because tax calculations require 100% correctness".

The benchmarks keep confirming it and stubbornly refuse to improve on schedule. On PolicyBench, a public board that scores language models on complete household tax-and-benefit calculations, the best 2026 models still get roughly one in six households wrong by more than a dollar; most miss a quarter or more, and the misses are structural—a blown Medicaid determination, a wrong SNAP amount—not rounding. The answer was never a better prompt. It was a tool: a deterministic, auditable engine the model can call, that computes the number while the model explains it. Which raises the real question—what would it take to build that engine for all of tax and benefit law, and to make anyone believe its numbers?

Administrative quality is part of the product

Before the build, a correction to what "quality" even means, because the hardest problems in benefits software turn out not to be mathematical. Whether a formula is legally correct is one question. How fast a rule change becomes publishable, how many misunderstanding loops open between policy experts and engineers, how much hand-holding a partner developer needs before making a correct API call, how often a tool hands a household a misleading answer because its interface cannot express a real-world exception—those decide whether the software is any good, and none of them is a math problem. They surface as operational metrics: time to publish, discrepancy rates against authoritative examples, onboarding time, the rate at which a support ticket exposes an assumption no one had written down.

Households experience policy through administration. A tax credit that exists in statute but takes six months to reach a screener is not, from where the household sits, fully real. A calculator that silently misses an immigration rule or a state-specific exception is not merely imperfect; it erodes trust, burns caseworker time, and suppresses the very take-up the program was built to produce.

The stakes turn visible when the administration fails at scale. During the 2023 Medicaid unwinding, CMS ordered states to restore coverage for roughly half a million children and families after finding a defect in the automatic-renewal logic of their eligibility systems. That is administrative quality seen from outside: the entitlement exists, the renewal process exists, and a software error grows large enough that federal officials have to command a mass correction. Up close it is more concrete still. KFF Health News documented how fixes in Deloitte-run eligibility systems can take months or years—in Kentucky, one limitation that cost a resident his Medicaid coverage took about ten months, more than 3,500 hours of work, and over $500,000 to resolve; in Georgia, officials were still untangling a defect affecting more than 25,000 SNAP and TANF cases nearly two years after it was first reported. Deloitte-built eligibility systems span roughly twenty-five states and some $6 billion in contracts. Benefits screeners carry a quieter version of the same danger: Virginia Eubanks and colleagues have shown that a screening tool can look authoritative while its logic stays opaque, so people act on wrong predictions without ever thinking to challenge them. What these failures share is opacity: a defect buried in a vendor system that no outside party—and often no inside one—could inspect, trace to a rule, or fix quickly. The eligibility logic was right in statute and wrong in software, and the gap between the two stayed invisible until it produced a headline.

And in 2025 policy began to price administrative accuracy directly. Under the One Big Beautiful Bill Act, enacted July 4, 2025, states will for the first time—beginning in fiscal 2028—pay a share of SNAP benefit costs pegged to their own payment error rate: nothing while the error rate stays under 6 percent, then 5, 10, or 15 percent of benefits as it crosses 6, 8, and 10 percent. The national payment error rate for fiscal 2025 was 10.6 percent, an average that would put many states straight into the top penalty tier. The state share of SNAP administrative costs rises from 50 to 75 percent in fiscal 2027, and Medicaid's new work requirements oblige states to stand up verification systems by the end of 2026. A payment error rate used to be an audit statistic; it is now a line in the state budget—which is exactly the cost that verified, provenance-carrying rules infrastructure exists to lower. A state that can trace every determination back to the governing rule, and check it against an independent calculator before the payment goes out, has a mechanism for driving that error rate down. Accuracy priced as a penalty is accuracy that infrastructure can help buy back.

The downside is not hypothetical, and the precedents are sobering. Australia's Robodebt scheme raised automated debts against welfare recipients on faulty income-averaging, was unwound in the courts, and put ministers before a royal commission. Michigan's MiDAS system issued tens of thousands of false fraud determinations against unemployment claimants. Arkansas replaced a nurse's judgment with an algorithm for allocating Medicaid home-care hours, cut care for disabled residents, and drew litigation. In each case the arithmetic was

automatable and the judgment was not—and automating the judgment without a check caused real harm.

The administrators drawing the lines seem to know it. USDA's Food and Nutrition Service has held that AI may not replace state merit personnel in SNAP eligibility determinations —a boundary that falls exactly where this book does. The infrastructure is decision support, not decision replacement: a human makes the determination, and the tool's job is to make that determination checkable.

The Axiom Foundation

This is the gap the Axiom Foundation exists to close, and by the middle of 2026 the closing was well underway. Axiom is a Delaware nonprofit, fiscally sponsored by the PSL Foundation, anchored by funding from Ballmer Group; Ariel Kennan became its president on July 1, 2026, and it launches publicly on July 28. Its charter is narrow and immodest at once—encode the law itself, exactly, as open infrastructure. PolicyEngine had proved that tax and benefit rules could be encoded accurately at all; Axiom is the attempt to encode all of them, in a form anyone can run and anyone can check. The ambition is larger than any single government's analytical office attempts, and the wager is that agent-drafted encoding behind merge-blocking checks is what makes that scope tractable for a nonprofit at all. Its sibling institute, Thesis, takes the other half of the problem—forecasting what government will actually do—and belongs to later chapters; this one is about the rules.

An Axiom encoding is not a program in the ordinary sense. Its format, RuleSpec, is a set of YAML files that hold the law's logic and the law's numbers apart, each file versioned, each testable, each stamped with the dates over which it takes effect. The logic that computes a credit lives in one place; the parameters it reads—a rate, a threshold, a phase-out amount—live in another, so that when an agency publishes next year's inflation adjustment, a parameter value changes and the logic does not. An act of Congress changes the logic; an annual revenue procedure changes only the numbers; and because both are versioned files with effective dates attached, an encoding can answer not merely "what does the law say" but "what did it say last March, and when did we learn it had changed." Take the Earned Income Tax Credit. The statute defines a credit that climbs with earned income at a set rate up to a ceiling, plateaus, and then phases out above a higher income threshold that shifts with filing status and number of children. In RuleSpec that becomes a formula referencing named parameters—the phase-in rate, the earned-income ceiling, the phase-out rate and threshold—each carrying its own citation, effective date, and source link. A reader can see, for any date, which value was in force and which line of the encoding produced it.

Beneath the rules sits the corpus—the source text itself. Axiom ingests statutes, regulations, agency manuals, and sub-regulatory guidance as anchored provisions, each clause individually addressable, organized against a registry of roughly 41,000 legal concepts. The corpus is what makes provenance possible. An encoding earns trust not because its author was careful but

because it points back to the exact governing words, and those words live in a service anyone can open and read. Anchoring matters because law does not live only in statute. The rule a caseworker actually applies is often buried in an agency manual or a guidance letter, and an encoding that could cite only its enabling statute would miss where the binding detail sits. The corpus is built to hold all of it—primary law and the sub-regulatory layers beneath it—at clause resolution.

The encoding pipeline

Encoding law by hand is slow and does not scale; a team can keep one country current or attempt fifty, not both. The reason Axiom can aim at the whole of it rather than a corner is a pipeline called axiom-encode, which turns encoding from a task measured in analyst-months into one measured in agent-runs. It runs in stages. It pulls the relevant provisions from the corpus. It scaffolds a prompt around them—the statute text, the concepts they touch, the shape of the encoding to produce. It hands that to a language model, which drafts the RuleSpec. Then, before anything merges, the draft runs a gauntlet in which every gate can block the merge: the code must compile, its tests must pass, a proof step must validate the logic's internal consistency, the output must be compared against independent reference calculators, and every monetary obligation must be grounded in the source. Only a draft that clears all of them yields a signed manifest—a cryptographic record of what was encoded, from which sources, having passed which checks—so a later reviewer, a skeptical agency, or a court can reconstruct the provenance rather than take it on faith. And the human review sits deliberately at the top of the stack, not the bottom: the machine drafts and the gates screen, while a person adjudicates the judgment calls the statute leaves open, the cross-references and exceptions no gate can settle.

The gates are not redundant; each defends a different failure. Compilation catches an encoding that does not run at all. The tests catch a change that breaks a case that used to work. The proof step catches logic that contradicts itself. Comparison against an independently built calculator catches an encoding that runs, passes its own tests, and is still wrong—the failure no self-check can see, and the one the next chapter is about. And the last gate catches the failure particular to machine-drafted law: a number with nothing behind it. None of these gates trusts the model that wrote the draft, which is the point. The pipeline runs several language-model backends and stays indifferent to which one produced a given encoding, because the guarantee lives in the gates, not the generator. A better model drafts faster and trips fewer checks; it never earns the right to skip them.

One of those gates deserves its own name, because it is the emblem of the whole method. The money-atom gate demands that every monetary obligation an encoding produces—every dollar figure the law commands—trace to a quoted excerpt of the governing source. Not a section number dropped in a comment, but the actual authorizing words, pulled from the corpus. There is a reason the gate targets money rather than every clause: a wrong eligibility flag is a bug, but a wrong dollar figure is a household underpaid or a program overexposed—the failure

mode with a face on it. Grounding every commanded amount in the words that command it puts the tightest check where the damage is largest. If a figure cannot be grounded that way, the build does not warn. It fails.

Provenance stopped being a promise and became a merge condition.

That is a mechanical fact with an outsized consequence. Ordinary software cites its sources in documentation that drifts out of date the instant someone edits the code; a merge-blocking check cannot drift, because nothing merges until it passes. The claim that every number traces to the law is therefore not a virtue the project professes but a condition its build enforces—and the machinery that holds that line, and keeps the encodings from quietly going stale, is the subject of the next chapter.

What counts as law

By July 2026 the corner Axiom had turned was not a small one. The US repository held on the order of 3,000 rule files with matching tests, covering federal law plus twenty-eight state codes, absorbed with their full git history. Absorbing a state code with its history rather than retyping it means the encodings inherit years of prior fixes and the reasoning behind them—a change of representation, not a fresh start with fresh bugs. Separate monorepos carried the United Kingdom, Canada, New Zealand, and Belgium, alongside a set of African lanes validated against established reference models that a later chapter takes up in full. The point is not the tally. It is that a single pipeline produced all of it, which puts the marginal cost of the next jurisdiction in units of agent-time rather than institution-years.

And the scope is deliberately, almost provocatively, total. Axiom's charter is not "the parts of law that compute a number." It is all public policy—statutes, regulations, agency manuals, statutory guidance, grant conditions, and the eligibility scheme of a single London borough. Among the early UK encodings is the council-tax-reduction policy of the Royal Borough of Kingston upon Thames: a local scheme binding on a few tens of thousands of residents, encoded with the same machinery as federal income tax. That is not reach for its own sake. A resident of the borough is governed by its council-tax-reduction scheme as concretely as by any act of Parliament, and a stack that encoded only national law would be blind to the rule that actually sets her bill. Encoding the small, the local, and the merely-binding-on-a-few-thousand is the point, not the exception. The principle behind that choice is stated as a rule of its own.

Bindingness is metadata, never a scope filter.

Whether a provision is hard law or soft guidance is recorded as a property of the encoding, never used to decide whether it is worth encoding—because a household is governed by the manual a caseworker actually follows as much as by the statute behind it. That is also why Axiom models authority explicitly, representing the chain by which a statute delegates power to an agency and an agency binds itself through its own published policy. The doctrines are old—in US law an agency must follow its own regulations; in English law it must follow its

own stated policy absent good reason —and encoding them lets a model answer not only what a rule computes but whether the body applying it had the authority to. All of it ships under permissive licenses, Apache 2.0 for the code and Creative Commons Attribution for the content, so that no one need ask permission to run, fork, or build on the encoded law.

Much of law, in fact, is not "compute an amount" at all. It is "does this conclusion hold, fail, or remain undetermined, given a history of events?"—whether a notice is valid, whether a deadline was met, whether an agency followed the procedure it bound itself to. A stack that could only multiply rates by thresholds would capture the arithmetic of policy and miss its logic; modeling authority, delegation, and legal judgment is what lets the encodings represent that second shape. It is the difference between encoding a calculator and encoding the law.

Governing a commons

Building this as a public good removes the sharpest objections—no shareholders, no paywall on the law—without removing the need for governance. A shared rules layer has failure modes of its own, and they are worth naming plainly.

Whose rules come first. Funders and large institutional users have priorities, and the risk is a roadmap that tracks whoever pays the bills rather than where accurate rules matter most; when a database project deprioritizes a feature the stakes are uptime, but when a rules project deprioritizes a jurisdiction's benefit code the stakes include whether vulnerable households get accurate answers. The guardrails are the ordinary ones for a commons—transparent governance, a published roadmap, and the standing fact that anyone can fork and encode what they need.

Capture. Even a nonprofit can be captured—by a dominant funder, or by a maintainer monoculture—until "open" is a brand rather than a fact. Independent governance and a genuine plurality of contributors are the only durable defense.

Staleness. Public goods are chronically underfunded, and the characteristic failure is not bankruptcy but quiet drift, encodings falling out of date as laws change while no one notices. This is the fear the method answers most directly: the conformance ratchets of the next chapter turn "keep it current" from an aspiration into a gate, one under which coverage can only rise and unexplained gaps can only fall.

Openness is not incidental to these defenses; it is the mechanism behind them. A fintech integrating the encodings can read exactly what they compute, an agency can confirm they match statute, and a citable, inspectable rule is far more defensible than a model's unexplained output when a regulator or a court asks for the audit trail. The ability to fork is the ultimate check on capture: a commons no one can leave is not a commons.

The deeper commitment is structural. The encoding of law is reference infrastructure—closer to a dictionary or a map projection than to a product—and it is not well served by a private gatekeeper sitting between the public and the statute. So the rules layer is a commons, and the commercial activity that will grow around it—managed runtimes, service-level guarantees,

applied products—is built on the commons, never owner of it. One such operator already exists in formation, organized around graded simulation; it is not yet public.

The line the foundation commits to publicly is plain: the commercial service tier will always be a separate entity, so the foundation never competes with builders on its own commons. The law stays free to run; the convenience of not having to run it yourself is the only thing with a price.

An honest failure

A method is worth only as much as the tests it is willing to fail, so here is one Axiom failed. In July 2026 it ran an experiment on a seductive premise: that encodings are essentially cache. If every rule is deterministically derived from source text, the reasoning went, no encoding needs to be a durable artifact—you could discard any module and regenerate it from the corpus on demand, cheaply, whenever it was wanted. The experiment, known internally as the B1 probe, put that premise to twenty-five modules.

It failed, and the way it failed is the useful part. Regeneration was genuinely cheap—about eight cents of compute per module—but in twenty-three of the twenty-five cases the original encoding was the correct one, and the freshly regenerated version introduced naming instability that would have broken every downstream consumer. Same source, same pipeline, different output; the difference was silent damage. The conclusion Axiom adopted cuts against the elegant version of its own story: encodings are not disposable cache. They are durable artifacts with provenance, and regeneration, at the current configuration, is not yet trustworthy enough to lean on. The finding changed how encodings are treated—as artifacts to be stored, versioned, and migrated with care, not outputs to be discarded and re-derived—and regeneration went back on the shelf as a research problem rather than a shipping feature.

That is the discipline working, not failing. A project convinced of its own story would have shipped regeneration and met the breakage in production. A project that runs the probe, reads the result, and writes the negative finding down is one whose other claims are worth more for it.

Which leaves the question this chapter has circled without answering. Encoding all of public policy at agent speed is worth nothing unless the encodings are right, and "a language model wrote it and the tests passed" is not, by itself, a reason to trust a number that decides whether a family makes rent. Compilation proves the code runs. The money-atom gate proves each dollar traces to the law. Neither proves the encoding matches the world.

That proof is a separate discipline, and it is where the real confidence in this method is won or lost: comparison, case by case, against independent calculators built by other people with other tools—and the harder practice of explaining, rather than waving away, every place the two disagree. How to trust law encoded by machines is the subject of the next chapter.

Chapter 10: The verification problem

The first time we raced an encoding of Belgium's social-security contributions against EUROMOD, the two disagreed. Our version computed roughly €20.9 billion in employee and employer contributions across a simulated population of workers; EUROMOD, the European Union's reference tax-benefit model, came out €643 higher.

Was that €643 a success or a failure?

The number alone cannot tell you, and the reason it cannot is the reason this chapter exists. Six hundred forty-three euros is consistent with a dozen different underlying realities. It could be floating-point dust—hundreds of thousands of rounding differences, each a fraction of a cent, smeared across the population and piling up into something that looks like money. It could be a single catastrophic error: one worker whose contributions the encoding got wrong by €643 while every other record matched to the euro. It could be a small systematic bias, a rule applied a hair too generously to everyone. Or it could be the worst case of all, the one that looks best from a distance: large errors in opposite directions—tens of thousands of euros too high in one place, tens of thousands too low in another—canceling almost perfectly on their way to a reassuring total.

An earlier chapter watched that last failure mode play out in the wild. When CBO scored the Affordable Care Act in 2010, its projection of the total uninsured rate landed within a percentage point of what actually happened—while getting the composition badly wrong, overstating exchange enrollment by more than half and understating Medicaid by nearly as much, the two errors partly canceling on the way to a roughly-right aggregate. The total was approximately right for the wrong reasons. If your only test is whether two totals are close, you cannot distinguish a model that is right from a model that is wrong in a flattering way.

So we did not compare totals. We compared workers. For each of the 78,479 simulated workers, we lined up the encoding's contribution against EUROMOD's and took the difference. The largest single-worker discrepancy in the entire population was three cents a year. Not three cents on average—three cents at the very worst. Every one of the 78,479 workers matched the reference model to within the width of a rounding error, and the €643 in the aggregate turned out to be exactly what it looked like once we refused to look at it from far away.

Aggregate agreement can hide anything. Per-unit comparison hides nothing.

That sentence is the whole chapter, and it is the answer to the question that hangs over everything agents now build: how do you trust law encoded by a machine? You don't. Trust is the wrong frame. You verify, one unit at a time, against something that is true independently of the machine—and you wire the verification so tightly into the pipeline that an encoding which fails it cannot ship. Verification is not a quality-assurance step bolted onto the method. It is the method. An encoding no one has checked against ground truth, at the level of the individual determination, is not a cheaper version of a validated model. It is confident fiction with good production values.

The ladder of checks

Before an encoding is ever raced against another model, it has to survive a more basic demand: every number in it must come from somewhere. In the pipeline we built at the Axiom Foundation, the first gate is one we call the money-atom check, and it is merge-blocking. Every monetary obligation in an encoding—each rate, threshold, taper, and cap that actually moves money—must be grounded in a quoted excerpt from the governing source: the statute, the regulation, the agency manual, quoted verbatim with a pointer to where it lives. An amount with no provision behind it does not get flagged for a reviewer to consider later. It fails the build, the way a syntax error fails the build. When we finished the Belgian encoding, it carried 539 distinct monetary obligations, and the gate demanded a source excerpt for every one of them; the count of ungrounded obligations was zero. The interesting part is not that it reached zero. It is that the gate is configured so the count can only ever fall.

Grounding is necessary but not sufficient. A number can be faithfully quoted from the statute and still be wired into the wrong formula, or into the right formula with the wrong sign. So the encoding also has to compile, pass the companion tests that ship beside every rule, and clear a proof pass that checks the logic holds together—that the pieces refer to quantities that exist, in units that match, along paths that resolve. These are the unglamorous gates, the ones that catch the errors a careful person would eventually catch too, only faster and without getting tired. They establish that the encoding is internally coherent and externally sourced. They do not establish that it is correct.

For that, the encoding is raced against an oracle. An oracle, in this sense, is an independent implementation of the same law—usually built by other people, often over many years—whose answers we treat as a second opinion we did not write and cannot quietly influence. The oracles are the best independent tax-benefit models in the world. TAXSIM, the NBER calculator that has anchored academic tax research for four decades. The classic PolicyEngine engine. UKMOD, for the United Kingdom. EUROMOD, for the member states of the European Union. The SOUTHMOD family that UNU-WIDER maintains for a growing list of developing economies. And, wherever a tax authority publishes one, its own official calculator. Chapter 3 laid out three levels at which any such model is validated—component, aggregate, and predictive. What the oracle program does is take the first level, component validation, the one that used to mean spot-checking a married couple with $100,000 of income against an IRS worksheet, and industrialize it: replay entire populations through two engines and demand agreement on every record. It is component validation grown teeth.

There is a reason to race whole populations rather than a curated handful of cases, and it is not thoroughness for its own sake. A test suite a person writes tests the situations that person thought of. It will faithfully check the married couple, the pensioner, the family with three children—and it will say nothing at all about the case nobody imagined: the worker who crosses two thresholds in the same year, the household that lands in the seam between two rules, the combination that never came up in anyone's examples. Replaying a full calibrated population through two independent engines tests those cases too, because the population

contains them whether or not anyone anticipated them. The pathological record that would otherwise have shipped undetected is precisely the one a broad enough replay puts in front of you.

What parity looks like

The United Kingdom is where the encodings and the oracle agree most exactly, and it is worth being specific about what "exactly" means, because the word gets used loosely. For a deterministic calculation there is a single right answer, and a tolerance band is mostly a comfortable place for a wrong answer to hide. UK income tax matches to the pound—not close, not within a tolerance, to the pound—across five test suites, including the awkward withdrawal of the personal allowance, the taper that claws back the tax-free band from high earners and produces the marginal-rate spike that trips up so much informal tax advice. The Scottish income tax, which runs on its own six-band structure, matches on nine cases out of nine. Savings income and dividend income, each taxed on its own schedule with its own allowances, match exactly. National Insurance matches or is explained: where the encoding and UKMOD diverge, the divergence traces to a documented convention—UKMOD computes contributions weekly and then annualizes, while the encoding works in annual terms—and once you account for the convention, the two reconcile.

Then there is Universal Credit, where the raw numbers look, at first, like a problem. Of nineteen Universal Credit test cases, eight matched. Eight out of nineteen would be a damning result if the eleven gaps were errors. They are not. UKMOD applies a stochastic take-up draw: household by household, it decides at random whether an eligible family actually claims the benefit it is entitled to, because in the real world many eligible families never claim. Our encoding computes statutory entitlement—what the household is owed if it claims. Those are different quantities by construction, and every one of the eleven mismatches resolves precisely to that difference. Nothing is wrong. The two systems are answering two different questions, and once you know which question each is answering, the gap is not a discrepancy but a definition.

This is where one word carries the load: disposition. In the conformance program, a raw mismatch is neither a failure nor a pass. It is an open question, and it stays open until it is closed with evidence. An explanation, in this vocabulary, is not a shrug and not a hand-wave. It is a checked artifact—an arithmetic reconciliation that reproduces the gap to the penny, a citation to the statutory provision the oracle implements differently, or a documented account of an oracle's own modeling convention, like the Universal Credit take-up draw. A mismatch is either dispositioned with one of those, or it is unexplained, and unexplained mismatches are the only kind that count against the encoding. Cast that way, the standing UK result is the one that actually matters, and it is not the raw match rate. It is this: across every compared surface, one hundred percent of mismatches explained, zero unexplained.

Belgium tells the same story through a different institution. The regional and municipal surcharges that sit on top of Belgian income tax match on nine cases out of nine. A greenfield encoding of student study allowances—a body of rules with no prior implementation anywhere to copy from—matches on six out of six. The contributions match per worker to three cents. Every mismatch on the Belgian conformance board carries its disposition, and none of them is attributed to an error on our side. The point of saying so is not to boast about a clean board. It is that "clean" here has an auditable meaning: not that nothing disagreed, but that everything that disagreed was run to ground and written down.

When the oracle is wrong

The dispositions discipline has a consequence we did not fully plan for. If every mismatch must be chased to evidence, then sometimes the evidence exonerates the encoding and implicates the reference model. Race a fresh, exhaustively grounded implementation of a law against a model that people have maintained by hand for years, and now and then the fresh implementation turns out to be right where the mature one is wrong.

What makes that possible is not cleverness; it is exhaustiveness. Nobody maintaining a mature model by hand replays tens of thousands of records against a second independent engine on every change—until recently the compute and the tooling to do it that cheaply did not exist. When you do, an inconsistency that surfaces in only a handful of unusual cases stops hiding in the tail of a distribution and shows up as a specific record with a specific wrong number, which is a thing a person can then investigate. The precision that catches float dust at three cents is the same precision that catches a genuine bug.

Chasing one such mismatch through the UK savings rules, we found the discrepancy was not in our encoding at all. UKMOD was mishandling an interaction involving the personal savings allowance—a corner of the rules where two provisions meet and the combined behavior is easy to get subtly wrong. We wrote it up the way the discipline demands: the inputs, the statutory result we expected and why, the reference model's actual output, and the provision each of us was implementing. Then we filed it publicly, as an issue on the UKMOD-PUBLIC repository, for its maintainers to weigh.

A second case was stranger. Running the EUROMOD comparison, we noticed that identical input cases were sometimes scoring differently depending on where they sat in a batch—the same household, the same year, two different answers, the result determined by its neighbors in the queue rather than by its own circumstances. That is the fingerprint of state leaking between records that are supposed to be independent, and we root-caused it to a batch-processing contamination bug in the adapter layer that runs cases through the model. We reported it to the maintainers with the reproduction in hand.

It would be easy to tell these as gotcha stories. They are the opposite. These reference models are the reason the encodings can be trusted at all; strip the independent second opinion away and "the machine says so" is exactly the epistemic position this whole project

exists to escape. TAXSIM has been maintained for forty years. EUROMOD and its national variants represent decades of patient institutional work by the people who built the field of tax-benefit microsimulation in the first place. That a model of that size contains a bug is not a scandal—every large model does, ours emphatically included—and finding one is not a victory over its authors. It is what collaboration looks like when it runs in both directions. A reference model built and maintained across decades, in the act of validating a newcomer, gets something back that it rarely gets from anyone: free, specific, evidence-backed quality assurance from a system precise enough to notice a three-cent gap and disciplined enough to chase it to its cause. Openness means the errors flow both ways. We file our bugs upstream and theirs downstream, and both models come out better.

A one-way valve on correctness

The hardest honest objection to any public rules layer is not that it will be wrong on the day it launches. It is that it will rot. Laws change every year; the failure mode of shared infrastructure is not a dramatic collapse but quiet staleness—encodings that drift out of date as statutes move, coverage that erodes at the edges, a model that was accurate when it shipped and is subtly wrong three budgets later, with no one the wiser. Chapter 9 named that as the real governance fear, the one that durable funding alone does not answer. A verification suite that runs once and is admired is no defense against it. A verification suite wired into a ratchet is.

The conformance gates are configured as one-way valves. Coverage—the number of statutory surfaces under test against an oracle—may only rise; a change that would drop a jurisdiction or a program below its current tested coverage fails continuous integration and cannot merge. The count of unexplained mismatches may only fall; a change that introduces a new unexplained divergence, or that quietly un-explains one that was previously dispositioned, fails too. The count of ungrounded monetary obligations may only fall toward the zero it already reached. None of these numbers can slip the wrong way by accident, because the pipeline refuses to let a commit that moves them the wrong way in at all.

The valves hold because the full conformance suite runs on every change, not on a schedule someone has to remember to honor. When a new budget moves a threshold, the new value lands as a dated addition sitting beside the old one, and it has to pass against the oracle for its own year before the change can merge—last year's cases still checked against last year's law, this year's against this year's. Nothing is overwritten in place, so nothing quietly falls out of coverage when the parameters move. The moment the encodings drift from the law is the moment the build stops going green.

This converts an anxious, human, permanently-behind maintenance problem into a mechanical invariant. Staleness does not become impossible—a rule can still go un-encoded, a new program can still escape coverage entirely. But silent staleness does become impossible, and silence was always the dangerous part. The ratchet makes no promise that the encodings are complete.

It promises something narrower and far more enforceable: that correctness, once won and checked, cannot be lost again without the build going red for everyone to see. That is a weaker guarantee than "always current," and a much stronger one than "trust us to keep it current."

The gates are for us first

It would be comfortable to describe all of this as a wall against untrustworthy AI—a filter that catches the machine's mistakes before they reach anyone. That is true, and it is not the honest emphasis. The gates catch our own mistakes first, and the most useful thing I can report about them is the times they turned red on us.

One premise we genuinely held, and set out to test, was that encodings are disposable. If an agent can encode a statute from source, the reasoning went, then any given encoding is just a cache: regenerate the module on demand from the underlying law and throw the stored copy away. It is an appealing idea, and it would have meant the artifacts barely mattered—only the source and the pipeline would. In July 2026 we ran the experiment. We took twenty-five existing modules, regenerated each one fresh from its source, and compared the new output against the version already in production. Regeneration was almost free, about eight cents of compute per module. The results refused to cooperate. Of the twenty-five, twenty-three came back with the old, in-production version correct and the freshly regenerated version worse—most often by introducing naming instability, renamed variables that would silently break every downstream consumer still referring to the old names. The premise failed. Encodings are not cache; they are durable artifacts with provenance, and regeneration, at least as we currently run it, is not yet trustworthy enough to be automatic. We know that not because we reasoned our way to it in advance but because a gate compared regenerated output to a known-good baseline and twenty-three of twenty-five came back wrong. The discipline told us something we would rather not have heard.

The deeper lesson was about provenance. An encoding is not just an answer; it is an answer with a history—which source it came from, which version of the law, which call was made where the text was genuinely ambiguous. Regenerating it from scratch discards that history and, worse, produces a new artifact that looks every bit as authoritative while quietly disagreeing with the one that every downstream system already relies on. Durability here is not sentiment about an old file. It is the property that lets a consumer trust that the number it read yesterday still means the same thing today, and that when the number does change, it changed because the law did.

It was not the only time. The benchmark from Chapter 8, the one that measures how badly language models do full household calculations, went through its own version of the same lesson. An early layer that narrated each result in prose was caught inventing derivations—in one case describing elderly enrollees as beneficiaries of a "non-elderly expansion"—and any audit that read those confident narratives inherited the fabrication whole. The fix was the same fix it always is: ground the explanation in the engine's actual internals, and add a validator

that rejects any mechanism the underlying trace does not support. Even the audit layer needs ground truth; you give the judges traces, not prose. And when we adapted a US-calibrated population to the United Kingdom and wrote up the result, a summary statistic that no run had actually produced—a fabricated number—found its way into a draft and was caught at a verification gate before publication, and removed.

None of these is an embarrassing exception to an otherwise spotless record. They are the record. A verification regime that only ever vindicates the people who built it is not a verification regime; it is a marketing department with test coverage. The gates earn their standing precisely by the frequency with which they turn red on their own authors.

The admission rule

Underneath all of it sits a single rule, and it is the rule the rest of this book is built to honor. Simulation is admissible only where its verification chain terminates in ground truth—a forecast score, an oracle parity, a statutory exactness, a calibrated marginal. Not a plausible-looking number. Not a fluent narrative. Not an aggregate that lands in the right neighborhood. A chain of checks that bottoms out in something true independently of the model that produced it: a reference implementation, a quoted statute, a resolved outcome, an administrative total. Each terminus is a different kind of true thing, checked a different way. An oracle parity is the subject of this chapter—an independent implementation agreeing on every record. A statutory exactness is the money-atom carried to its end, an obligation traced to the quoted provision that fixes it. A calibrated marginal is an input distribution reconciled against a number the government actually counted, so the population the rules run on is not merely plausible but pinned to something real. And a forecast score is the terminus that only arrives later—a prediction with a published interval, graded when the official figure lands. They are not interchangeable, but they share the one property that makes any of them admissible: each bottoms out somewhere the model's own confidence gets no vote. Where that chain exists and holds, the estimate is admissible, and you can say how you know. Where it does not, the estimate—however fluent, however cheap, however fast the agent produced it—is confident fiction, and the honest move is to say so and mark the boundary rather than let the fluency stand in for the check.

Which surfaces the question this chapter has quietly leaned on the whole way through. Everything here rests on the existence of an oracle: a UKMOD to disagree with, a EUROMOD to file bugs against, an official calculator to match to the pound. The verification chain for an encoded rule terminates cleanly for the United Kingdom and Belgium and the United States because, for those places, a second opinion already exists. But most of the world's tax and benefit law has never been encoded by anyone, and for much of it no reference model exists at all—no calculator to race against, no mature implementation to disagree with, no independent answer to check. The discipline that makes the encodings trustworthy is built entirely on having something to check them against. What becomes of it where there is nothing?

Chapter 11: Microsimulation anywhere

In the second week of July 2026, five countries acquired public, inspectable models of their own tax and benefit systems. None of them had one before. The work took about a week.

For most of this field's history, a national tax-benefit model has been the work of years — a team, a budget, a data-sharing agreement, a maintenance commitment measured in decades. That is why the models chapter 2 described lived inside finance ministries and a handful of well-funded institutes, and why most of the world has none. A researcher in Lusaka or Kampala who wanted to know whether a proposed change fell hardest on the poorest fifth of households had, until recently, to wait for a ministry to answer or to take the answer on faith. The week in July is the story of that wait collapsing — and, just as much, of what has to be true for the collapse to be worth anything.

Five countries is not a large number, and this is not a finished project. But the count was never the point. Two things had to be true at once for the week to mean anything: that a model could be built this fast, and that each one arrived carrying the evidence for whether it was right. Most of this chapter is about the second condition, because without it the first is a liability.

It started on July 8, with two countries finished on the same day. Ghana came first: its 2025 tax-and-benefit system, encoded provision by provision and tested across its full surface against GHAMOD, the Ghanaian member of SOUTHMOD — UNU-WIDER's family of tax-benefit models covering fourteen developing economies. The comparison produced thirteen findings, thirteen catalogued places where the encoded statute and the reference model disagreed, each one run down, explained, and posted to a public ledger anyone can open and read.

Uganda was finished the same day, in a single day of work, reusing the recipe Ghana had just proven. Its ledger recorded no findings at all. The encoding was, in the phrase recorded in that ledger, statutorily exact everywhere tested; the widest disagreement anywhere in the suite was four-tenths of one Ugandan shilling per year — rounding, and nothing else.

Thirteen and zero, produced the same day by the same method. Neither is the number you would reach for as a headline, and both are good news — but only once you know what a finding is, which is most of this chapter.

By the end of the week there were five. Zambia landed on the 9th. Ethiopia landed overnight. Rwanda closed it out. Two more sat at the edge of the week, and they matter more than the tidy five.

The rest of the week

Zambia's system was encoded in about a day and checked against MicroZAMOD, and it returned eight findings. It was the first of the five to turn up divergences on both the tax and the benefit sides of the system rather than the tax side alone — among them a definition of the pay-as-you-earn base, and a set of excise rates that had been carried a year stale. Eight findings on a first pass across an entire tax-benefit system is a low number, and the fact that they straddled both sides is itself informative: it is where the two halves of a system meet that encodings and reference models most often drift apart, and Zambia was the first case with enough surface tested to see the drift on both.

Ethiopia was done in a single overnight run and compared against ETMOD. Two findings, and both sat in the same place: the new minimum alternative tax introduced only months earlier, by Proclamation 1395/2025. A statute that new has barely been implemented by anyone, and two first-pass disagreements about how it works are close to what you would predict — both of the understandable kind, reasonable readings of a fresh law that differ, rather than arithmetic that is simply wrong.

Rwanda closed the week with five findings against RWAMOD, and it was the first lane where the comparison was executed live rather than replayed from a fixture: the encoded model and the reference model were handed the same eighty-six cases and asked to agree. The run surfaced the five findings, and once those were dispositioned the two agreed across all eighty-six.

Then the two that matter most. Tanzania is under way. Nigeria has begun — and Nigeria has no oracle. It is not one of the SOUTHMOD countries, and no other reference model or administrative ground truth is available to check the encoding against. This book has held to one rule about when a simulation may be believed: only where its chain of verification ends in something real — a reference model it matches, a statute it reproduces exactly, a number the world will later publish. Nigeria's chain does not yet terminate in any of those. The encoding can still be built, and it has been; but the claim that it is correct stays weaker than the same claim for Ghana or Uganda, and the honest thing is to say so in the same breath that reports the encoding exists. The boundary of the method is part of the method.

What a finding is

A finding is not a bug report and not a failure notice. It is a dispositioned divergence: a specific case where the encoded statute and the reference model produce different numbers, chased until someone can say why, and recorded together with the evidence for the explanation. "Explained" here is a checked artifact, not a shrug — an arithmetic reconciliation, a citation to the governing text, or a documented convention of the reference model. A divergence nobody

can account for is not allowed to sit quietly; it is the one kind of result the process treats as unfinished.

The explanation can land on either side. Sometimes the encoding is wrong — a misread threshold, a missing phase-out, a rate applied to the wrong base — and the fix goes into the encoding. Sometimes the reference model is the one that is wrong, or out of date, or has made a defensible modeling choice the statute does not actually require; and then the finding travels the other way, written up with its arithmetic and its citation and reported upstream to UNU-WIDER, who maintain the SOUTHMOD models. The reference models are not adversaries in this. They are the instruments that make the check possible at all, and like any instrument they are improved by being read carefully against their source. The countries that never had a public model get one; the reference models get free, evidence-backed quality assurance in return. Openness runs in both directions, or it is not openness.

There is a quiet reversal of trust in that arrangement, and it is worth dwelling on. The usual assumption is that an established, decade-old reference model is the authority and a freshly generated encoding is the thing on trial. Most of the time that is right, and most findings are the encoding's to fix. But not all of them. When a divergence is chased to ground and the statute plainly says one thing while the reference model does another, the encoding becomes the instrument that caught it, and the finding is a contribution back to a model that researchers across the field rely on. Neither model is presumed correct; the statute is, and both are measured against it. That is what keeps the exercise collaborative rather than competitive — nobody is grading anybody, and everybody is grading the law.

Each country's ledger is a public GitHub discussion. Ghana's thirteen are there to read in full; so are Uganda's zero, Zambia's eight, and Ethiopia's two, with Rwanda's five alongside them. Which is what makes Uganda's empty ledger worth as much as Ghana's full one.

Zero findings is not the absence of a result. It is one.

UGAMOD turned out to be statutorily exact everywhere the suite reached, and the encoding matched it to the shilling — a fact established by evidence, not assumed by silence. A model that has been checked against an independent reference and agrees is in a different epistemic state from a model no one has checked, even when the two would print identical numbers. Uganda's zero is a compliment paid to a reference model, in evidence, and it is in its way the most reassuring of the five results: it says the method can tell the difference between right and merely unexamined.

The recipe

The recipe was the same in every country. Capture the governing acts from the official gazette, each provision anchored to a page in a signed source manifest, so that every rule traces back to

the exact text it came from. Encode the system one instrument at a time. Put every change through the same merge-blocking gates the previous chapters described — if it does not compile, does not pass its proofs, does not clear the oracle comparison, it does not merge. Run the oracle suites first case by case, then across a whole simulated population, so that agreement holds not only on the examples someone thought to write but on the shape of the distribution. Publish the findings ledger. Report what it turned up back upstream. And once a country reaches parity, the gates ratchet: coverage can only rise and unexplained gaps can only fall, so the model cannot quietly rot back to wrong. The cost of all this, per country, was measured in days of agent time — not the years of institutional effort that a national tax-benefit model has meant for the entire prior history of the field. Across five countries with five different statutes, the recipe barely changed.

The moat was never only American

Chapter 2 called it the analytical moat: government tax models run on the universe of actual returns, while outside analysts get public-use files that are sampled, top-coded, and blurred, and no amount of open-source software fully closes the gap. That was told as an American story, with an American statute — Section 6103 — as its cause.

But the moat was never only American, and in most of the world it is not really a moat at all. It is a vacuum. The situation is not a gap between the government's model and a challenger's model; it is the absence of any public, inspectable model on either side. A citizen of most countries on earth cannot check what a tax change would do to a household like theirs, because no one outside a ministry has built the tool, and often no one inside one has published it. The asymmetry chapter 2 diagnosed in Washington — where at least half a dozen institutions could argue over a tax bill within days of its introduction — is, across most of the map, closer to silence.

Consider what a public model changes for the people around a single tax decision. When a finance ministry proposes to broaden a value-added tax, or to replace a fuel subsidy with a cash grant, the question everyone actually cares about — who gains, who loses, and by how much across the income distribution — has an answer that lives inside a model. Where that model is the ministry's alone, the answer arrives as an assertion, and the opposition, the press, the civil-society analyst, and the affected household can only accept it or reject it whole. Where the model is public and checkable, the same people can run the reform themselves, disagree about assumptions in the open, and be wrong in ways that others can catch. The point is not to take the tool away from the ministry. It is to end the ministry's monopoly on being able to answer at all.

What the week in July changed is the marginal cost of ending that silence. Building a national tax-benefit model went from a multi-year institutional undertaking to a job of days, and there

are fourteen countries in the SOUTHMOD family alone with reference models standing ready to check the work. That much, on its own, would be worth reporting. But a model that costs days and that no one can check is not an advance over a model that costs years — it is only a faster way to be confidently wrong. What actually traveled from Ghana to Uganda to Zambia was not the encoding speed by itself. It was the discipline bolted to it: the gazette manifests, the merge gates, the oracle suites, the public ledger of everything that did not match. The verification shipped in the same box as the model. That is the entire point of the exercise, and it is the only part worth being emphatic about.

When the data can't travel

Rules are the easy half. A statute is public by definition; encoding it needs the text and a way to check the result, and this chapter has been about how cheap both of those have become. The harder half is the people. A tax-benefit model is only as good as its picture of who actually lives under the rules — at what incomes, in what households, drawing which benefits — and that picture comes from microdata, and microdata is where the openness usually runs out.

The asymmetry is not incidental; it is structural, and it runs the opposite way from what you might guess. A statute is written to be public — promulgated, gazetted, binding precisely because anyone can read it — so encoding it in the open takes nothing that was ever meant to be hidden. Microdata is the reverse: it is a record of actual people, protected by law and by promise, and its value to a model comes from exactly the granular, individual detail that makes it impossible to release. Law is portable because it is already public; microdata is stuck because it is already private. Any honest attempt to build models everywhere has to be designed around that fact rather than in spite of it.

So the summer's second experiment asked a harder question than the SOUTHMOD week did. Take the calibrated synthetic population that the data layer builds for the United States — populace, the commons introduced in Part II — and adapt it to a country it was never built for. Then grade it, honestly, against that country's real data. The United Kingdom was the natural test case, because it has both an open reference model and a high-quality household survey to grade against. The adapted population was run through a pre-registered evaluation harness — the scoring rules fixed and published before any of the answers were known — and scored against held-out ground truth from Britain's Family Resources Survey.

The verdict was mixed, and it was reported as mixed: a credible first pass for aggregate, earnings-centric analysis — not a substitute for native microdata on benefits, distribution tails, or subnational detail. Two specifics carried it. One was a defect found and fixed: a stale data feed had zeroed out state pensions, and the error cascaded downstream into pension-credit calculations; it was caught, traced to its source, and corrected. The other was a defect found and deliberately left open, because it cannot be papered over. Capital gains had been carried

across one-to-one into a survey surface that captures almost none of them, and the honest fix is not to copy realizations from one country into another but to model asset positions and the rules under which gains are realized — a rebuild, designed but not yet done. The transferred population is good at what surveys are good at and blind where surveys are blind, and pretending otherwise would have been easy and wrong.

One smaller episode from the same work is worth naming, because it is part of why the verdict can be trusted at all. A summary statistic in an early draft of the write-up turned out to be fabricated — fluent, plausible, and unsupported by the run it claimed to describe — and it was caught at a verification gate before publication and struck. The gates in this book are aimed first at our own output, not at anyone else's. A discipline that only checks other people's work is not a discipline; it is an accusation.

Where both halves can be built, they close tightly. In Belgium, the same machinery was calibrated to twenty-one administrative targets and hit all twenty-one within 1.8 percent; and on the same population, per-record social-security contributions agreed with the EUROMOD reference model to within three eurocents per person per year. That is what a trustworthy result looks like when the reference model exists and the population can be built and run: not a promise that the model is right, but a small, checkable number that states exactly how close it came.

An open referee

Belgium points straight at the harder case. For many countries the microdata simply cannot leave the building. National statistical offices hold survey and administrative records under legal obligations that do not bend for a research project; you cannot download the file, and you should not want to be able to. That looks, at first, like the end of the road for open modeling in those places: you can encode their laws in the open, but you can never open their data, so how would anyone outside ever know whether a transferred population resembles the real one?

The answer is to stop trying to move the data and to move the referee instead. Give the institution that holds the microdata two things: the transferred population, and a pre-registered evaluation pack — the scoring code pinned to an exact version, the configuration frozen, and a scorecard schema that fixes in advance which numbers will be reported. The institution runs the pack inside its own walls, against its own protected records, and publishes only the scorecard: a short, low-dimensional set of results saying how well the transferred population matched the ground truth it was graded against. Because the scorecard was specified before the run, no one can fish the data for a flattering statistic after the fact; the referee's questions are fixed before it ever sees the answers. No record leaves the building. The score does. An

outside analyst never sees a single citizen's data and still learns, in public and on terms agreed in advance, whether the open model can be trusted for that country.

A scorecard is a deliberately small thing — a handful of numbers, fixed in advance, that say how closely the transferred population reproduced the real one on the dimensions that matter: how the income distribution lines up, how many households the model places on each benefit, how far the tails diverge. It carries none of the underlying records and none of their risk, and it is exactly the kind of low-dimensional summary a verification chain is allowed to end in. You do not need to see a country's microdata to trust a model of it. You need a referee who has seen it, a test written down before the answers were known, and a score published where anyone can read it. That is less than full openness, and it is enough.

Open microsimulation anywhere doesn't require open microdata anywhere — it requires an open referee.

That sentence is the thesis of the chapter, and it dissolves the tension the chapter has been carrying. The encoded law can be fully open, because law is public. The microdata can stay fully closed, because it must. What sits between them — the referee — is a small, honest, pre-committed test that either party can run and everyone can read. The verification chain still terminates in something real: not a dataset anyone downloaded, but a scorecard a custodian ran and stands behind.

Notice what made every step of this possible. The rules, the data, and the prediction never had to travel together. Ghana's statutes could be encoded in the open while Ghana's household records stayed home. A population built for the United States could be shipped to the United Kingdom and graded there. A statistical office could run a referee no outsider was ever allowed to watch. Each of those moves works only because the law, the population, and the forecast are genuinely separable things — built by different methods, checked against different ground truth, and, it turns out, best kept in different hands. So far that separation has been a technical convenience. The next chapter is about what happens when an organization decides to take it seriously enough to build itself around it.

Chapter 12: The decomposition

Organizations restructure for ordinary reasons — a merger, a funding round, a founder's exit, a board that wants cleaner reporting lines. It is rare for one to restructure around a claim about knowledge.

This one did.

Building the engine had taught its builders something they did not set out to learn: that rules, data, and prediction are separable concerns. For most of a decade the three had lived

together inside one tool — one set of repositories, one brand, one habit of mind — and the joints between them stayed invisible because the bundle worked and nothing ever pulled on them. Verification pulled on them.

Once you require each part of a policy estimate to prove itself against ground truth, the parts stop resembling one another. A rule proves itself against the statute it encodes and against a reference calculator — TAXSIM, UKMOD, a SOUTHMOD model — that has read the same statute and either reproduces the encoding's answer to the last cent or forces the difference to be explained. It does this now, at the moment the rule is written, before it touches a single household. A population estimate proves itself differently: against surveys, and against the administrative totals the government already publishes — a calibration you can check the day you build it. A prediction can prove itself in neither way, because the thing it describes has not happened yet. It waits. It is scored only when reality arrives and the official number lands.

Three kinds of claim, then, on three different clocks. What is true of a rule is settled today, against a document. What is true of a population is settled today, against a count. What is true of a forecast is not settled until some future Thursday when a statistical agency posts a release. And the clocks turn out to imply the failure modes. A rule fails by contradicting the law — a bug against a fixed target, findable and fixable. A calibration fails quietly, by drifting away from a population it used to resemble. A forecast fails loudly and unarguably, by being wrong about the world — a miss you cannot litigate away and can only record. You do not manage these three failures with the same reflexes, or the same people, or — this is the chapter's claim — inside the same institution.

Take a single tax credit. Whether it has been encoded correctly is answerable this afternoon, against the section of the code that defines it and an oracle that computes it independently. Whether the households it reaches look like the country is answerable this afternoon too, against totals the tax agency already reports. But how many families will actually claim it next filing season is answerable only next filing season, when the agency counts them. The first two are audits. The third is a bet.

Axiom states, Thesis predicts

The reshaping that followed took its cue from that observation rather than from the forces that usually move an org chart. PolicyEngine divided along the seam that verification had exposed, into two poles that are opposite in the most elementary way two claims can be.

An axiom is an accepted truth. It is not on trial; it is the thing you reason from. The question *what does the law say?* has exactly that shape — the answer was fixed by a statute someone already wrote, and the only live question is whether an encoding faithfully matches it. A thesis is the mirror image: a proposition advanced in order to be tested, granted no standing until the evidence rules on it. The question *what will happen?* has that shape instead — no document settles it, and any answer stays provisional until the world supplies the verdict. Two words,

two epistemic tempers, and the entire reorganization is an attempt to stop housing them in one body.

The Axiom Foundation is the determinism pole. It encodes law exactly — federal statutes, state codes, agency manuals, the council-tax reduction scheme of a single London borough — and it asks its agents to state, never to guess. Its public launch is set for July 28, 2026. Everything it ships is meant to be right the way a proof is right: checkable line by line against a source, and wrong only if the source is.

The Thesis Institute is the prediction pole. Its product is a public docket of forecasts about government metrics — the first print of the unemployment rate, a program's caseload, a wait time — including forecasts made under explicit policy counterfactuals. One live page asks what Medicaid call-center wait times will be in March 2027 if the work-requirement deadline is delayed: a number that does not exist yet, about a world that has not happened yet, staked out in advance. Its public launch is set for around September 15, 2026. Everything it ships is meant to be provisional — a value with an interval around it, entered into the record early enough that reality can grade it.

Opposite as they are, the two poles are built to compose. The Medicaid question is really two questions stacked. What the rule would be if the deadline slipped is an axiom — encoded exactly, by the determinism pole. What the caseload and the wait times would then do is a thesis — forecast, and graded, by the prediction pole. A policy counterfactual is precisely that: an encoded change to the law, run forward against a population, scored against the world that results. The determinism pole supplies the exact *if*; the prediction pole supplies the uncertain *then*. Separating them into two institutions does not cut the pipeline between them. It makes plain which half of every counterfactual is a matter of settled fact and which half is a matter of open forecast.

Here the discipline this book has been pressing becomes a literal feature of the product. Every forecast on the docket carries a published interval and will be graded when the official number lands. As of this writing, none has resolved. The scoreboard is live; the grades arrive with reality.

The engine, recomposed

Where does the original tool go, in a world divided between an axiom pole and a thesis pole? It goes to both, because it was always both, bundled together tightly enough that no one had to notice. For years one team shipped one release, and the rules, the data, and the projections rode out together; nothing in the daily work ever forced the question of which of them was a proof and which was a guess. The oracles and the scoreboard forced it, each part suddenly made to answer for itself. PolicyEngine is being recomposed into the two layers that were underneath it the whole time. Its rules — the thousands of encoded provisions that answer what a household owes and is owed — become part of Axiom. The calibrated microdata beneath every population estimate becomes populace, the shared substrate both poles stand

on. The model code that used to live in PolicyEngine's own repositories retires downward into those two layers, and the product on top keeps its name, its interface, and its users.

That last point is the one to hold onto, because it is where the recomposition stops being an internal diagram and starts mattering to anyone outside. The teacher checking a family's benefit cliff, the journalist scoring a bill, the congressional staffer running a distributional table — none of them sees a seam. The thing they use answers the same questions it always did. What changed is underneath: it now stands on two legs that can each be checked, a rules layer verified against statute and a data layer verified against administrative totals, rather than on one monolith whose internals you had to trust as a whole.

Populace deserves one plain sentence and no more fanfare than that. It is the calibrated-microdata commons — households synthesized and reweighted until the totals they imply line up with the counts the government already publishes — and it is the ground the entire stack rests on, because an exactly encoded rule and an honestly scored forecast both still have to be applied to a representative population before they mean anything about the country. The methods matter; the brand does not. Name it once, and lean on the work.

What the recomposition actually buys is not neatness. It is accountability, of a kind the field has never really had. Route the population layer into a forecast, post that forecast on the Thesis docket with an interval and a resolution date, and a policy estimate stops being a number a model asserts and becomes a claim the world will eventually grade. "This reform costs fifty billion dollars" turns into "this reform costs fifty billion dollars, here is the interval, and here is the date we will find out." That closes a loop the institutions producing official estimates leave open. The Congressional Budget Office publishes a score and moves on. The Tax Policy Center publishes a distributional table and moves on. EUROMOD publishes and moves on. Nothing circles back to keep the tally. The recomposition wires the tally in — not as a report card imposed from outside, but as the native output format of the prediction pole.

Wrongness, isolated

Splitting the poles into separate institutions is not brand hygiene. It is a firewall around one specific hazard: being wrong.

The Thesis Institute has to be wrong in public, and often. A docket that never missed would be a docket with its intervals padded wide enough to be useless, or one quietly declining to post the questions that are actually hard. The worth of a graded forecast comes precisely from the fact that some of the grades will be bad — visibly, on the record, with the interval printed beside the miss — because that is the only thing that makes the good grades worth anything. Public wrongness is not a risk the prediction pole runs despite its mission; it is the mission. A forecaster who cannot be caught out is not being honest. He is being vague.

The determinism pole cannot live that way for a moment. A calculator that a benefits screener consults, that a state agency wires into its workflow, that a tax product ships to customers,

has exactly one job — to be right about what the law says — and its cardinal virtue is that it is boring. Same input, same answer, and the answer matches the statute every time. Housed alongside the forecasting work, that calculator would inherit a reputation it has no business carrying. Every published miss on the scoreboard would raise the wrong question about it: is the engine that just under-forecast a caseload the same engine a caseworker leans on to check whether this family qualifies for SNAP? Technically the two share code and a population. Epistemically they make opposite kinds of claim, and the surest way to keep opposite claims from contaminating each other is to keep them under different roofs, with different promises attached to their names. One roof promises to be interesting and sometimes wrong. The other promises to be dull and always faithful. Wrongness isolation is a property of the design, not a vanity of the org chart.

A commons and its operators

There is a second reason to make the structure explicit, and it predates the two-pole split. An earlier chapter argued that a layer this fundamental — the encoding of the law itself, the calibrated portrait of the population — is not well served by a private gatekeeper; it is closer to a dictionary or a map projection than to a product, and it should be open, forkable, and free to audit for the same reasons those are. That principle survives intact. The 2026 structure gives it a sharper, two-sided form.

On one side stand the nonprofit institutes, stewarding the commons: the rules, the data, the scoreboard, all public. On the other stand commercial operators, building on that commons the things production actually demands — managed runtimes, uptime guarantees, support contracts, applied products for the banks and agencies and firms that need someone to answer the pager at two in the morning. The distinction the whole structure turns on is a single preposition. Operators build *on* the commons. They never own it. The foundation states the commitment plainly: the commercial service tier will always be a separate entity, so that the institution stewarding law-as-code never finds itself competing with the builders who depend on it. A hospital system or a state agency cannot run its eligibility screening on a best-effort open endpoint; it needs a contract, a service level, someone accountable when the pager goes off — and providing that is honest work, worth paying for. The structure's only insistence is that whoever provides it be constitutionally unable to fence off the commons underneath. A steward that also sells is a steward with a thumb on the scale.

One such operator already exists, in formation, organized around graded simulation offered as a service. It is not yet public, and — this is the point — nothing about the commons depends on it. The rules and the data would stay open whether or not any company ever billed for a runtime built on them. That is the test of whether a commons is real rather than rhetorical: it has to survive its operators, outlast any one of them, and owe none of them anything.

Which leaves the question every reorganization eventually gets asked: who owns the whole? Nobody — and not as an evasion but as load-bearing design. A 501(c)(3) cannot be owned at

all; the licenses on the rules, the data, and the scoreboard let anyone build without asking; and no holding company sits above the poles, because the coherence lives in the pattern rather than in any body that could be bought, captured, or pressured into changing course. Astronomy has a word for this. A constellation is official — eighty-eight of them, their boundaries chartered by the International Astronomical Union, one authority deciding where Orion ends. An asterism is the other thing: a shape everyone recognizes — the Big Dipper, the Summer Triangle — made of stars that belong to different constellations or to none, visible without any body ever declaring it. This structure is an asterism, and typography even supplies the mark: , three stars in a triangle, one for each of the three questions. Ask who owns it, and the answer is the design itself. An asterism is a pattern, not an institution.

The arrangement is old enough to have a classical name. When the scattered villages of Attica joined into a single polis — the act tradition credits to Theseus — the Greeks called it *synoikismos*: separate communities becoming one society without dissolving into one another. And the etymology hands over a small gift it would be a shame to leave unclaimed. Tradition connects Theseus's name to the same root as *thesis* — *tithenai*, to place. The founder-myth of federation without absorption shares its root with the institute that places propositions before reality and waits for the grade.

The pattern underneath

Pull back from the org chart and a repeatable pattern comes into focus, one with nothing intrinsically to do with taxes or benefits. Every project here has the same three-part shape. Find a corpus that is complete, public, and — the surprising part — un-computed, sitting in plain sight in a form no one has turned into anything queryable. Point agents at it. Ship the verified, open version of what was there all along. Axiom encodes the world's policy corpus. Populace integrates the world's microdata. Thesis forecasts the government's metrics. Three instances of a single move: the analytic stack of government, rebuilt in the open, one verifiable layer at a time.

The move comes with a gate, and the gate is the whole discipline of this book compressed into a test. A corpus qualifies only if its output can be checked, per unit, against ground truth — a rule against its statute and an oracle, a household against administrative totals, a forecast against the number reality eventually prints. Where that per-unit check exists, agents can work at a scale no institution could ever staff, precisely because each of their millions of outputs can be caught the moment it goes wrong; the check is what converts raw generation into trustworthy production. Where the check does not exist, the same agents at the same scale produce the exact opposite of knowledge — fluent, voluminous, and confidently false. That is the line separating encoding the law from generating plausible law-shaped prose, and it does not move for enthusiasm or for compute. It is the difference between a corpus you can verify and one you can only admire.

For most of history the corpus of public life sat un-computed not because it was hidden but because computing it did not pay. Encoding a single statute faithfully took a trained specialist days, and there are millions of provisions spread across thousands of jurisdictions; forecasting the budget of every county took analysts no one was ever going to fund. So the law stayed on paper, the microdata stayed in survey files, and the metrics stayed unforecast — public in principle, computed almost nowhere. What changed was the unit cost. An agent can encode a country's tax-benefit system in days rather than years, as the five-country week showed; it can hold a corpus of tens of thousands of provisions in view at once; it can draft a forecast for every metric on a docket and never tire. The corpus was always open. The missing piece was a way to compute it that is at once cheap enough to attempt at scale and disciplined enough to trust — and the gate is what supplies the second half.

The gate admits more than tax and benefits, which is where this goes next. Local zoning codes are public, largely un-encoded, and checkable line by line against the text of the ordinance — a national map of what may legally be built where, assembled the way the tax code was. The fiscal futures of tens of thousands of county and municipal governments are forecastable and, in time, gradable against the budgets those governments actually adopt. Bills introduced in a legislature could be scored the day they are dropped, run through the same engine that scores enacted law, so that a proposal arrives with its distributional consequences already attached. None of these is a promise; each is a direction, and each earns its place only if it clears the same gate — per-unit verifiability against something real. But the list runs long, because most of the corpus of public life shares the one property that matters here: it is already written down, and no one has computed it yet.

The third question

Two of the three questions now have homes. *What does the law say* belongs to Axiom; *what will happen* belongs to Thesis. Between them they cover a remarkable amount of what a policy argument is actually made of — the facts of the statute and the facts of the forecast. But they do not cover the part that decides the argument once the numbers are agreed. The oldest question is still open: *what do we want?*

That one has no institute, and it may be the hardest of the three to anchor in ground truth, because its truth is not a statute or a first-print statistic. It is something latent in a population's own preferences — harder to observe, easier to distort, and quick to punish anyone who claims to speak for it. Yet the embryo is already here: a survey-simulation tool that tries to elicit and forecast values the way the other layers elicit rules and metrics, and that the next chapters take up in earnest. Rules, predictions, values — the third primitive is at once the frontier of the whole program and its least finished piece, the place where a book about computing the government has to become a book about what the government is for. There is a version of this stack whose deepest payoff is not any single number but a shared, verifiable substrate everyone can reason against, so that public disagreement can finally be about ends rather than about whose figures are right.

That is the horizon. It is not, however, the next step. Before values, the book owes a debt it has been quietly running up — a question every estimate in every preceding chapter has managed to dodge. Each of those numbers arrived without an honest account of its own uncertainty. This reform costs fifty billion dollars: plus or minus what, and how would we know? The prediction pole exists to answer exactly that — to turn a bare point estimate into a graded interval with a date on it. And so Part IV begins not with what we want, but with how sure we can be of what we already claim to know.

Part IV: The prediction pole

Chapter 13: The uncertainty gap, and the scoreboard

In 2017, the Congressional Budget Office released its analysis of the American Health Care Act, the Republican plan to repeal and replace major provisions of the Affordable Care Act. The headline number: 23 million fewer Americans would have health insurance by 2026.

Not "approximately 23 million." Not "between 18 and 28 million." Just: 23 million.

The number was devastatingly precise, and that precision was an illusion—not because CBO erred, and not because 23 million was a poor central estimate, but because any forecast nearly a decade out carries enormous uncertainty that a single figure cannot express. Economic conditions might change. Behavioral responses might differ from historical patterns. The labor market might evolve in ways no model anticipated. Yet the public debate treated "23 million" as a measurement rather than an estimate—a fact to attack or defend, not a distribution to weigh.

It is the PreCrime problem from *Minority Report*. In the 2002 film, three "precogs" foresee future murders with seeming perfection, and the authorities arrest people before they kill. The predictions look like facts: displayed on screens, exact to the location and the minute, no probability attached. But the story's twist is that the system produces *three* forecasts that usually agree and sometimes diverge, and the dissent—the "minority report"—is suppressed to preserve the illusion of certainty. When a prediction becomes the basis for action, admitting uncertainty becomes politically inconvenient, and so the uncertainty disappears. The suppressed interval on a budget score is the minority report of policy analysis—the dissenting range everyone in the process knows exists and no one is rewarded for showing.

This is the dirty secret of microsimulation. The models produce point estimates without confidence intervals, and that precision is largely manufactured. This chapter is about the gap between the single number and the honest distribution behind it—and about the institution now being built, in public, to close it.

The point-estimate problem

Every microsimulation result in this book so far has been a single number. The reform costs £12 billion. The policy cuts poverty by 15 percent. The marginal tax rate is 47 percent. Each figure emerges from a calculation over millions of simulated households, and each arrives with no indication of how much confidence it deserves.

This is not a bug in a particular model. It is structural. The microsimulation paradigm, from Orcutt onward, was built to answer "what would happen if?"—it produces scenarios, not probability distributions. And four distinct kinds of uncertainty ride along inside every scenario, invisible in the output:

Input-data uncertainty. The model runs on survey data—the Current Population Survey in the US, the Family Resources Survey in the UK—that samples households from the whole population. The CPS draws about 100,000 households from a nation of some 130 million; individual cells (self-employed workers in Montana with investment income, say) may hold only a handful of observations, and when a national estimate sums weighted household impacts, that sampling error propagates. Calibration to administrative totals—the Enhanced CPS cut deviations from known targets by roughly 97 percent (Chapter 8)—pins down the questions already asked, but does nothing for the variance of a question nobody has posed yet.

Parameter uncertainty. Tax brackets and benefit rates are usually known exactly. Behavioral parameters are not. How much do people change their labor supply when marginal rates change? A meta-regression of 1,720 estimates from 61 studies put the mean elasticity of taxable income near 0.30—but individual estimates ran from near zero to above 1.0, driven by differences in method, sample, and country. The range is not academic. A reform that raises the top marginal rate by ten points might add $100 billion in revenue at an elasticity of 0.2, or $60 billion at 0.5. Same reform, same model, opposite headlines, all turning on which study you trust for one number.

Structural uncertainty. The model encodes assumptions about how programs interact, how households respond, how the economy adjusts. Different assumptions, embedded differently in the code, would produce different results.

Future uncertainty. Any projection beyond the current year rests on forecasts of wage growth, inflation, and demographic change. These are uncertain, but they enter the model as fixed numbers.

None of these is exotic. Every number in this book carries all four at once, and the tidy figure on the page is the sum of four kinds of not-knowing reported as if it were one kind of knowing. Each layer compounds the ones before it, and the collapsed result is what gets quoted in the hearing, printed in the story, and defended on the floor, as if it had been measured.

Why it matters

You might treat uncertainty as a methodologist's footnote. It is not; its absence changes how analysis is used. False precision breeds false confidence: a number reported without a range invites its audience to defend it to the decimal, and the more confidently it is quoted, the less anyone asks what it was ever able to know. When CBO says "23 million," legislators treat the figure as a fact to be attacked or defended rather than an estimate to be understood. When PolicyEngine says a reform costs £12 billion, users ask "is it worth £12 billion?"—not "how sure are you?" And when two reforms score $50 billion and $48 billion, the $2 billion gap reads as a real difference even if the uncertainty on each is ±$10 billion, which makes it noise. The same blindness defeats risk assessment—policymakers often care more about the downside than the central case, and a point estimate cannot say whether there is a one-in-ten chance the reform costs twice what was projected—and it undermines model comparison, because when Tax-Calculator and PolicyEngine return different numbers for the same policy, a reader with no interval cannot tell whether one is wrong or both sit comfortably inside the same uncertainty.

The persistence of the single number is not mainly a technical oversight. It is a political equilibrium. Reconciliation rules need one cost figure; you cannot hand a scorekeeper who must certify a total a bill that "probably costs between $40 and $65 billion." Journalists need a headline. Advocates need a talking point—"this reform lifts two million children out of poverty" mobilizes support in a way that "between 1.4 and 2.6 million" does not. Producers of analysis know the estimates are uncertain; consumers treat them as certain; and producers, knowing the consumers will not engage with a distribution, stop reporting one. The illusion becomes self-reinforcing.

The cost of that illusion shows up wherever anyone checks. The Penn Wharton Budget Model, one of the few groups to compare its projections systematically to outcomes, found its estimates generally accurate but with meaningful variance. CBO now publishes retrospective evaluations of its own accuracy: its June 2024 projection underestimated fiscal-year 2025 federal revenues by about 6 percent—$334 billion—largely because it had not anticipated a round of tariff increases, and across 1984–2023 its record shows revenue is harder to forecast than spending and that errors compound with the horizon. None of this is incompetence; CBO's economic forecasts beat the Administration's, the Blue Chip consensus, and the Survey of Professional Forecasters. It is evidence that even the best forecasters, with the best data, face irreducible uncertainty—and that the scores driving legislation are almost never published with an interval and then graded, one by one, against what actually happened. CBO evaluates its aggregate forecast after the fact. No standing, public loop grades each policy score when reality arrives.

Partial solutions

The problem is recognized, and partial answers exist, though none is complete. The most direct is Monte Carlo simulation: run the model many times with inputs drawn from probability distributions, and report the spread. I have used this in EggNest, a retirement-planning tool

that, rather than promising "you will have $1.2 million at 65," runs ten thousand scenarios over distributions of market returns, inflation, and wage growth and reports "a 90 percent chance of between $800,000 and $1.8 million." The obstacle for policy microsimulation is cost. PolicyEngine already computes over millions of households; running that ten thousand times multiplies the work by four orders of magnitude, and calculations that take seconds would take hours.

Bootstrap resampling is the tractable middle ground. Draw many samples from the microdata, reweight each to population totals, run the policy on each, and read the distribution of results. It is embarrassingly parallel—each replicate is independent, so 500 of them add minutes rather than hours on a cluster—and it captures the source that dominates many estimates: the fact that the survey is a sample, not a census. Its limit is honest and sharp. It quantifies input-data uncertainty and nothing else, saying nothing about wrong elasticities or a wrong model. For the everyday question—"how much should I trust this cost estimate?"—input-data uncertainty is usually the leading term, and bootstrapping answers it directly.

Bayesian methods go after the parameters bootstrapping leaves alone, treating a contested elasticity not as a constant but as a distribution—a prior centered near 0.30, updated as evidence accumulates—though buying that requires rebuilding models that currently hard-code such numbers as fixed values. Further out sit probabilistic-programming tools like Squiggle, a language for propagating explicit distributions through a calculation—useful for Fermi-style estimates, but awkward to wrap around ten thousand lines of microsimulation—and plain scenario analysis, which reports a baseline, an optimistic case, and a pessimistic one, conveying sensitivity without attaching any probability to the branches.

A demonstration

To make the leading term concrete, I ran a small paired-subsample experiment in PolicyEngine US on March 31, 2026. Drawing ten 1,000-household subsamples from the then-current Enhanced CPS—the data layer since rebuilt as populace—I ran the same stylized EITC reform on each baseline/reform pair and recorded the change in aggregate EITC and in household net income. The helper script lives in `scripts/policyengine_uncertainty_demo.py`.

(A table appears here in the print and web editions.)

Ordinary survey sampling alone—before touching a single behavioral elasticity, forecast, or modeling assumption—moved the national estimate from about 12 percent below the point estimate to about 19 percent above it. A swing of that size is not a rounding detail; it is large enough to carry a reform across a budget threshold that the single reported figure would have made look safe. This is not a production-grade confidence interval; the subsamples are deliberately small and few, and the exercise captures only input-data uncertainty. But it demonstrates the intuition the single number hides: the reported estimate is not the result. It is the center of a distribution that most policy tools never show.

That experiment was a one-off, run by hand. The point of populace, the calibrated-microdata commons the earlier chapters described, is to make it standing infrastructure. Where the Enhanced CPS treated its imputations and weights as finished inputs, this layer keeps the machinery that produced them—imputation by quantile regression forests, synthesis across integrated sources, calibration to administrative totals, all shipped as certified release bundles. The imputation and synthesis error this chapter cares about can then be quantified and versioned rather than buried in a fixed file: the paired-subsample idea, grown into a property of the data itself. A later analyst does not have to trust that the weights were right; the machinery to re-derive them, and to measure how much they could have differed, ships with the data.

How other fields handle uncertainty

Policy microsimulation's silence on uncertainty is the exception, not the rule. Most quantitative fields have confronted the problem, and one has essentially solved it.

Weather forecasting is the standard to beat, and it sets the bar this chapter holds everything else to. Modern weather models do not produce a single forecast; they run an ensemble of simulations from slightly perturbed starting conditions and report a probability distribution over outcomes. "A 60 percent chance of rain" is a *calibrated* probability: across many such forecasts, it rains on close to 60 percent of the days so labeled. What makes that trustworthy is verification—forecasters track whether events predicted at 60 percent happen 60 percent of the time, and when the numbers drift, the models are recalibrated. Skill has improved steadily and measurably for decades, precisely because every forecast is eventually checked against what occurred. The parallel to microsimulation is exact in structure—both are complex nonlinear systems with many interacting variables—and the difference is only that meteorology spent decades building the verification loop while policy analysis has not begun. That loop—forecast, resolve, score, recalibrate—is a procedure, not a metaphor to admire from a distance, and it can be built for policy the same way it was built for the weather.

The rest of the sciences cluster near that standard. Clinical trials report a drug's effect with a confidence interval, because "15 percent lower mortality (95% CI: 8–22%)" and "15 percent (95% CI: −2–32%)" are different findings, and the interval is the core information, not the garnish. Financial risk management lives on Value at Risk and implied volatility; the 2008 crisis, which exposed how badly those models underweighted tail risk, provoked better uncertainty modeling rather than its abandonment. Climate science runs large model ensembles and reports calibrated language—the IPCC's "likely" means at least 66 percent probability, "very likely" at least 90 percent. Election forecasting moved from punditry to probability: a model that gave one 2016 candidate a 29 percent chance and then watched him win was not refuted, because 29 percent is not zero. Each field learned the same lesson—a probability, stated in advance and then checked, survives outcomes that a point prediction cannot. None of them is simpler than policy analysis; they differ only in having decided, at some point, to close the loop between what they predicted and what occurred.

Uncertainty about structure

Beneath parameter uncertainty lies a harder problem: we do not always know whether the model is right. Parameter uncertainty assumes the structure is correct and only the numbers are in doubt. But microsimulation models assume people respond to incentives as they did in historical data—data drawn from a particular world of labor markets, norms, and policies. When a policy changes that world dramatically, behavior can move in ways the model was never fitted to see.

Universal basic income is the sharp case. Most models estimate labor-supply responses from small marginal-rate changes that leave the structure of work intact. Finland tested whether that extrapolates. In 2017–2018 the government ran a randomized controlled trial: 2,000 unemployed people received €560 a month unconditionally, against a control group of 175,000. Microsimulation had predicted that cutting participation tax rates by 23 points would raise employment; the first-year result showed no statistically significant effect on days worked, and the eventual published analysis found only modest second-year gains concentrated in particular subgroups. The models had the incentive right—participation tax rates did fall by 23 points—and the response wrong, because a permanent unconditional floor is not a marginal tax tweak.

The ACA individual mandate tells the same story from the other direction. When Congress zeroed the penalty in 2017, CBO projected 13 million fewer insured by 2027, from models in which the mandate was a powerful driver of enrollment. The actual effect was far smaller. People had signed up for subsidies, for security, for the momentum of already being enrolled—reasons the models underweighted because historical data could not separate "enrolled because of the mandate" from "enrolled while the mandate happened to exist." No amount of Monte Carlo rescues you here. You can propagate uncertainty through a wrong model and get a precise answer that is precisely wrong. We can quantify the uncertainty we know we have; structural error is the uncertainty we do not know we have—and the only thing that catches it is reality. You do not find a missing mechanism by resampling the data or widening a prior; you find it only when a forecast you committed to in advance misses by more than its interval allowed, and the miss forces the question the model never contained.

The scoreboard

Which is why the honest answer to the uncertainty gap is not a better error bar on a point estimate but a standing commitment to be graded—and that commitment now has an institutional home.

The Thesis Institute is the prediction pole of the recomposed stack, the counterpart to Axiom's determinism, under a two-word division of labor: Axiom states, Thesis predicts. An axiom is accepted truth; a thesis is a proposition put up to be tested. And the thing being tested is a forecast of a government metric, published with an interval, before the number is known.

The mechanism is a public docket. Each entry is a forecast of an official statistic—the first-print unemployment rate for a given month, say, meaning the value as the Bureau of Labor Statistics first publishes it, before later revisions, so that the target is unambiguous. Fixing the target on the first print, rather than a figure that will be quietly revised for years, is what makes the eventual grade well-defined instead of a matter of argument. Each forecast carries an interval, not a point. Each is timestamped and locked. And each will be graded when the official number lands: the entry resolves, the ledger records the fact, marks the forecast against its stated interval, and leaves the score where it was made—a public record of first-print facts and resolutions that anyone can read. This is the accountability loop that CBO, the Tax Policy Center, and EUROMOD do not have: not a retrospective study of aggregate accuracy, but a per-forecast, interval-bearing, publicly scored commitment made in advance.

The same discipline reaches past passive metrics to policy questions, which is where it touches everything else in this book. A counterfactual forecast asks not only "what will the metric be?" but "what will it be under this policy?" One live entry forecasts Medicaid call-center wait times in March 2027 if the work-requirement verification deadline is delayed—a question with a real administrative answer that will eventually exist, attached to a policy choice that has not yet been made. Counterfactuals are the harder case, because the world runs only one branch: if the deadline is delayed, the not-delayed number never appears to be checked. But the branch actually taken still prints a real figure the forecast can be graded against, and a docket of such calls, resolving as the choices are made, is how a model earns or loses standing over the years. This is the bridge from simulation to prediction: the microsimulation estimates that fill the earlier chapters—a reform's cost, its poverty effect, its distributional incidence—become, one by one, forecasts with published intervals that reality will grade. Every estimate turns into a scoreable claim.

This is what the two poles are for. The determinism pole answers *what does the law say?* and is checked against statute and against reference calculators—an answer that is right or wrong today, on a case you can construct now. The prediction pole answers *what will happen?*, and no amount of statutory exactness settles it, because the four uncertainties this chapter cataloged all live in the gap between a deterministic calculation and a claim about the future. That gap cannot be closed by being more careful with the arithmetic. It can only be closed by making the claim in advance and letting the world grade it—which is why prediction needs its own verification loop, and its own institution, rather than borrowing determinism's.

Hold this to the chapter's own weather standard, and be exact about where it stands. Weather earns trust because its probabilities are verified against outcomes and recalibrated when they miss. The docket is built to be verified the same way—intervals stated in advance, graded against first-print reality, recorded in the open. But building the loop is not the same as having run it.

As of this writing, not one of those forecasts has resolved. There is no track record—only a mechanism for acquiring one, in public.

That is not an apology; it is the whole point. A scoreboard that could show a glowing record on the day it launched would be a scoreboard you could not trust, because the record would have been assembled somewhere you could not see. The only credible way to earn trust in a forecast is to publish it before the outcome is known and let the outcomes accumulate in the open, however long that takes and whatever they show. The docket is live, every forecast on it carries an interval, and every one of them will be graded when its official number arrives—and until those numbers arrive, the honest claim is exactly that and nothing more. The discipline this whole book argues for, that no simulation is admissible unless its verification chain terminates in ground truth, applies most severely to the institution built to enforce it. The prediction pole does not ask to be believed. It asks to be checked, later, against numbers it does not yet know.

Toward epistemic humility

Until those grades accumulate, the practical posture is the one this chapter has argued for throughout: treat point estimates as central tendencies, not truths. When PolicyEngine says a reform costs £12 billion, read it as "our best guess is near £12 billion, and the true value could reasonably sit twenty percent to either side, or further in unusual conditions." Read model disagreements as information about uncertainty rather than proof that one model is wrong. Ask which assumptions move the answer most. And be most skeptical of the most novel policies, where structural uncertainty dominates and the historical record is thinnest. Remember, too, that an imprecise estimate is still information: knowing a reform costs roughly $50 billion rules out $5 billion and $500 billion, and even a wide interval narrows the world.

Consider what this looks like on a concrete reform. Suppose PolicyEngine scores making the Child Tax Credit fully refundable—removing the refundable cap and the earnings phase-in that currently hold the poorest families below the full credit, measured against 2026 law. The cost comes back as roughly $23 billion for the year, with child poverty falling by about 2.6 percentage points—from 16.6 to 14.0 percent. An uncertainty-aware version would attach a band to that figure, driven mainly by how many currently non-filing families would claim, and a second band to the child-poverty reduction, driven by the systematic income undercount among low-income households in survey data. The central estimates would not move. But the reader would learn something the single number hides: the poverty effect is robust—even the pessimistic branch shows a meaningful reduction—while the cost has real range. That is more useful than one clean figure, not because it is more precise, but because it is more honest about what is known and what is not.

This reform costs $50 billion.

Or: this reform probably costs between $40 and $65 billion, the distribution is roughly normal with a slight right skew, and there is perhaps a one-in-twenty chance the model is missing something that would change the answer entirely.

Which statement serves the public better?

The scoreboard is how the second statement stops being rhetoric and becomes a record—a forecast, an interval, and a grade that arrives when reality does. But it can only grade what reality eventually prints: an unemployment rate, a caseload, a wait time, a poverty figure the Census will one day publish. The next question has no such number waiting for it. What people believe, prefer, and are willing to trade away has no first-print release date and no administrative office to certify it. To simulate that is to step past the edge of what ground truth can grade—and that is where we turn next.

Chapter 14: Simulating opinion

In 2023, a research paper with a provocative title appeared: "Out of One, Many: Using Language Models to Simulate Human Samples". Researchers at Brigham Young University had done something that would have looked like science fiction a decade earlier. They asked GPT-3 to adopt specific demographic personas—a 65-year-old Black woman from Mississippi, a 28-year-old white man from Oregon—and answer survey questions, then compared the model's answers to real survey data. Conditioned carefully on demographics, the model tracked human voting patterns across the 2012, 2016, and 2020 presidential elections. It captured the partisan divide by education. It reproduced regional variation. In many respects it answered like the population it was standing in for.

That opened a strange question. We had spent this book simulating how policies move through households. Could we simulate what households *think* about those policies?

The emerging field

Argyle and colleagues did not publish into a vacuum. Their paper landed inside a fast-growing literature testing whether language models can stand in for human survey respondents—and the literature was, from the start, an argument.

The optimists had real results. Mei and colleagues found that GPT-4 reproduced human behavior across six canonical psychology experiments—the kind of well-studied pattern a model has seen described thousands of times. Argyle's own voting result worked for a related reason: decades of polling have documented so thoroughly how demographics track the vote that the model had absorbed the correlation and could play it back. For questions where the training data carries strong demographic-to-opinion correlations—voting, partisan identity, familiar consumer preferences—the models generate distributions that look plausibly human.

The skeptics had equally real results, and it is worth stating them before the sell. Santurkar and colleagues asked a pointed question—whose opinions do these models actually reflect?—and found that a base model's default view is representative of no real population, skewing instead toward the liberal, educated, Western profile of its training data, a skew that fine-tuning with human feedback shifted but did not erase. Prompting could move the default, but sometimes

only into caricature: pushed to speak for a group, a model may embody a cartoon of it rather than the real range within. Bisbee and colleagues compared ChatGPT's simulated responses against real survey data and found the silicon version too tidy: less variation than real humans show, and regression coefficients that differed, sometimes sharply, from the human estimates. Park and colleagues had shown the far edge of the idea with "generative agents"—AI characters who lived in a simulated town, formed plans, and produced emergent social behavior—which is captivating and also a caution: fluency is not fidelity.

The honest reading is nuanced. Silicon sampling does best for well-studied populations answering well-studied questions, and worst for underrepresented groups facing novel ones. Marketing researchers have converged on a modest role for it—pretesting instruments and running pilots, not producing final answers. The disagreement in the literature is less a contradiction than a map: the optimists tend to test familiar behaviors on well-covered groups, the skeptics probe the tails and the underrepresented, and both are telling the truth about the region they sampled. It is a tool with an operating range.

The cost of asking

Why would anyone want this? Because asking humans is slow and expensive. Every year, companies and governments spend on the order of $140 billion on the insights industry—surveys, panels, focus groups, the whole machinery of finding out what people think. A well-designed survey of a thousand respondents can cost tens of thousands of dollars and take weeks to field. Reword the question and you pay again. Break the results across fifty subgroups and the cost multiplies.

Large institutions can absorb that. Startups, nonprofits, local governments, and lone researchers often cannot, so many questions worth answering simply never get asked. When it costs $50,000 to learn what people think, only the well-funded get to ask—which concentrates information, and therefore power, among those who can afford it. Cheap, approximate answers would change who gets to pose the question at all: a first-time candidate testing a message, a neighborhood association gauging support for a development, a graduate student chasing a hunch without a grant. That is the same democratizing move this book keeps returning to—the one that turned budget scoring from an institutional privilege into something a citizen can run—now pointed at the survey.

In 2026, a company called Simile began packaging a broad version of this pitch, describing itself as "the simulation company" and arguing that simulation, not prediction alone, is AI's next frontier. Read one way, that is evidence synthetic social research is escaping the lab. Read another, it raises the bar. For public-interest use, a plausible demo is not enough. What matters is whether the numbers are calibrated, whether the uncertainty is stated, and whether anyone has checked them against ground truth. That is the test I decided to put my own version of this idea through.

Cells, not personas

HiveSight is the tool I built to explore synthetic opinion, and its short history is a compressed version of this book's argument. Start with the naive version and its failure. Ask a frontier model "Do you support a $15 minimum wage?" and it returns a careful, hedged essay about tradeoffs—the average of everything it has read, which is no individual's answer and no population's distribution. Turning up the temperature does not fix this; it adds random noise, and random noise is not structured human diversity. A wealthy retiree in Florida and a young barista in Seattle do not differ randomly—they differ systematically, along income, age, geography, and a dozen correlated axes. The obvious repair is to condition the model on those axes. That is what the first prototype did.

The first prototype did the intuitive thing—the thing the literature was debating. It built a demographically detailed persona from microdata and asked the model to answer as that person:

> *You are a 45-year-old woman in rural Ohio. You work as a nurse, earn $62,000, attend church weekly, and voted Republican in 2020. Answer as this person would: do you support raising the federal minimum wage to $15?*

Ask a thousand such personas, each drawn from real demographic distributions, and you get something shaped like a survey. It demos beautifully. Then I benchmarked it against known 2024 survey values, and it fell apart: persona roleplay landed roughly 25 points of mean absolute error, on both toplines and subgroups. It demoed like an oracle and scored like a guess.

The intuitive mechanism failed the benchmark; the boring one won.

What shipped in the July 2026 rebuild is unglamorous. Instead of role-played individuals, HiveSight builds post-stratification cells—roughly 150 demographic cells per geography, drawn from the same calibrated population data, populace, that the rest of the stack shares. For each cell it asks the model not to *be* a person but to estimate a *distribution* of answers, and it aggregates those cell-level distributions with calibrated weights. The idea that survived is microdata grounding. The mechanism that died is the persona.

The distinction is subtler than it sounds, and it is the whole point. A persona asks the model to collapse a group into a single imagined individual, and a language model asked to be one person does what it was trained to do—produce the most probable, most agreeable, most legible version of that person. It reaches for the stereotype or hedges to the center. But a survey never measures one person; it measures a group's *spread*. Post-stratification asks for exactly that—not "what would a rural nurse say" but "across people like this, what is the distribution of answers"—cell by narrow cell, with the model reporting a spread instead of impersonating a point. The microdata was doing the useful work all along, defining who is in the room and in what proportion. The role-play was the part that added the error.

None of this machinery is an AI invention. Post-stratification is a workhorse of modern survey statistics: pollsters have long chopped a population into demographic cells, estimated opinion

within each, and reweighted the cells to match the known composition of the electorate—it is how a badly skewed online panel can still call an election. HiveSight keeps that apparatus intact and swaps a language model in to fill each cell. The move that worked, in other words, was the one that changed the least about survey methodology and the most about who has to be recruited to answer.

The results are registered, and reported with their ties intact. The benchmark froze a 63-item anchor bank—52 attitudes from the 2024 General Social Survey and 11 from the 2024 Survey of Household Economics and Decisionmaking—under a pre-analysis plan committed before any answer was elicited, and against a model whose training cutoff predates every 2024 target, so it could not have memorized them. On that bank, the cell estimator scored about 9.2 points of mean absolute error on toplines and 9.8 on subgroups. A simpler contender—just asking the model for the whole population's distribution directly—scored about 8.6 and 9.8. On toplines that is a tie, and the paper says so: the two are statistically equivalent within a ± 1.5-point margin. Direct estimation is not worse.

Where the cells earn their keep is structure. They track the *shape* of opinion across subgroups better—a rank correlation of 0.62 against 0.48—they are far steadier run to run, with roughly nine times lower variance, and they hold up when the questions are reordered. The failures are in the paper too. At the state level—four items across forty-nine states, scored against a different survey—the cells beat a naive baseline, but a national-constant guess, which simply assumes every state looks like the country as a whole, stayed competitive. That result is reported, not buried. It is an honest admission that the model's geographic signal, on those four questions, is thin: cheap national estimates are one thing, trustworthy fifty-state breakdowns another, and the paper does not pretend otherwise.

None of that honesty is automatic. The reason the tie and the competitive baseline appear in the paper at all is that the suites, the arms, and the scoring rules were registered before the results existed—a pre-analysis plan is a promise you make to your future self that you will not quietly relabel a loss as a win once you have seen the numbers. A tool built to be sold would have led with the subgroup victory and let the topline tie go unmentioned. A tool built to be trusted publishes the whole scorecard and lets the reader see where it merely ties.

That is the book's method turned on its own speculation: benchmark against ground truth before believing the demo.

It is worth being precise about what this does and does not establish. HiveSight ships, and it publishes its benchmark; that alone puts it ahead of most synthetic-research tools. But shipping is not validation. Shipping is not accuracy, and the honest tie on toplines is printed on the tin. What the tool earns is a defined operating range with a measured number attached to it—which is exactly the currency the rest of this book has traded in. A demo persuades by looking right; an instrument persuades by telling you how often it is wrong, and on which questions. HiveSight is trying to be the second thing, and the registered benchmark is its receipt.

Inside that range, it is genuinely useful. Suppose a city council is weighing congestion pricing—a $15 charge to drive downtown at peak hours. A real survey might cost $30,000 to $50,000 and take three weeks. HiveSight can take the city's demographic composition, elicit a distribution over each cell, and return an estimate in minutes: perhaps 62 percent opposition overall, but with a sharp gradient—heaviest among middle-income commuters who cannot easily switch to transit yet feel the fee acutely, lighter among low-income residents who do not drive and high-income residents who shrug it off. Would those match a real poll? Approximately, for the broad pattern; probably not for the exact percentages. The value is not replacing the survey. It is deciding whether the survey is worth fielding. If the estimate shows 90 percent opposition across every group, the council can shelve the idea without spending $40,000 to confirm it. If it shows a close, structured split, the estimate has already earned its keep by pointing at where the real fight is—which subgroups to engage, which objections to answer—and that is precisely when the real survey earns its cost.

Systematic biases

The limits that remain are not random noise. They are structural, which is the good news: structure is partly predictable and partly correctable.

WEIRD bias. Models are trained mostly on text from Western, educated, industrialized, rich, democratic populations. They simulate Americans better than Nigerians, college graduates better than high-school dropouts. The gaps in the training data become systematic errors for exactly the groups that data underrepresents.

Homogeneity bias. Models drift toward central tendencies and sand off the extremes. Real opinion has long tails—outliers, contradictions, people who defy their own demographics. Silicon samples come out too clean.

Temporal anchoring. A model trained to a cutoff can render the world up to that date and not past it. It cannot answer to an event it never saw—a pandemic, a crash, a viral moment that moves sentiment overnight. This is also why the contamination control matters: a model that has already seen 2024 is not forecasting 2024.

Social desirability bias. Safety-tuned models are trained to be agreeable, which makes them reluctant to voice the extreme or unpopular opinions that some real, matching humans hold. What comes back is a sanitized public.

Stereotyping. Handed a demographic label, a model sometimes returns the caricature instead of the range. Prompted as a rural evangelical, it may produce the cartoon rather than the spread of views such people actually hold.

Because these biases are structural, they can be handled the way survey research already handles its own: with weights learned against known answers. Take historical data where we know both the demographics and the real responses, measure where the model runs hot or cold, and correct future estimates the same way a pollster corrects for an oversampled group. The

right output of a calibrated system is then not "58 percent support this" but "58 percent, give or take eight, given this question type, this population, and our measured error on questions like it." The hard part is that a correction learned on last year's questions has to hold on next year's, or it is just overfitting with extra steps—which is why the honest version of this work grades itself on held-out items it never got to tune against. That is the difference between a demo and an instrument. The field is not there yet; the discipline is knowing that, and saying so.

Adversarial considerations

Silicon sampling does not only have limitations. It has misuses. If you can cheaply simulate how a demographic reacts to a message, you can craft the message that moves it, and a campaign can A/B test divisive framing without ever exposing a real person to it. Because silicon results wear the costume of real ones—tables, percentages, crosstabs—a bad actor can pass a simulation off as a poll and manufacture the appearance of public support the methodology never earned. If institutions start acting on synthetic opinion, and those actions reshape the information environment that trains the next model, the estimates can begin to reflect opinions the tool helped create rather than opinions people actually hold. And because models default to the majority within a group, the dissenters—the conservative professor, the working-class environmentalist, the minority view inside a minority—get quietly rounded away, undercounted twice.

None of this is an argument against building the tool; traditional polling is manipulated too. The difference is that a real poll at least sampled somebody, while a simulation can conjure a consensus from no one at all and dress it in the same chart. It is an argument for building it in the open, with the model, the cells, the calibration, and the error bars all disclosed, and with results labeled as estimates rather than dressed up as a survey. Approximate knowledge, widely distributed and clearly labeled, can serve the public better than precise knowledge held by whoever can afford a poll—but only if "clearly labeled" is taken as seriously as the estimate itself.

An operating range, not a substitute

Silicon sampling is not a general replacement for asking people. It is a tool with an operating range: strongest for well-studied populations on well-studied questions, weakest exactly where representation is thin and the cost of being wrong is high. The honest verdict is the one the benchmark forces—useful inside a measured band, and candid about the edges of it.

It is worth naming why this is harder than anything else in the book. A tax flows through rules; a benefit has eligibility criteria; those can be encoded exactly and checked against an oracle to the dollar. A first-print unemployment number will arrive and grade a forecast. An opinion has no such clean settlement—it emerges from a whole life, and when a model says "I

strongly oppose this," no one is home who opposes anything. That is why opinion simulation is the embryo of the book's third primitive—values elicitation, the layer that sits beside rules and prediction—and why it is still only an embryo: the ground truth is real but partial, so the discipline has to carry more of the weight.

There are two ways to read what the tool produces, and both are true at once. On one reading it predicts: given how people like this have answered questions like this, here is the likely distribution—the same extrapolation from sample to population that survey research has always performed, with a model standing in for the sampling weights. On the other reading it fabricates a plausible artifact that resembles opinion with no opinion behind it. For deciding whether a survey is worth fielding, the first reading is enough. For anything that touches democratic legitimacy—treating a number as the authentic voice of a public—the second reading is the warning, and the labeling has to hold that line.

There is a version of this tool that plugs straight into the rest of the stack: simulate a policy change against the population, then simulate the public's reaction to it, both drawn from the same profiles, both uncertain, both better than no signal at all. That is the direction. But a distribution of what people *think* is only an input. The harder question is what a society *does* with those opinions—how it counts them, weighs them, and turns them into a decision. That is where we go next: from what people think to how democracies aggregate it.

Part V: The horizon

Chapter 15: Simulating democracy

Note to readers: Democrasim, the model at the center of this chapter, is a toy — a thought experiment in code, built to make one relationship concrete enough to reason about, not to contribute to political science. It is not a validated production system like PolicyEngine, nor a benchmarked estimator like the opinion work of Chapter 14. It makes simplifying assumptions that political scientists would rightly criticize. Read it for intuition about information and democracy, not as established fact.

Why do democratic outcomes so often diverge from voter welfare? The question has occupied political scientists for decades, and their answers should humble anyone who expects better information tools to fix democracy.

Christopher Achen and Larry Bartels, in *Democracy for Realists* (2016), argued that the textbook story — informed citizens evaluate policies, pick the candidate whose platform fits their preferences, and elections translate public will into government — is largely fiction. In their telling, voters mostly choose parties by social identity and group loyalty, not by policy evaluation. They punish incumbents for droughts and other things no government controls. They reward growth that began before the incumbent took office. The "folk theory" of the rational, policy-weighing voter bears little resemblance to how democracy actually runs.

Bryan Caplan went further in *The Myth of the Rational Voter* (2007): voters aren't merely ignorant, they are *systematically biased*. Drawing on a survey that set the public's economic views beside professional economists', he identified four persistent distortions — anti-market, anti-foreign, make-work, and pessimistic bias. These are not random errors that cancel out in aggregation. They are directional, and they push democratic outcomes away from what most economists would call welfare-improving policy.

Against that backdrop, the question I want to explore is deliberately modest. Not whether information can fix democracy — it can't — but whether voter accuracy about policy impacts matters at the margin, and whether tools can improve it. I take the skeptics seriously enough to start from their side of the ledger: if identity and loyalty carry most of the vote, then anything an information tool can touch is, at best, a minority share of the decision. The argument of this chapter lives entirely inside that minority share, and it is careful never to claim more.

The perception problem

Suppose a voter — call her Sarah. She teaches school, she has two kids, and she is choosing between two candidates with different platforms. One would expand the Child Tax Credit; the other would repeal the state income tax. Which leaves her family better off?

Without doing the arithmetic, Sarah has to guess. Maybe she has a vague sense that she pays state income tax and that it stings. Maybe she remembers the expanded credit during the pandemic and that it helped. Maybe she has heard talking heads argue both, each framing built to persuade rather than inform.

Her vote will be some blend of what she actually values, what she believes each policy would do, what her party and community support, and plain noise — a candidate's charisma, the morning's headlines, the weather. Even if her values are clear — say she cares most about her family's bottom line — her vote may not track them, because she cannot see the true impacts. She votes on a noisy signal.

Now multiply Sarah by a national electorate: each person deciding on imperfect perceptions, some biased one way and some the other, most well-informed about one issue and clueless about the next. The aggregate only roughly, noisily tracks what voters would choose if they could see clearly. Achen and Bartels would say this is too generous — that many voters aren't evaluating policy at all. Caplan would add that the ones who try are systematically wrong. I

think both are partly right, and that the interesting question is narrower than either: to the extent *any* part of the vote is policy-driven, does the quality of that signal matter?

A toy model

To reason about that, I built a small simulation called Democrasim. The emphasis belongs on *small*. Each simulated voter carries three things: a set of weighted preferences across policy dimensions — how much they care about the economy versus the environment versus social issues, which are their true values; an accuracy level, meaning how close their perception of a policy's effects sits to the truth; and a bias, a systematic lean beyond random noise, such as consistently overstating a tax burden or understating an environmental cost. Perceived impact is true impact plus noise plus bias, where the noise shrinks as accuracy rises. Voters compare candidates on perceived impacts weighted by their preferences, and the majority wins.

The question the model asks is simple: as voter accuracy varies, what happens to the welfare quality of who wins?

What the model shows

The results are intuitive; putting numbers on them just sharpens the stakes. When accuracy is high, electoral outcomes track welfare — the candidate whose policies would actually improve voters' lives tends to win. When accuracy is low, the link frays; winners might have good policies or bad, because the signal is too corrupted to sort them. The model also shows something like a threshold. Below a certain accuracy, elections become essentially random with respect to welfare, bearing no reliable relationship to what would help voters; above it, the relationship strengthens quickly.

A hypothetical makes the mechanism visible. The numbers here are round and invented, chosen to illustrate rather than to report a calculation. Suppose Candidate A expands the Child Tax Credit and pays for it with a small rise in the top marginal rate, while Candidate B repeals the state income tax and pays for it by cutting food assistance. For a middle-income family with children and no food assistance, the credit expansion is worth more than the tax repeal, and the offsetting rate increase never reaches them — so they are better off under A, even though "repeal the income tax" is the phrase that sounds like the bigger deal. A voter perceiving accurately picks A; a voter perceiving through noise may well pick B, against her own interest. Scale that across an electorate and the pattern compounds: lower-income families do far better under A, upper-income families modestly better under B, and whether the election reflects that real distribution of gains depends on how much of the vote is signal rather than framing.

Bias bends outcomes a different way. If voters systematically overvalue tax cuts relative to equivalent benefit increases, elections will systematically favor tax-cutting candidates regardless of welfare — Caplan's point, reproduced in miniature.

I want to be careful not to overclaim. These results come from a model with extreme simplifications; real voters have identities, loyalties, and psychology that Democrasim ignores entirely, and it assumes voters are trying to maximize welfare through policy evaluation — the very folk theory Achen and Bartels dismantle. What it captures is one real dynamic: to the degree that *some* of the vote responds to perceived impacts, the quality of those perceptions matters. Even if only a fifth of the vote is policy-driven and the rest is identity, cleaning up that fifth still moves outcomes.

That one dynamic is also why this toy earns its place in the book. Read Democrasim backwards and it is the motivation model for everything the earlier chapters built: if outcomes track welfare only where perceptions of policy impacts are accurate, then the machinery that makes accurate perception cheap — rules encoded exactly, populations calibrated honestly, forecasts graded in public — is not a technocratic garnish on self-government but an input to whether it works at all. The stack exists to raise the accuracy term in this model. Democrasim is the sketch of why that term matters.

What the evidence shows

Democrasim is a toy, but the question underneath it — does voter information quality change outcomes? — has been studied for real, and the evidence is mixed in a way that is itself informative.

Information sometimes moves votes, when it reaches people. In Brazil, Claudio Ferraz and Frederico Finan found that municipalities where federal corruption audits were released before an election punished corrupt incumbents at the polls — but the effect concentrated where local radio carried the results. Information that existed but did not travel changed nothing. Nonpartisan voter guides in the United States sit at the weaker end of this: they tend to raise political knowledge without much shifting vote choice, because in polarized settings most voters have already committed to a party. Arthur Lupia and Mathew McCubbins showed that voters lean on trusted-source shortcuts rather than full information anyway — which cuts both ways, since a shortcut can approximate an informed choice or can simply entrench identity over evaluation.

The noise is not always innocent, either. One of the more robust findings in political communication is that accurate information and strategic misinformation do not compete on equal terms: a well-crafted misleading claim travels farther and sticks longer than a dry correction. Democrasim, by treating noise as random, actually understates the problem — in the real world some of that noise is engineered by people with an interest in distortion.

When information finally arrives as lived experience, it does register, though often at ruinous cost. In 2012, Kansas enacted large tax cuts its backers billed as a "real live experiment" in supply-side growth; independent analyses, of the kind PolicyEngine now produces, projected large shortfalls instead. The shortfalls came — years of budget crises and school-funding cuts, ending in a bipartisan reversal of the cuts in 2017. Voters did not need a simulation

to learn the lesson; they paid for it in billions of dollars and lost services, over years, and better information infrastructure might at least have shortened that loop. The Affordable Care Act shows the subtler failure. As Chapter 3 described, the CBO got total coverage gains roughly right but badly missed the composition, overestimating exchange enrollment and underestimating Medicaid. Voters who opposed the law over predicted exchange disruption, and voters who backed it for Medicaid, both experienced something other than what they had been told, and the gap between perception and reality persisted for years. A 2017 survey found that about 35% of Americans did not know the ACA and "Obamacare" were the same law — a failure of basic information transmission that no simulation can fix, but a fair measure of how far real voter knowledge sits from the idealized version.

Some limits no information tool touches at all. Kenneth Arrow's impossibility theorem (1951) proved that no ranked voting system can satisfy a short list of reasonable fairness conditions at once; Gibbard and Satterthwaite showed that any non-dictatorial system can be gamed by strategic voting. Better inputs do not repeal those constraints. The honest summary is that better voter information is necessary but nowhere near sufficient: it is one lever among identity, institutions, and strategy, and the only one that infrastructure can move directly rather than through the slow work of changing culture.

The perception problem, against ground truth

Democrasim can only toy with the perception problem because it has no ground truth to check itself against — its voters misperceive a policy's effects, but the "true" effects are just more numbers the model invented. Chapter 13 described the version of this problem that does terminate in ground truth: forecasting a government metric under a policy counterfactual — what the unemployment rate will actually print next quarter, or how long a Medicaid call-center wait will run in a given month if a work-requirement deadline slips — and grading the forecast when the official number lands. That is the accuracy question Democrasim only gestures at, lifted out of the toy and onto a public docket where reality settles it. The difference is the whole difference between a thought experiment and a measurement: one grades itself against numbers it made up, the other waits to be graded by numbers it did not. None of those forecasts has resolved yet; the point is not that the empirical version has been scored, but that the perception problem has an empirical form at all.

PolicyEngine as one input

This is where a tool like PolicyEngine becomes democratically relevant — not as a cure for democratic dysfunction, but as one input among many. Instead of "the average family pays X," it computes what a specific household, with its actual circumstances, would pay or receive under a given policy. It is free to anyone with a browser. And its transparency is no longer just a claim about open-source code: the rules it applies are open encodings, checked against

reference calculators — the verification machinery of Chapter 10 — which is a stronger warrant than "inspect the source if you like" ever offered.

In Democrasim's vocabulary, that makes PolicyEngine an accuracy multiplier for the policy-evaluation slice of a vote. It moves a voter from "I sense this would hurt me" to "this changes my household's income by roughly this much." For Sarah, it turns the guess between the credit and the tax repeal into a number she can see. It does nothing for the identity-driven part of her vote, does not override party loyalty, and does not resolve Arrow's problem in the voting rule itself. It removes one source of noise from one channel. That is a smaller claim than "informed voters fix democracy," and it is the right size.

Futarchy, and a separation

There is a more radical proposal for wiring information into governance: Robin Hanson's *futarchy*. Its slogan is "vote on values, but bet on beliefs." Democratic processes decide *what we care about* — a welfare metric, a child-poverty rate — and prediction markets decide *which policies achieve it*. A legislature proposes a bill; markets price the welfare metric conditional on passage versus failure; if the market says welfare is higher with the bill, it becomes law.

No democracy has adopted it, for reasons that are easy to see: markets can be manipulated by deep pockets, most policy questions never draw enough trading to price reliably, "welfare" is contested to define, and legitimacy depends partly on citizens feeling heard, which a price does not supply. But the thought experiment isolates the distinction that matters here — between values, the outcomes we want, and beliefs, the policies that would produce them. PolicyEngine works the beliefs side; it never tells anyone what to want. Neither does Democrasim.

Values are for humans; facts are for tools.

Objections

The accuracy-and-welfare framing oversimplifies real democratic life, and four objections deserve a straight answer.

Maybe the preferences are the problem, not the perceptions. Perhaps voters perceive fine and simply want policies that harm others. True — but orthogonal. If voters want harmful things and perceive accurately, they get harmful things; that is a different failure from wanting good things and misperceiving. Better perception at least delivers people what they actually want, which is the precondition for holding them responsible for it.

Information won't reach the disengaged. People who do not vote will not open a policy calculator either, and the ones who would are already relatively informed. Partly true. But new modalities — an AI assistant that volunteers a policy's household impact, a calculation embedded where people already are — can reach past the self-selected few, and moving the moderately informed to well-informed still improves the signal.

Calculated self-interest isn't good citizenship. Democracy might want voters weighing the common good, not just their own household. Fair — and PolicyEngine computes societal impacts too: poverty rates, inequality, total cost. The point is not selfishness; it is replacing perception with calculation, whatever a voter chooses to weigh.

Arrow's theorem means none of this matters. No system aggregates preferences perfectly, so why polish the inputs? Because Arrow constrains the *mechanism*, not the quality of what feeds it. An imperfect aggregator still yields better outcomes from better-informed inputs. Arrow proved no perfect system exists; he did not prove every system is equally good regardless of what goes in.

The democratic case for open infrastructure

The argument, then, is bounded on every side, and stronger for it. Democratic outcomes track voter welfare only to the degree voters perceive policy impacts accurately — and that is one factor beside identity, institutions, and strategy. Most voters perceive those impacts poorly, through noise and bias and thin information. Tools exist that can sharpen the policy-evaluation slice of the decision. So investing in accessible, trustworthy policy analysis has civic value beyond any one person's tax optimization — not as a fix for democracy, but as one contributor to informed self-government.

That reframes what PolicyEngine is for. Not a calculator for people who want to trim their taxes, but one component, among many, of democratic infrastructure.

Chapter 16: Simulating values

Every earlier chapter in this book obeyed one discipline: a simulation earns trust only where its verification chain ends in ground truth — a forecast graded against the official number, an encoding checked against a reference calculator, a statute quoted to the letter. This chapter reaches past that line. The verification chain here does not yet terminate in ground truth, and it is worth saying so at the outset: read what follows as horizon, not foundation.

We have simulated households, policies, voters, and opinions. The deepest question in the set is harder than any of them, and it is not "what do people want today?" It is what people would want after reflection — and whether that is something a model could ever forecast.

Start with the humbling part, because it is the honest place to start. In 2024 I ran the first systematic test I know of on whether a language model can forecast value change, and the headline result was a miss. The General Social Survey's long-running measure of whether

same-sex relations are "not wrong at all" had climbed for decades to nearly 63% by 2022; in 2024 it fell to just under 56%. Every method I tried — the language model, a linear trend, a no-change baseline — expected the climb to continue. Every method missed it.

That miss is the center of this chapter, so hold onto it. First, though, the idea it was testing.

Could you have forecast the change?

Consider same-sex marriage. Gallup began asking about it in 1996, when 27% of Americans were in favor; by the early 2020s about seven in ten were. The change was not random — it followed exposure, generational replacement, and the slow crystallization of arguments — and in retrospect it looks almost legible. Could you have called that trajectory in 1996? More to the point: could you predict, now, which of today's contested values will look obvious in a generation and which will reverse?

This is not a search for moral truth. It is a forecasting question — the same shape as forecasting inflation or rainfall, done not perfectly but better than chance. And it is the same apparatus the book has used elsewhere, pointed somewhere new: a survey, a time series, a model, a calibrated interval, and a grade when the next reading lands. The only thing that has changed is the quantity on the vertical axis, which is now an attitude rather than a dollar or an unemployment rate.

It matters because decisions get made against contested values all the time, and increasingly some of those decisions are made by automated systems. "Do what humans want" fractures the moment you ask which humans, and whether you mean what they say now or what they would endorse on reflection. Stuart Russell built his account of controllable AI on exactly that uncertainty — machines should be unsure what humans want and learn it from behavior — but behavior reveals present preferences, which are precisely the ones most likely to move. If value change has structure, some of that structure might be forecastable. That is the whole of the claim, and it is smaller than it first sounds.

The experiment

The General Social Survey has asked Americans the same attitude questions since 1972, with wording stable enough to form real time series. I took 17 of them — spanning sexual morality, drug policy, gender roles, race, religion, confidence in science, guns, capital punishment, and free speech — and gave a frontier model each variable's history through 2021, asking not for a single guess but for a spread: the tenth, twenty-fifth, fiftieth, seventy-fifth, and ninetieth percentiles. The point of the spread was to force the model to express uncertainty rather than default to false confidence.

Two deliberately simple baselines set the bar. A linear extrapolation fits a trend line to the history and extends it. A no-change baseline predicts that the next reading equals the historical

average. Any method worth having should beat both, and beating a straight line is the harder and more meaningful test, because a straight line already captures the fact that most of these attitudes have been moving in one direction for decades.

The model's forecasts for 2024 landed at a mean absolute error of about 4.8 percentage points, against about 7.2 for the linear trend — roughly 1.5 times better — and about 10.6 for the no-change baseline, roughly 2.2 times better. Naming the baseline matters here, because it is easy to quote the flattering number and leave off the comparison: "2.2 times better" is true only against the weakest baseline, the one that assumes nothing changes, and the honest headline is that the model beat a straight line by about half again. The advantage was largest where trends bent — accelerating acceptance on some questions, decelerating change on others — exactly the shape where a straight line goes wrong and where, presumably, whatever the model had absorbed about how attitudes actually move earned its keep. The intervals needed work too. Raw, they came out too narrow — the familiar overconfidence of language models, which state a distribution far tighter than their errors justify — and had to be widened with a standard calibration step borrowed from weather forecasting before the 80% intervals actually covered 80% of the outcomes.

Read carefully, that is a modest, real result: on three-year forecasts of American social attitudes, a language model beat naive baselines, by more against the weakest and less against the sterner one. And then it, and everything else, missed the reversal.

Same-sex acceptance was the variable with the cleanest story. It had climbed for half a century, through the same generational replacement and exposure effects that make these trends look like laws; it had every hallmark of a one-way trend, and it was exactly the kind of series a forecaster feels safe about. The 2024 reading turned down, nearly seven points off its 2022 level. Seven points is not a large move by the standards of survey noise, and it may yet prove to be noise, or a methodological artifact of how the survey was run, or the leading edge of a genuine backlash — that question is still open. But its size is not the point. The point is direction: every method predicted up, and the number went down, on the one series a forecaster would have staked the most on. A method beating its baselines by four points on average was humbled by a seven-point turn it did not see coming, in the place it was most confident. Value change carries genuine surprises that no model, statistical or neural, reliably anticipates, and the honest lesson of the experiment is carried less by the errors it beat than by the one it did not.

The miss sent me back to rebuild the experiment properly, and the rebuild cut deeper than the miss. In July 2026 I pre-registered a replication before making a single model call: twenty GSS items this time, survey-weighted, each model shown the series only through 2022 and asked for 2024, with every training cutoff checked against the data's release date so that no arm could be "forecasting" a number it had already read. The baselines got sterner too. The one that matters is persistence — the forecast that 2024 simply equals 2022.

Persistence beat everything. The language models' errors ran 3.9 to 4.1 points against persistence's 3.15 (n = 20); a frontier reasoning model did no better than the cheap ones, and

raising a model's reasoning effort made its accuracy slightly worse, not better. The models' 90% intervals covered the truth roughly half the time; properly computed classical intervals covered at their stated rate. And the 2024 wave turned out to be stranger than one famous reversal: eight of the twenty items broke their pre-2022 trend, and four of them — approval of the school-prayer ban, same-sex relations, rejection of traditional gender roles, marijuana legalization — posted the largest single-wave declines in their items' recorded histories. Every arm, model and baseline alike, missed the same-sex reversal again, with point forecasts between 60 and 66 against an actual just under 56; the only interval that contained the truth did so by being twenty-one points wide.

How does that square with the earlier run, where the model beat its baselines handily? Baselines are the answer, and they are most of the result. The earlier comparison set the bar at a trend line and at the historical average — and against an average over decades of lower readings, any method that merely notices the recent level looks brilliant. Persistence is the bar that matters for slow-moving series, and nothing cleared it.

The rebuilt experiment also included the control the earlier ones lacked, and it explains where much of the apparent forecasting skill in this literature comes from. Run a model on a series it can name, and it is not only extrapolating; it is remembering. Shown the same-sex series through 2010 *with its identity visible*, one model "forecast" the present at 78 — a level the series has never reached in fifty years; shown the same numbers with the name stripped, the same model said 47.5. A thirty-point swing from the label alone. Identified backtests inside a model's training window are recall tests wearing a forecast's clothes, and the diagnosis reaches past this chapter: a growing literature evaluates language models by "predicting" surveys, elections, and experiments the models have already read about. Two protocols fix it — strip the identities, or forecast strictly forward, where no training corpus can help — and the rebuilt experiment adopts both, with forecasts for the 2026 and 2028 GSS waves registered before the data exist.

There is even an instrument for reaching further back than any living forecaster. Language models trained exclusively on text from before 1931 now exist, and such a model has read no poll it could parrot: Gallup's first national surveys came in 1935, the GSS began in 1972, and the entire quantitative record of public opinion lies beyond its horizon. Early evaluations locate their weakness at the elicitation layer rather than the knowledge layer — they can complete what they cannot converse about — which is an engineering problem rather than a wall, and stronger checkpoints at the same cutoff are expected within months. The eventual prize is ninety years of graded actuals: every reversal, plateau, and backlash in the polling record, out-of-sample by construction.

The same apparatus can be pointed much further out — to 2050, or to 2100 — and it will dutifully return a tidy median and interval for each. I once tabulated exactly those long-horizon numbers; after 2024, I have cut the table, because the only honest way to read such figures is as an illustration of what the method emits when asked, never as a finding about the future. The further past the data you push, the more the interval, and not the median, is the result worth reporting.

Heterogeneity is the target, not the noise

A tempting mistake is to imagine forecasting converges on a single answer — the value system humanity is "heading toward." It does not, and should not. Even granting perfect foresight, people would still hold different things dear; the object worth forecasting is a distribution across a population, not a point on a line.

That reframing has teeth for anything that would consume such a forecast, because it changes what the forecast is for. A point estimate invites a system to optimize toward it. A distribution refuses that: it says that at the end of any plausible reflection there remain people who weigh liberty over welfare and people who weigh welfare over liberty, and that both are part of the answer, not error bars around it. A system taking the distribution seriously would favor actions that score acceptably across several value systems rather than maximally within one; it would grow cautious in precisely the regions where the value systems disagree most; and it would treat a move that catastrophically violates any large minority as disqualified rather than merely outvoted. That is value pluralism recast as an engineering constraint rather than a philosophical preference — closer to a robustness requirement than to a moral theory. And it is why I will resist the temptation, which the old draft did not, to print a table of invented population shares. The distribution is the thing to estimate, not to assume; a made-up table with tidy percentages would be exactly the confident fiction this book is against, dressed up as a finding.

Two kinds of uncertainty

The forecast then carries two different uncertainties, and they do not reduce to each other. One is aleatoric — the genuine spread of values across people, which no amount of learning erases, because it is not ignorance, it is reality; even a perfect model of a population that will never agree still reports disagreement. The other is epistemic — our uncertainty about what that spread even is, a distribution over distributions that better data and better methods narrow. The two behave differently and demand different responses: epistemic uncertainty is the part you can pay to reduce, with more surveys and longer histories and better calibration, and aleatoric uncertainty is the part you must instead design around, because it will still be there when the study is done. Both have to be quantified, and quantified separately: not "humanity will value X," but a probability that it does, with an interval around that probability, and a clear account of how much of the interval is ignorance and how much is real variety. It is the same discipline Chapter 13 applied to policy costs, turned on values instead of dollars.

Reflection is not the same as time

Here is the deepest problem, and the one the empirical machinery cannot dissolve. Temporal change is not reflection. The whole appeal of forecasting values is the hope that where attitudes are heading is somehow better than where they are — that the forecast points not just forward

in time but forward in judgment. Nothing in the data licenses that hope. That support for same-sex marriage rose from 27% to roughly seven in ten over a generation shows that time passed and attitudes moved; it does not show that anyone reasoned more carefully. The engines of value change — generational replacement, media exposure, information cascades, tribal signaling — are sociological, not epistemic, and a model that predicts them is predicting a social process, not a deliberative one. Some shifts plainly track moral progress: abolition, the widening of the franchise. Others may be drift with no progress in them at all. Still others might be backlash to perceived overreach, which is one live reading of the 2024 reversal — and if it is, then the model that "should" have predicted continued acceptance would have been not just wrong but wrong in a morally loaded direction.

The philosophers who have pressed hardest on this offer warnings more than solutions. Rawls distinguished reasonable from unreasonable comprehensive doctrines — not every value system deserves equal weight in political life — but "reasonable" resists being turned into a forecasting rule; you cannot regress on it. Habermas set conditions for undistorted deliberation, equal participation and no coercion, that no historical stretch of attitude change is guaranteed to have met, and mostly did not. Sen and Nussbaum warned about adaptive preferences: values formed under oppression, stable and sincerely held, that it would be perverse to extrapolate forward as though they were considered choices. A person raised to expect little may sincerely want little; a society that has normalized an injustice may show stable, well-measured preferences for it. A forecasting model does not know the difference between a preference that has been reasoned into and one that has been ground into place — both are just points in a time series — and predicting the second one accurately would be an achievement in the service of nothing good. The honest position is that we have no clean account of when temporal change tracks reflection and when it is mere drift. Predicting well is evidence of underlying structure; it is not evidence that the structure is worth honoring. That is a permanent reason for humility, not a bug that a larger model or a longer history removes.

Whose trajectory?

The GSS is American, and value trajectories are not obviously universal — everything in this chapter has been measured on one country's attitudes, which is a narrow base from which to speak about humanity. The World Values Survey, run across roughly a hundred countries since 1981, shows both convergence and its opposite. On some dimensions there is a broad drift toward self-expression values with economic development and generational turnover, in the pattern Ronald Inglehart called post-materialist: as societies grow wealthier and more secure, priorities shift from survival toward autonomy and tolerance. On others — religion, family structure, the norms around sexuality — cultural zones persist, and different starting points produce genuinely different paths rather than the same path at different speeds. Recent years have added pointed reversals in several countries, movements that reject cosmopolitan liberal values outright. Any global forecast would therefore have to model heterogeneity at the level of whole societies, not just individuals, which deepens the problem rather than resolving it. A

projection tuned to one society would misread another, and there is no neutral vantage from which to call one "ahead" and the other "behind" — only different histories producing different distributions, each of which a model would have to learn on its own terms.

A few objections

A few objections are worth stating plainly, because they are the ones a skeptical reader raises first. The first is that this is moral relativism dressed up in statistics. It is not: forecasting where values are heading is not the claim that all values are equally valid, any more than forecasting a hurricane endorses the storm. If the honest projection were that a reflective public would come to reject some cruelty, that would be a prediction, not a shrug. The second objection is that values are too contingent to predict at all — that they turn on accidents no model can see. Perhaps; but that is an empirical question, and the validation step is precisely how one answers it. The 17-variable test says the answer is "partly," and the 2024 miss marks exactly where "partly" runs out. The third objection is the one that should worry anyone who builds this: the same understanding that lets you forecast a value shift could help someone accelerate or suppress it. That is true, and it is true of all social science, and it is not a reason to stop understanding things — but it is the reason the next question is not about method at all.

The governance questions

Suppose, despite all of this, that value forecasting one day worked well enough to be worth consulting. It would immediately become a technology of power, and who holds it would matter as much as how accurate it is. A government could dress paternalism as foresight — "we are enacting what you will come to want, so your objecting now is beside the point." A firm could anticipate and steer preferences rather than merely serve them. A lab could align a system to projected values that happen to coincide with its own interests. None of these is an exotic risk; they are the ordinary consequences of forecasting in a world of unequal power, and the difference between the tool being useful and being dangerous is almost entirely a matter of the arrangements around it. Any serious version of this work has to answer several questions before it earns any authority at all.

Who generates the forecasts? Academic researchers, government agencies, and private companies each bring different incentives and different accountability, and none should hold a monopoly; distributed production under open methods is more trustworthy than any single center, public or private, because it can be checked.

Which conditioning assumptions are on the table? A forecast conditioned on broad deliberation differs from one conditioned on polarized media, and the choice between them is itself a value judgment. The assumptions encode values, and they must be stated where they can be seen and argued with, not buried in a model card.

What is democratic input for? Forecasts should inform deliberation, never stand in for it. Citizens have to be able to examine the method, challenge the assumptions, and reject a forecast-based justification outright — the forecast is evidence entering an argument, not a verdict ending one.

How are disputes resolved? Contested forecasts need appeals — replication, public comment, independent review — not an oracle's say-so. A number that cannot be contested is not a forecast; it is an assertion wearing a forecast's clothes.

What happens when the forecast and today's expressed values disagree? That is the hardest question, and honest answers to it stay uncertain. If a projection says a reflective public would reject something the present public endorses, which one governs? The 2024 reversal is the cautionary tale: a 2020 projection of continued acceptance, used to justify anything, would have been overconfident within four years. A governance system for this has to be built to absorb surprise and revise — which is another way of saying it has to be built like the scoreboard in Chapter 13, not like a prophet.

What this has to do with alignment

The connection to AI alignment, which is where a version of this argument usually begins, belongs at the end and in one restrained form. If value forecasting ever earned admission on this book's own terms — calibrated, validated across variables and countries and horizons, adversarially tested — it would bear on how AI systems weigh contested human values: as one input, evidence about where considered preferences might sit, never an authority over them. Today it has earned no such thing. It is a research question with one suggestive backtest, one humbling miss, and one pre-registered replication in which nothing beat persistence — and the fuller argument, what admission would require and what it would mean, belongs where a research program can be laid out honestly, not in a book about verified systems.

The institutional shape, if it ever cohered, is already visible in outline. Value forecasting is at bottom a forecasting problem — a probability distribution over what people will come to value, scored against what actually happens — which is exactly the kind of claim the Thesis Institute exists to post with an interval and grade when reality arrives (Chapter 13). None of it has resolved; there is no track record here to point to, only a method and a discipline for eventually being wrong in public. One could imagine chaining the book's tools into a single machine — a forecast value distribution feeding a democratic simulation feeding a policy model feeding measured welfare — but that is an unbuilt composition of mostly toy parts, and drawing the diagram is not the same as building the machine.

It is worth returning, at the end, to where this chapter began. Everything before this part of the book earned its place by terminating in a check against the world — a forecast grade, an oracle parity, a statute quoted to the letter. This chapter did not, and the reader is owed that distinction kept sharp rather than blurred at the finish. The value-forecasting work is a genuine experiment with a genuine result and a genuine, disciplining failure. It is also the

one place in the book where I am reasoning past the edge of what can yet be verified, and the correct posture toward it is interest without belief.

What we can honestly say from inside that posture is smaller and sturdier than the grand version. We can ask people what they value, model the patterns, project them forward with calibrated uncertainty, and stay loud about how uncertain the projection is. Surveys capture current states, not trajectories; forecasting is one way to reach for the trajectory, and the 2024 miss is a standing reminder of how easily that reach exceeds its grasp. We cannot know what humanity would choose after reflection, and we should distrust anyone who says they can. We might, someday, forecast it — badly at first, a little better with each miss we are forced to admit in public. That is the most this method can promise, and it is worth doing only if the promising stays that honest.

The fork in the road, though, is already here. Whatever the horizon holds for forecasting values, the choice the next chapter returns to — open or closed, graded or oracular, plural or singular — does not wait on any of it. It is being made now, with the tools already built.

Chapter 17: Society in silico

We return to where we began: Engerraund Serac in his control room, watching Rehoboam's predictions cascade across the displays. Individual lives reduced to trajectories. A society optimized to one man's definition of optimal. "I don't predict the future," he says. "I create it." That is one vision of society in silico.

This book has traced another.

Two paths

Both visions start from the same recognition: that complex social systems can be modeled computationally—that policies can be simulated before they are enacted, that a population can be represented as millions of synthetic households, that the consequences of a choice can be estimated before anyone has to live them. They diverge on everything that comes after.

Serac's path is closed. The model is his alone; the predictions are hidden from the people living inside them; the objective function is singular, chosen by one actor and imposed on the rest. Uncertainty is suppressed, because Rehoboam speaks in certainties. Power concentrates, because those without access to the model are its subjects rather than its authors.

The open path inverts each of these. The model is public, and anyone can read the code. The predictions are queryable, so a person can ask what a policy does to their own household. The objectives are contested—many stakeholders, many goals, no single optimizer. Uncertainty is quantified rather than hidden, reported as intervals instead of false precision. And the capacity to analyze becomes public infrastructure rather than private advantage. The difference is not

that one path uses models and the other doesn't; both do. The difference is who can see them, question them, and correct them. Every chapter of this book has been about building the second path.

What we've built

Guy Orcutt imagined simulating the economy household by household in 1957. For half a century, that vision lived almost entirely inside government agencies—the Congressional Budget Office, the Joint Committee on Taxation, Treasury—available to the powerful and invisible to the public. Rebuilding it in the open took faster computers, better and more accessible data, and code that anyone could inspect and run. Most of that rebuild is no longer a plan. It shipped.

That sentence is worth pausing on, because it is also a fact about this book. Chapters drafted a year ago as descriptions of what someone would need to build now read as descriptions of what got built—the future tense overtaken by the past tense between one draft and the next. That is not a tidy way to write a book. It is, however, the most direct evidence it can offer for its own claim about the pace of the agent era: the distance from aspiration to working infrastructure collapsed while the manuscript sat open. A book about the pace of change that was itself outrun by it is an awkward artifact, but an honest one—and it argues, better than any forecast could, that the window in which this infrastructure gets built open or built closed is not decades away. It is now.

The open engine. PolicyEngine encodes thousands of federal and state tax-benefit rules and shows them from two angles: the household view—*how does this policy affect my family?*—and the society view—*who gains and who loses across the population?* Its microdata is calibrated to over 9,000 administrative totals; validation partnerships with the National Bureau of Economic Research and the Federal Reserve Bank of Atlanta cross-check its calculations against independent implementations; and benefit screeners such as MyFriendBen run families across four states on the same underlying engine.

The rules commons, at scale. The Axiom Foundation, launching on July 28, 2026, has encoded roughly 3,000 US rule files spanning federal law and 28 states, with companion repositories for the United Kingdom, Canada, New Zealand, and Belgium—the stated mission being to encode all public policy, from federal statute down to a single London borough's council-tax reduction scheme. In one week it encoded five African tax-benefit systems and validated them against UNU-WIDER's SOUTHMOD models, reporting its findings back to the models' authors. Every monetary amount in an encoding is traced to a quoted excerpt from its governing source; and wherever encodings were compared against a reference model, the mismatches were 100 percent explained. This is the single strongest piece of evidence for the book's thesis, and it happened while the book was being written.

The data commons. populace assembles calibrated population data—synthetic households, tuned to administrative totals, that let the model represent anyone in the country rather than

only the survey respondents who happened to be sampled. It is the successor to the enhanced survey files the earlier chapters described, and the answer to the permanent asymmetry those chapters diagnosed: outsiders were never handed the government's confidential records, so the open stack synthesizes and calibrates its own.

The scoreboard. The Thesis Institute publishes forecasts of government metrics—the first print of an unemployment number, a Medicaid call-center wait time under a policy that may or may not take effect—each carrying an explicit interval, each to be graded when the official figure lands. Its honest status belongs in the record: the scoreboard is live, the grades arrive with reality, and not one forecast has resolved yet.

The measured boundary. PolicyBench puts today's best language models on a public board and scores them on complete household tax-and-benefit calculations. As of mid-2026, the best model still gets roughly one in six of them wrong by more than a dollar; most models miss a quarter or more; and the errors are not rounding but family-level—a wrong Medicaid eligibility, a wrong SNAP amount. That gap is why the deterministic layer has to exist at all.

These are not five separate projects so much as one structure pulled apart along its natural seams. PolicyEngine keeps its name and its users; underneath, it is being recomposed into two open layers—Axiom, which answers what the law says, and populace, which answers who the people are—with the Thesis Institute as a distinct, third pole answering what will happen. An axiom is an accepted truth; a thesis is a proposition to be tested; and the two are verified in entirely different ways—rules against statute and against reference calculators, forecasts against reality when the number finally arrives. The organization came to mirror the questions it was built to answer.

What the decomposition buys is an accountability the closed stack never had. When rules, data, and prediction are separate layers, each can be checked on its own terms, and an estimate need no longer be a single number issued and forgotten. In principle it becomes a forecast with a published interval and, eventually, a grade: the office that produced it can be told, when the official figure lands, how close it came. The Congressional Budget Office, the Tax Policy Center, and Europe's reference models all run without that loop. Building it is the reason to split the stack this way—even though, as the next section admits, the loop has not yet closed on a single resolved forecast.

What we haven't built

Honesty requires the other ledger. Uncertainty is still mostly unquantified: a microsimulation answers *this reform costs fifty billion dollars* when it should answer *fifty billion, give or take, and here is the range and why.* The scoreboard is the institutional response to exactly this gap—it forces every forecast to carry its interval—but inside the everyday tools, most numbers still arrive as point estimates dressed as precision. The methods to fix it exist; wiring them through production is real work that is not finished.

Adoption is still early. The tools exist, but most policy debates run without them. When Congress passed the One Big Beautiful Bill Act in July 2025 and extended the 2017 tax cuts, the public argument was shaped by CBO scores and think-tank estimates—not by citizens running their own analyses. Scores mattered enormously; they moved a live, enacted law worth trillions of dollars. But they were institutional scores, and the shift from *available* to *expected* is the kind of cultural transition that took weather forecasting decades to complete.

The prediction pole has no track record. This is worth stating flatly, because it is exactly the thing a less careful book would blur: the Thesis Institute has not resolved a single forecast. Its scoreboard is a set of promises with intervals attached, not a record of hits. Until reality grades them, they are predictions, not evidence—and the discipline of saying so, out loud, is the entire reason to build a scoreboard instead of a megaphone.

The AI layer is only half-arrived. The rules and the data now exist at production scale, and an AI assistant can call them today—that part is real and shipped. What has not happened is that calling them becomes the default. Ask most assistants about tax policy right now and the answer still comes from training data rather than a calculation engine. The infrastructure that would make tool-calling automatic is built; the habit of reaching for it is not.

So this book describes as much aspiration as achievement. The rules layer is further up the mountain than the last draft dared claim; the prediction pole is barely off the trailhead. We are partway up, and honest about the altitude.

Why it matters

If a society cannot reason about itself, it cannot govern itself. And the alternative to open simulation is not human judgment uncorrupted by models—it is the set of failure modes that already constitute the default.

Agencies make black-box decisions with proprietary tools: the Joint Committee on Taxation produces the revenue estimates Congress relies on using models Congress cannot inspect, and you are asked to trust outputs you cannot verify. Debate runs on vibes: politicians assert, pundits assert louder, and numbers float past without sources or audit trails. AI systems answer policy questions from training data, confidently and often wrongly—the family-level errors PolicyBench measures, shipped at the scale of every chatbot. And power concentrates among those with compute and data: a hedge fund can model the distributional impact of a tax reform in minutes while a community organization cannot, so the fund's version of events arrives first and with numbers attached, and sets the terms of the argument regardless of whether it is right.

These are not hypothetical. They describe how policy is analyzed today. Open simulation answers each one in kind—auditable code instead of black boxes, reproducible results instead of vibes, tools an AI can call instead of fiction it invents, public infrastructure instead of private advantage. None of these substitutions is automatic, and each can be done badly—an open

model can still encode a bad assumption, and transparency does not make a mistake correct. What it makes a mistake is findable. It is not guaranteed to help. But the counterfactual is worse.

The deepest version of the case is democratic. Democracy asks people to vote on policies without knowing their effects, to debate taxes without calculating impacts, to judge officials without auditing their claims—and for most of its history this was not ignorance by choice but ignorance by limitation. The models that could answer those questions sat inside institutions; expert analysis was expensive and late, when it came at all. Citizens had opinions and experts had models, and the two rarely met. Open microsimulation does not settle the value questions—it can't, and shouldn't—but it lets anyone bring real numbers to them. And if enough people can close that gap, the dynamics shift: officials can be held to calculable claims, platforms stress-tested against distributional analysis, and the signal-to-noise ratio of the whole deliberation improves. That is a modest mechanism, not a revolution—but modest mechanisms, compounded across millions of choices, are how a public slowly changes what it will accept as an argument.

What that buys, concretely, is a shared factual floor. Picture the next time a candidate releases a tax plan. Instead of two campaigns trading unsourced numbers, independent analysis appears within hours—not from a single think tank but from many people running the same open model with different assumptions. A voter enters their own household and sees the specific dollar effect. A journalist checks a distributional claim against the actual calculation. The disagreement that remains is the real one, about values—how much redistribution is fair, how to weigh growth against security—argued from a common set of facts rather than dueling assertions. That is not utopia. Many will never use the tools; many will distrust them. But the option to know now exists, and it did not before.

The AI argument

There is a second reason the timing is not incidental. As AI systems grow more capable, more people will meet policy through them—asking an assistant what a child tax credit expansion would do, or how a carbon tax would land on their family, or which candidate's health plan helps them more. Such a system can answer in one of two ways. It can synthesize plausible text from its training data, inventing eligibility rules and hallucinating statistics—which is what happens today, and why the best frontier models still complete fewer than a third of real tax returns correctly and still miss household calculations at the rate PolicyBench records. Or it can call a reliable tool: look the parameters up in an open encoding, run the calculation deterministically, and return a result someone can audit.

The infrastructure for the second path is what the first half of this chapter described, and its value compounds in a particular way. A microsimulation model that takes ten minutes to learn serves one researcher; the same model, reachable by an AI agent, serves anyone who can ask a question in plain language. Every hour spent encoding a rule accurately, tracing it to its source,

and validating it against an oracle pays off in proportion to how many agents eventually call it—and that number is climbing. The case is not that AI will replace analysis. It is that AI will do the asking, and the tools it reaches for should be accurate, transparent, and maintained. And reliability here is a mechanism, not a promise: when the benchmark's own explanatory notes were once caught inventing the derivations behind their scores, the fix was not better wording but mechanical grounding—regenerate every explanation from the engine's internals, and reject any that cannot be traced. Even the audit layer needs ground truth.

This is not a lone bet. The same architecture—an AI front end over a deterministic back end—is turning up wherever institutions have tried to let people ask real questions: the IRS has put AI agents to work summarizing cases and searching guidance, and California's Poppy assistant helps state employees navigate dense policy catalogs. None of these is PolicyEngine, and that is the point—the pattern is converging because it is the one that works. The alternative is confident fiction about policy, delivered at the scale of every assistant that answers: more access to information that is less reliable, which is not progress.

Closing the loop

In *Westworld*, Serac's system fails. Rehoboam cannot handle Dolores and the other hosts—beings who do not fit its model, whose choices it cannot predict—and the closed system shatters on contact with genuine novelty. The fiction was too tidy, but the lesson holds.

Closed systems are brittle. They optimize for their own assumptions, and when the world shifts—as it always does—they break in ways their designers never anticipated. The government microsimulation monopoly of the 1980s was a mild version of the same fragility: models that could not be challenged could not be corrected. When the Joint Committee on Taxation produced an estimate, there was no mechanism for outside correction—you could disagree, but you could not demonstrate the error. The open-source movement changed this not by producing better models, but by making models *improvable*.

That is the through-line of everything this book has traced. Orcutt's 1957 paper proposed the idea; government agencies realized it behind closed doors; think tanks broke the monopoly; open-source projects made the tools public; and agents are now making them buildable at a pace that turned last year's aspiration into this year's field report. Each step widened the circle of people who could take part in the argument about what policy does, and to whom. An open system is never finished. But it is also never frozen: it exposes its assumptions, invites correction, and adapts as understanding improves. And openness here is machinery, not merely exposure: the same encodings anyone can read are held by checks that let coverage only rise and unexplained gaps only fall, so the worry that open models quietly rot has an operational answer rather than a hopeful one.

An invitation

So the fork is real, and it is not science fiction. Both paths are open right now, as live institutional choices. One rebuilds the analytic machinery of government behind glass—closed, singular, speaking in certainties. The other rebuilds it in daylight—inspectable, plural, and graded against reality when reality arrives. The difference between them is not the sophistication of the model. Serac's was sophisticated. The difference is whether anyone outside the room can check it, correct it, and build on it.

Which is why the invitation at the end of this book is not a sales pitch. The code is public; the rules are open; the forecasts will be scored where everyone can see them. Anyone can read the encodings, run the models against their own questions, file the bug they find—or reject the premise entirely, and argue that these tools entrench technocracy rather than democratize it, that what is measurable will crowd out what matters. That argument, too, is better conducted in the open, against specifics that can be examined, than as assertion in the dark. The work continues in public, because public is the only place it can be corrected.

What these tools finally do is narrow, and it is enough. They cannot tell a society what to value, guarantee that insight is used wisely, or stop a bad-faith actor from gaming them. What they can do is make the invisible visible—the distributional effects buried in a bill's fine print, the uncertainty a point estimate papers over, the benefit cliff that traps a family, the marginal rate that quietly punishes a raise, the interaction between programs that no single-program analysis reveals. Making these visible does not settle politics. It informs it. A voter who can see that a plan would raise their family's income by twelve hundred dollars may still vote the other way, for reasons of value or identity or loyalty that run deeper than arithmetic—and that is their right. But they can choose with open eyes rather than closed.

Guy Orcutt died in 2006, half a century after the paper almost nobody read. He lived to see microsimulation become institutional infrastructure, but not to see it become public infrastructure—the open models, the web tools, the agents that can now build and check the rules one household at a time all came after him. His core insight was per-unit—model society one household at a time, and a picture emerges that no aggregate equation can capture. What the agent era added is the other half of that idea: per-unit verifiability, the checking of each household's answer against a rule, an oracle, or a real determination. That is the discipline the whole stack now rests on. The arc from his idea to here bends toward openness: not inevitably, not smoothly, but detectably. Government monopolies gave way to competition, proprietary models to open code, expert-only interfaces to tools anyone can run. The next turn is happening now, as AI becomes the interface through which millions encounter what policy does—and whether that expands access or expands misinformation depends entirely on whether the systems doing the answering call tools that can be checked.

The machinery can simulate the futures on offer; only the public it serves can decide which one is worth building. Society in silico is not a destination. It's a method.